

UnityPatcher (v1.0.7)

by Ambigram



	Вступление.....	3
	Решение распространенных проблем.....	4
	Как запускать консольные команды.....	6
	Как узнать версию программы.....	7
	Команды и требования.....	9
	Список доступных команд.....	9
	Команда unpack.....	10
	Команда pack.....	12
	Команда search.....	14
	Дополнительные требования.....	15
	Пример команд.....	16
	Основы основ.....	17
	Список программ для моддинга.....	17
	Где получить помощь с моддингом.....	19
	Где опубликовать свой русификатор.....	21
	Типы юнити файлов.....	22
	Как узнать версию движка, на которой создана игра.....	23
	Как скачать любую версию юнити движка.....	25
	Как узнать имена скриптов для экспорта MonoBehaviour.....	25
	Как использовать UABEA.....	26
	Как искать текст в бинарниках (TotalCommander).....	38
	Всё про SDF.....	40
	Unity 4-5: создание SDF.....	40
	Unity 2017+: создание SDF и автоматическая адаптация.....	45

Unity 2017+: замена через UABEA.....	50
Возможные проблемы и их решение.....	50
Общая информация по созданию SDF.....	54
Как вычислить SDF шрифт.....	56
Плагин TextMeshPro.....	58
Правка SDF шрифта (@TMP_FontAsset).....	66
Правка SDF материала.....	66
Правка MonoBehaviour текста (@TextMeshPro).....	67
Хитрости.....	68
 Аудио и видео замена.....	68
Fmod bank – распаковка/пересборка.....	68
AudioClip/VideoClip: запаковка через UABEA + автоконвертация.....	71
VideoClip: решение проблем замены.....	74
VideoClip: команды ffmpeg для перекодировки видео.....	76
 Плагины VerInEx.....	77
XUnity.AutoTranslator – автоперевод юнити игр.....	77
UnityExplorer – изучение объектов запущенной игры.....	81
TMPPro_Translator – перевод строк TMPPro.....	89
 Работа с форматами и решение проблем.....	91
UABEA – решение ошибки “Asset failed to deserialize”.....	91
Решение ошибок “Stripped Unity version” и “version 0.0.0”.....	92
Решение бесконечной загрузки и непрогружающихся ассетов (патчинг catalog.json)..	93
Исправление обрезанной текстуры.....	95
Тьюториал по поиску текста.....	99
Совместимость патчера с AssetStudioGUI.....	104
Шифрованные бандлы (UnityCN).....	105
Nintendo Switch игры – распаковка и патчинг.....	107
Font: замена и исправление размера шрифта.....	107
DLL файлы: редактирование и перевод (dnSpy).....	109
Дампинг MonoBehaviour.....	113
Дампинг объектов другого типа (не MonoBehaviour).....	116
Титры.....	117



Вступление

UnityPatcher, или просто **Patcher** – консольная программа для замены файлов в юнити архивах.

[Готовые батники](#)

[Дополнительные скрипты и утилиты](#)

[Титры](#)

Как использовать: поместить в папку игры и запустить. Для патчинга и поиска строк понадобится небольшая настройка батника, а именно указание источника патч файлов/текстовика с поисковыми строками.

Особенности:

- Поддержка импорта/экспорта для следующих типов: **Texture2D**, **TextAsset**, **AudioClip**, **VideoClip**, **MonoBehaviour** и другие.
- Извлечение дампов **MonoBehaviour** до юнити примерно ~2021 версии.
- Модификация бандлов без необходимости в распаковке.
- Запаковка со сжатием, которое было в оригинале, и воссоздание пути.
- Извлечение в конвертированном виде/raw/json дампа.
- Возможность импорта конвертированного файла + дампа. Импорт произойдет в следующем порядке: сперва дампа, а потом текстура.

Самое главное – всё делается в пару кликов, а команды очень простые. Также нет привязки к папкам, потому что вся нужная информация о экспортированных ассетах хранится в именах файлов. А значит, вы можете сгруппировать их как угодно.

ВАЖНО:

- Вы должны паковать только отредактированные файлы.
- Программа ориентирована на юнити игры, сделанные для Windows. Работоспособность с другими платформами не гарантируется.

- Если игровые файлы довольно объемные, то они долго будут загружаться. UABEA меньше нагружает оперативку и это лучший вариант для редактирования больших юнити файлов.
- То же самое насчет data.unity3d (особенно весом более 5 Гб) – не рекомендуется перепакowyвать патчером, поскольку в разжатом виде вес может вырасти в разы. У вас вероятно не хватит оперативной памяти, а сама загрузка этого файла будет долгой и приведет к зависаниям компьютера. Лучше либо распаковать, а сам data.unity3d удалить, либо править через UABEA/UABEANext. Но можно также комбинировать: через UABEA делать запаковку/распаковку юнити-архивов, а патчером их модифицировать.
- В папке игры не должно быть файлов из разных игр или с одинаковыми именами, иначе запаковка пройдет некорректно. Вы можете воспользоваться опцией `--blacklist` для добавления определенных папок и путей в черный список. Пример: `--blacklist "DLC" "Update"`.

Решение распространенных проблем

- Если у вас не очень мощный компьютер и вы испытываете низкую производительность при использовании программы, возможно у вас открыто много всего. Вы можете уменьшить количество потоков опцией `--threads` и посмотреть, улучшится ли это ситуацию. (По умолчанию импорт – однопоточный, экспорт – многопоточный)
- Проблемы с загрузкой файлов при использовании команды `pack ("Can't load file")`: если вы уверены, что игровая папка содержит все файлы и вы по-прежнему сталкиваетесь с данной ошибкой, попробуйте загрузить всю игровую папку опцией `--load_all` (по умолчанию используется частичная подгрузка).

- Не импортируются определенные файлы: проверьте, не включен ли умный патчинг. Попробуйте отключить его и повторите снова. Проверьте, не включены ли фильтры.
- Ошибка "**stripped version**": разработчики вырезали версию движка из заголовков, поэтому программа не может узнать её автоматически. Укажите версию движка вручную. Пример: `--fallback_version "2020.3.47f1"`.
- [Решение ошибок "Stripped Unity version" и "version 0.0.0"](#)
- **MonoBehaviour** может потребовать дополнительные зависимости, например **globalgamemangers**. Что делать при возникновении ошибки "**Failed to read typetree**":
 - Прокрутите лог выше и убедитесь в отсутствии строк "**Typetree was not generated because Managed was not in the game folder**". Если ваша игра не содержит папку **Managed**, воспользуйтесь **Il2CppDumper** для генерации фиктивных библиотек при помощи файлов **global-metadata.dat** и **GameAssembly.dll**. Полученную папку переименуйте в **Managed** и поместите в **Data** папку вашей игры.
 - Возможно отсутствуют необходимые зависимости. Убедитесь, что в логе нет строк наподобие "**Can't load dependency**". Недостающие файлы надо поместить в Data папку игры. "**Can't load possible dependency**" — это просто предположительные зависимости, не факт, что они нужны на самом деле.
 - Подробнее про **MonoBehaviour**: [Дампинг MonoBehaviour](#)
- Для экспорта аудио и видео тоже понадобятся зависимости (файлы с расширением **.resource**). О недостатке этих файлов будет говорить ошибка "**Failed to parse object: can't load ... file**".

Как запускать консольные команды

Ну, вдруг кто-то не знает этого. 🧑

Способ №1. Запуск программы через консоль вручную из папки с программой:

1. Откройте проводник и перейдите в папку, где находится файл `Patcher.exe`.
2. В адресной строке проводника введите команду `cmd` и нажмите **Enter**. Это откроет **командную строку** в текущей директории.
3. В консоли введите имя программы и следом команду, например:
`Patcher unpack -i "MyGame" --text`

Способ №2. Запуск программы из любой папки через добавление пути в переменные среды:

1. Нажмите **Win + X** и выберите **Система** или найдите в Пуске **Этот компьютер** → **Свойства**.
2. Нажмите **Дополнительные параметры системы**.
3. Во вкладке **Дополнительно** нажмите **Переменные среды**.
4. В нижнем разделе **Системные переменные** найдите переменную `Path` и нажмите **Изменить**.
5. Нажмите **Создать** и введите полный путь к папке, где находится `Patcher.exe` (например, `C:\path\to\UnityPatcher\`).
6. Нажмите **ОК** для сохранения.
7. Теперь, когда путь к программе добавлен в `Path`, вы можете запускать её из любой директории, просто введя имя в консоли.

Способ №3. Запуск через батники:

Этот способ полезен для задач, которые вы планируете выполнять многократно, либо если вы хотите запускать команду с помощью простого открытия файла.

1. Откройте **Блокнот**.
2. Введите следующую команду:

```
@echo off
```

```
"C:\path\to\UnityPatcher\Patcher.exe" command  
"argument" --option1 --option2  
pause
```

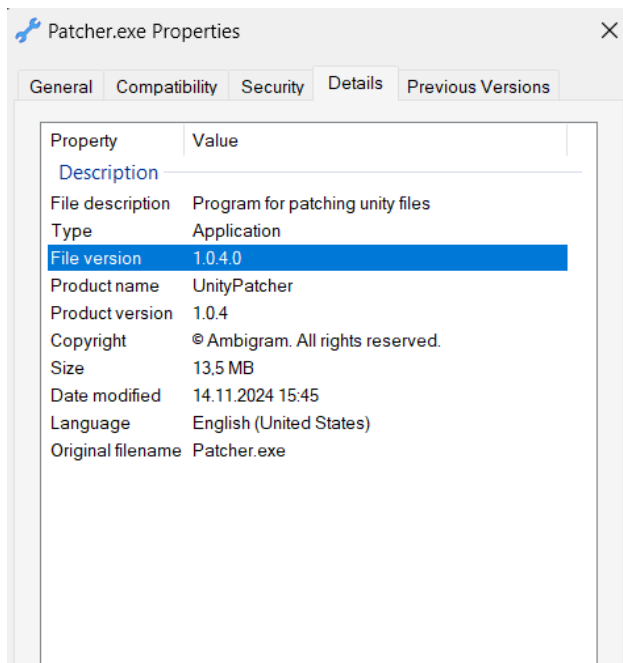
Убедитесь, что путь к программе указан правильный, или просто **Patcher**, если она была добавлена в **PATH**. Впишите нужные вам настройки.

Команда **pause** нужна, чтобы окно консоли не закрывалось сразу после завершения работы программы, и вы могли видеть результаты выполнения.

3. Сохраните файл как **run_program.bat**. Выберите тип файла "Все файлы", чтобы сохранить его именно с расширением **.bat**, а не как текстовый документ.
4. Теперь просто дважды кликните по созданному **.bat** файлу для запуска программы.

Как узнать версию программы

1. ПКМ → **Свойства**
2. Перейти на вкладку **Подробности**
3. Версия будет указана в поле **Версия файла**



Команды и требования

Список доступных команд

Программа **Patcher** предоставляет несколько команд для работы с Unity-файлами: распаковка, упаковка и поиск ассетов по текстовому содержимому. Ниже приведены команды и параметры для каждой из задач.

Общие параметры для всех команд:

- **-i, --input_folder <путь к папке Data>**: Путь к игровой папке **Data**. Если не указывать данную опцию, программа попытается найти папку с окончанием **_Data** в текущей директории (если запустили команду из папки игры после добавления патчера в системные переменные **Path**), а в случае не обнаружения появится диалоговое окно для выбора папки вручную.
- **--fallback_version <версия>**: Резервная версия Unity на случай, если не удастся определить автоматически.
- **--cn_key <ключ>**: Ключ для расшифровки AssetBundle (пригодится, если игра была создана на китайском движке Unity).
- **--debug**: Включение режима отладки.
- **--py_typedtree**: Использовать пайтоновский typedtree reader вместо C-расширения, может помочь с решением мертвых зависаний, но работает медленнее и с ошибками.
- **--blacklist**: Игнорировать определенные папки внутри игровой папки. Может помочь, если игра содержит несколько папок Data, что вызовет исключения. Принимаются относительные/полные пути/названия папок. Для указания нескольких штук, перечислите их в кавычках через пробел.

Примеры использования:

```
--blacklist "D:\Library\Game\Game_Data"  
--blacklist "folder1" "folder2"
```

Команда **unpack**

Описание: Извлечение ассетов Unity.

Пример использования:

Patcher unpack <параметры>

Позиционные аргументы: отсутствуют

Дополнительные параметры:

- **-t, --types <список типов>**: Указать типы ассетов для извлечения через пробел, без запятых и кавычек (например, **-t Texture2D Sprite TextAsset**). При указании **SDF** экспортирует материалы, атласы и **MonoBehaviour** дампы каждого найденного SDF шрифта. Либо можно воспользоваться готовыми командами для извлечения некоторых групп ассетов: **--audio**, **--texture**, **--video**, **--text**, **--font**, **--mb** (**MonoBehaviour**).
- **-c, --classes <список классов>**: Указать имена **MonoScript** классов через пробел, для извлечения связанных с ними **MonoBehaviour** ассетов (например, **-c TextMeshProUGUI Text**).
- **--id <список айди>**: Указать через пробел айди для дополнительной фильтрации ассетов. Можно использовать совместно с фильтрами по типам и **Mono** классам.
- **-m, --mode <тип>**: Режим экспорта (**raw**, **dump**, **normal**). По умолчанию используется **normal** — это значит, что программа сперва попытается извлечь ассеты в обычном виде (wav аудио, png картинка...), а если не получится, то в виде дампа (json файла с метаданными). В raw режиме помимо объектов также будет экспортирован их контент (текстуры, аудио, видео) в двоичном виде. Эти файлы можно будет импортировать, но формат должен быть точно как в оригинале!

Примечания:

- raw текстуры не содержат шапку, то есть если это dds dxt, у

неё будет вырезано первые ~128 бит.

- Дамп – это тот же самый raw объект, но только преобразованный в читаемый вид (json/txt).

- **-g, --group <тип>**: Опция, чтобы задать как группировать извлеченные файлы (**none**, **type**, **source**, **source_type**, **type_source**).

Подробное объяснение:

- **none**: никак не группировать, сохранять все файлы в одной папке – **Patcher_Assets/**

- **type**: группировать файлы по типу ассета (используется по умолчанию) – **Patcher_Assets/Texture2D/**

- **source**: группировать файлы по именам родительских архивов, откуда они были извлечены – **Patcher_Assets/resources.assets/**

- **source_type**: группировать по источнику + типу – **Patcher_Assets/resources.assets/Texture2D/**

- **type_source**: группировать по типу + источнику – **Patcher Assets/Texture2D/resources.assets/**

- **--threads <количество>**: Указать, сколько потоков будут выполнять одновременный экспорт. По умолчанию = количеству ядер, т.е. если у вас 8 ядер, будет запущено 8 потоков. Рекомендуется запускать потоков не больше, чем количество ядер, иначе это приведет к ошибкам. Однако, если они и так возникают, уменьшение количества потоков может помочь.
- **--all**: Извлечь абсолютно все ассеты.
Внимание: может потребовать много места на жестком диске, а также привести к непредсказуемым последствиям, таким как зависания и высокое потребление оперативки (прослеживается при экспорте дампов, содержащих множество данных, например, Shader). Предпочтительнее извлекать только нужные типы.
- **-o, --output_folder <папка>**: Папка для извлеченных файлов. По умолчанию – **Patcher_Assets**.

- **--managed <путь>**: Задать путь к папке Managed, если она находится за пределами папки игры.
 - **--once**: Завершить программу сразу по окончании работы, без запроса ввода следующей команды.
-

Команда **pack**

Описание: Упаковка ассетов Unity.

Пример использования:

Patcher pack “папка патча” <параметры>

Внимание: пакуйте только отредактированные файлы. Вы можете поместить их в отдельную папку, структура папки не играет никакой роли.

Позиционные аргументы:

- **папка патча**: Папка с патч файлами, которые вы хотите импортировать.

Дополнительные параметры:

- **-t, --types <список типов>**: Импортировать лишь некоторые типы, указанные через пробел. Либо можете воспользоваться готовыми группами ассетов: **--audio**, **--texture**, **--video**, **--text**, **--font**, **--mb**. Опция **--font** не импортирует SDF шрифты, т.к. это делается отдельно путем импорта **Texture2D** атласа и дампа **Monobehaviour** через отдельные классы.
- **-o, --output_folder <папка>**: Папка для сохранения модифицированных архивов. По умолчанию – **Patcher_Result**.
- **--tex_quality <тип>**: Задать качество сжатия текстур (**best**, **balanced**, **fast**), по умолчанию – лучшее качество. Стоит сказать, что тип сжатия выбирается на основе оригинального: если в оригинале текстура не сжата, то и импортируемая будет

такого же формата. При использовании balanced и fast могут появиться дефекты на текстурах.

- **--tex_mips**: Генерировать мипмапы для текстур.
- **--raw_texture**: Не сжимать текстуры перед запаковкой. Если запаковка происходит в файл с расширением .assets, то размер может значительно увеличиться.
- **--transcode**: Перекодировать видео перед упаковкой, а если формат вашего видео совпадает с оригинальным – просто запаковать. Если вы попытались импортировать **.mp4**, но в игре видите черный экран, значит некорректная кодировка. Перекодировать можно либо этой опцией, либо ffmpeg – в формат оригинала.

Внимание: кодирование может потребовать много ресурсов. Рекомендуется запускать в один поток. Однако лучше всего, если вы перекодируете видео заранее и просто упакуете патчером без опции --transcode. Так вы будете видеть прогресс и сэкономите много времени.

[Команды ffmpeg для перекодировки видео вручную](#) [Проблемы, связанные с заменой видео](#)

- **--transcode_quality <уровень>**: Качество перекодирования видео (**low, medium, high**). По умолчанию – среднее качество.
- **--packer <тип>**: Метод сжатия юнити архивов (**none, original, lz4, lzma**). По умолчанию – сжатие как в оригинале.

LZ4 – слабое сжатие, быстрая распаковка;

LZMA – хорошее сжатие, медленная распаковка.

P.S. Сжатие применимо только к web-файлам и бандлам.

- **--threads <количество>**: Указать, сколько потоков будут выполнять одновременный импорт. По умолчанию – один.

- **--raw_audio**: Не сжимать аудио в **fsb5** перед записью. Имейте в виду, что в таком случае формат импортируемого аудио должен быть точно как в оригинале.
- **--res_append**: Аудио и видео паковать в конец ресурсного файла вместо замены оригинальных данных (добавлено исключительно для отладки).
- **--outsamedir**: Паковать сразу в папку игры.
- **--smart**: Включить режим умной записи. Работает только с опцией **--outsamedir**.

Принцип работы: после первого запуска просто генерируется **json** с хэшами файлов, а уже начиная со второго будет происходить проверка — если какой-то патч файл не был отредактирован с прошлого раза, он не будет импортирован повторно. Это может сэкономить много времени, так как не придется паковать каждый раз всё. Если понадобится, для сброса хэшей удалите файл **hash_data.json** в той папке, где запускали команды.

- **--custom_res <имя ресурса>**: Паковать аудио/видео в отдельный ресурс. Примечание: к бандлам это не применяется.

Более подробное описание и зачем это нужно:

[Запись аудио/видео в отдельный ресурс](#)

- **--load_all**: Использовать загрузку всей папки игры вместо частичной подгрузки, может помочь если вы сталкиваетесь с ошибками “Can't load file”, но замедлит работу и увеличит потребление оперативной памяти.
- **--backup**: Автоматически создавать резервную копию файлов в папке BACKUP перед тем, как сохранять модифицированные файлы. Бэкап создается только один раз для каждого файла.
- **--recreate**: Пересоздавать output папку каждый раз при сохранении файлов, вместо дописи в неё. Не применяется, если указана опция **--outsamedir**.
- **--managed <путь>**: Задать путь к папке Managed, если она находится за пределами папки игры.

Команда **search**

Описание: Поиск текста в файлах Unity.

Пример использования:

Patcher search "строка поиска" <параметры>

Позиционные аргументы:

- **строка поиска:** Строка для поиска или путь к текстовому файлу с множеством строк для поиска.

Дополнительные параметры:

- **--case_sensitive:** Поиск текста с учетом регистра.
- **--entire_search:** Поиск во всех объектах, а не только в **TextAsset** и **MonoBehaviour**.
- **--export <тип>:** Экспорт ассетов, в которых был найден текст, в конвертированном, сыром виде, или в виде дампа (**normal**, **raw**, **dump**). По умолчанию – конвертированный экспорт.
- **-g, --group <тип>:** Способ группировки извлеченных файлов (**none**, **type**, **source**, **source_type**, **type_source**).
- **-o, --output_folder <папка>:** Папка для экспорта найденных ассетов. По умолчанию – **Patcher_Assets**.
- **--log:** Запись в лог-файл информации о найденных ассетах.
- **--once:** Завершить программу сразу по окончании работы, без запроса ввода следующей команды.

Дополнительные требования

- Для операций, связанных с **MonoBehaviour**, обязательно наличие в папке игры файлов **globalgamemangers** и папки **Managed**. У вас также должен быть установлен **.net 6.0** для генерации дампов.

Для указания своего пути к **Managed** (например после создания фиктивных библиотек при помощи **il2CppDumper**) добавить опцию `--managed <путь к папке с библиотеками>`

Подробнее о редактировании **MonoBehaviour**:
[Дампинг MonoBehaviour](#)

- Для импорта видео нужна утилита [ffmpeg](#), загруженная и добавленная в системные переменные сред, в **PATH**.

Пример команд

Извлечь дампы текстур и аудио без группировки в папку dumps:

```
Patcher unpack --texture --audio -g none -m dump -o  
dumps
```

Извлечь все ассеты с айди 10 и 50, указав версию юнити для избежания ошибок “Stripped Unity version”:

```
Patcher unpack --id 10 50 --fallback_version  
"2020.3.5f1"
```

Извлечь аудио, указав путь к игровой папке и игнорируя папку DLC при загрузке файлов:

```
Patcher unpack --audio -i "Path\to\Game_Data"  
--blacklist "DLC"
```

Извлечь MonoBehaviour, которые относятся к классам Text и TextMeshProUGUI, а также типы SomeType и OtherType:

```
Patcher unpack -c Text TextMeshProUGUI -t SomeType  
OtherType
```

Используя умный режим, запаковать сразу в папку игры только текстуры и видео из папки Patch. Задействовать максимум 3 потока:

```
Patcher pack "Patch" --texture --video --smart  
--outsamedir --threads 3
```

Запаковать со сжатием LZMA, аудио и видео паковать в отдельный ресурс с именем "my_res", автоматически делать резервные копии файлов. Результат записать в выходную папку с именем "Result", пересоздавая её при каждой запаковке:

```
Patcher pack "Patch" --packer lzma --custom_res  
"my_res" --backup -o "Result" --recreate
```

Найти объекты, содержащие строку "My name is Madness", учитывая регистр символов. Экспортировать в режиме raw и создать текстовый лог-отчёт.

```
Patcher search "My name is Madness" --case_sensitive  
--export raw --log
```

Найти объекты, содержащие текст какой-либо строки файла VoiceLines.txt. Искать во всех объектах. Экспортировать результат поиска, сгруппировав по имени архива-источника:

```
Patcher search "VoiceLines.txt" --entire_search  
--export -g source
```



Основы основ

Список программ для моддинга



Экспорт/импорт ассетов:

- [UABEA](#) (дополнение: [Audio Plugin](#)) / [UABEANext](#) (новая версия, в разработке) – программа с графическим интерфейсом, удобная для подмены текстур и редактирования свойств через дампы
- [UnityEX](#) – устаревшая, но всё ещё полезная программа, где большинство функций доступно только через командную строку

- [AssetStudioMod](#) от aelurum (только экспорт) – удобный GUI, экспорт моделей, текстур, музыки и т.д. Эта версия исправляет баги и дополняет функционал оригинальной AssetStudio, находится в активной разработке.
- [AssetRipper](#) (только экспорт) – изучение ассетов, преобразование ассетов в проектные файлы Unity
- [Creators Tool Suite](#) ([исходный код](#)) – андроид-приложение на основе AssetsTools.NET для модифицирования юнити ассетов

🎵 Перепаковка FMOD .bank:

- [FenixProFmod](#) – усовершенствованный [Fmod Bank Tools](#)
- [Soundbank generator](#) (официальная утилита, для скачивания которой нужно зарегистрироваться на сайте fmod) — создание и перепаковка банков

🧩 Загрузчики модов:

- [BepInEx](#)
- [MelonLoader](#)
- [Unity Mod Manager](#)

🔌 Плагины:

- [XUnity.AutoTranslator](#) — автоматический перевод текста в игре
- [Unity Explorer](#) — внутриигровой редактор Unity, поддержка прекращена с 2023 года
- [Runtime Unity Editor](#) — внутриигровой редактор Unity
- [TMPro_Translator](#) – перехват и подмена строк в объектах [TextMeshProUGUI](#)

📦 Библиотеки для патчинга и анализа ассетов путем кодирования:

- [UnityPy](#) — python библиотека
- [AssetsTools.NET](#) — .NET библиотека, используется в UABEA

🧬 Декомпиляторы кода:

- [dnSpyEx](#) — редактирование и декомпиляция .NET сборок (.dll файлов)
- [ILSpy](#) — анализ сборок, не позволяет вносить изменения
- [dotPeek](#) – декомпиляция .NET сборок в C# и IL-код

- [IL2CppDumper](#) (или же графическая обертка [IL2CppDumper GUI](#)) – создание фиктивных .NET сборок для чтения **MonoBehaviour** и не только (пригодится, если в директории интересующей вас игры нет папки Managed, но есть GameAssembly.dll / libil2cpp.so, и global-metadata.dat, краткая инструкция: [Дампинг MonoBehaviour](#)) – на момент написания поддерживает игры версий Unity 5.3 - 2022.2.
- [Cpp2IL](#) – наподобие IL2CppDumper, но пытается восстановить исходный код



Полезные утилиты:

- [AddressablesTools](#) от nesrak1 — патчинг catalog.json
- [GameObjectHierarchyTool](#) – извлечение иерархии **GameObject**, импорт в другой .assets/level файл
- [AudioClipAssetReplacer](#) – небольшая утилита для подмены аудио в объектах **AudioClip**. Примечание: не поддерживает импорт в бандлы, требует заранее конвертировать аудио в FSB5.
- [MetaDataStringEditor](#) – программа с графическим интерфейсом для редактирования строк в файле global-metadata.dat
- [Утилиты от dragonzh](#) – в этой теме вы найдете: dll дампер, патчер catalog.json, csv-распаковщик, утилиты для работы с текстом, двоичными файлами, и тому подобное.



Также могут пригодиться:

- Нех-редакторы — HxD (бесплатный), 010 Editor (платный) (для редактирования сырых объектов в формате .obj)
- Редакторы кода — VS Code, Sublime Text, Notepad++ (для редактирования .txt, .json)
- Инструменты сравнения — WinMerge, Meld, Beyond Compare (для сравнения текстовиков и дампов)
- Инструменты поиска по двоичному содержимому, такие как Total Commander
- Движок Unity — для:
 - генерации SDF-шрифтов
 - сборки модов (например, UI или кастомных 3D моделей)

Конечно программ для моддинга гораздо больше, но я постарался привести только самые основные и востребованные.

Где получить помощь с моддингом

- <https://discord.gg/C6txv7M> (англ.) – сервер пайтон модуля UnityPy. Очень хороший актив, могут помочь с перепакровкой ресурсов, дата-майнингом и тому подобным. Чаще всего обсуждают китайские андроид игры, сделанные на юнити.
- <https://discord.gg/hd9VdswWZs> (англ.) – сервер UABEA
- <https://discord.gg/qEzKUq2SZ4> (рус.) – сервер сообщества TDoT, где есть форум переводчиков и канал для обсуждения разбора ресурсов
- [Вскрытие игровых ресурсов - Zone of Games Forum](#) (рус.) – здесь много толковых ребят. Основная проблема форума, что с большой долей вероятности вам никто не ответит, но попробовать всё равно стоит.

Внимание: перед тем как писать сообщение или создавать новую тему на форумах, обязательно попытайтесь разобраться в своей проблеме самостоятельно – никто не любит помогать тем, кто не заинтересован в решении, а лишь ждет готового ответа. Попробуйте погуглить, желательно на английском языке, либо поискать в специальных дискорд серверах (ссылки приведены выше). Также убедитесь в отсутствии тем или сообщений касательно интересующей вас игры или вопроса, чтобы активные участники не отвечали на одни и те же вопросы кучу раз.

Как правильно задать вопрос? Обязательно укажите:

- Название игры
- Вкратце суть своей проблемы
- Версию движка ([Как узнать версию движка, на которой создана игра](#))
- Приложите образец файла игры (по возможности меньшего веса). Не заливайте всю игру, так как это будет считаться

нарушением авторских прав (но допустимо, если вы скидываете ссылку на полную игру в личные сообщения, а не публично).

- Опишите, какими программами вы пытались решить свою проблему, какие выполняли шаги, чтоб можно было это воспроизвести.
- Главное, пытайтесь объяснять понятно.

Пример плохого вопроса:

“Здравствуйте, столкнулся с проблемой при переводе игры: аудио-файлы извлеклись в формате .bytes, и я не знаю, как из них достать звуки. Может есть какой-то инструмент, позволяющий доставать звуки из .bytes?”

(Написано очень поверхностно. Чем извлекали аудио? Как извлекали? Из какого конкретного файла, какой игры, и т.п.? Как выглядит содержимое этого .bytes файла?)

Где опубликовать свой русификатор

Форум **Zone Of Games**:

- Вам нужно зайти в раздел *Русификаторы*
- Используя поисковую строку проверить наличие темы с переводом.

Если её нет, то создать: вкратце описать игру, когда вышла, кто разработчик (просто скопируйте из шапки другой темы, отредактируйте под свою игру) и добавить небольшой абзац текста, содержащий информацию:

- о том, какой контент вы перевели,
- о участниках перевода,
- скриншоты с демонстрацией перевода,
- инструкцию по установке русификатора,
- ваши соцсети для обратной связи,
- и так далее.

Если тема есть: всё то же самое, но вам уже не надо создавать новую тему. Вы заходите в существующую и

добавляете новый комментарий с информацией касательно своего русификатора (то есть авторы, инструкция по установке, и т.д.).

- Через время администратор форума заметит ваш комментарий/тему, добавит тег *“русификатор”*, и опубликует новость в банере *“Последние релизы”*. Это может занять до 3-х дней, поэтому не переживайте, если не получаете ответ сразу. А публикация в банере обычно происходит ещё позже.
- Ну и плюс администратор может создать руководство в стиме. Оно будет содержать короткое описание русификатора, скриншоты, и ссылку на форум **ZOG**.

Главные плюсы публикации переводов на **ZOG**:

- Это самая большая русскоязычная площадка с переводами.
- Другие площадки обычно заимствуют русификаторы с **ZOG** и дальше распространяют у себя.

Другие варианты:

- Создать руководство в **Steam**
- Опубликовать на **PlayGround**
- Выложить на торрент-трекере
- Выложить на бусти, в своих соцсетях

Типы юнити файлов

- **.assets** – содержат информацию о ассетах. К ним могут прилагаться ресурсные файлы.
- **.resource** – содержат сырые данные аудио, видео. Они обязательны, если вы хотите экспортировать эти типы ассетов из **.assets** или импортировать, иначе получите ошибку **"expected str, bytes or os.PathLike object, not NoneType"**.
- **.resS** – тоже ресурсные файлы, содержат данные текстур и меш данные. Они нужны только для экспорта текстур, при

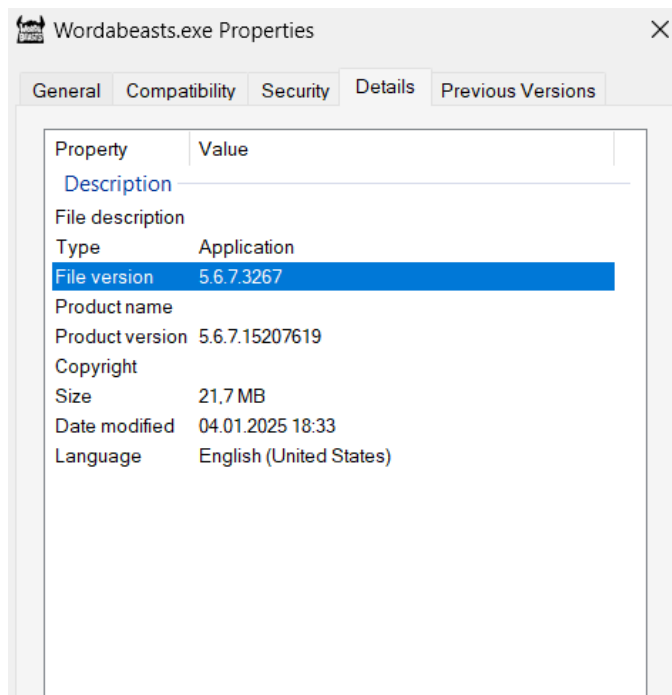
импорте новые данные будут записаны напрямую в `.assets` файл (это называется встроенный ресурс).

- `.bundle` – бандлом называется файл, который содержит множество архивов (`.assets`) и их ресурсов (`.resource`, `.resS`). В очень редких случаях может содержать другие бандлы.
- `AssetBundle` – то же самое, что и бандл, только содержит один архив с ресурсами, и обычно не имеет никакого расширения.
- `data.unity3d` – сжатый архив, куда помещены все файлы: уровни, архивы, ресурсы. Это сделано просто для экономии памяти, и чтоб вместо сотни файлов нужно было скачать один. Если его распаковать в папке `Data` и удалить, то всё будет работать как и раньше.

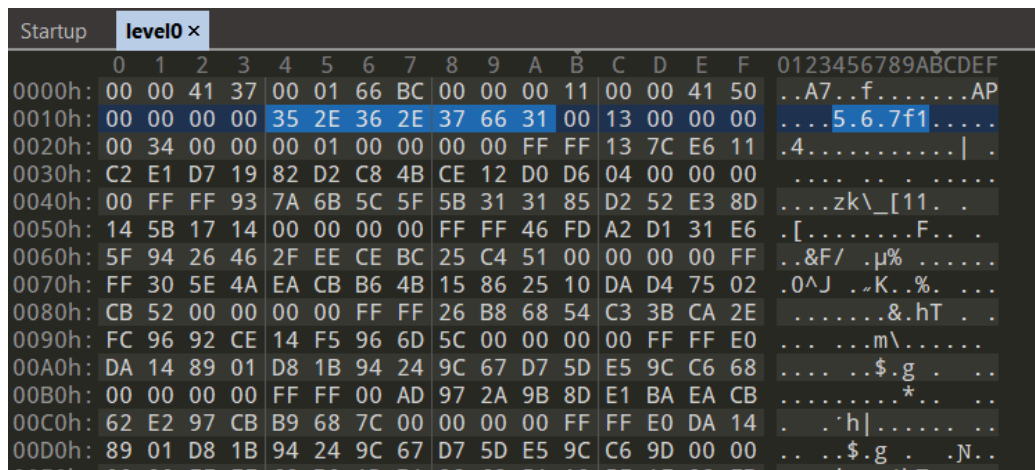
Как узнать версию движка, на которой создана игра

Есть много разных способов. Самый надёжный – открыть свойства исполнительного файла игры и посмотреть в поле **Версия файла**. Важными являются только первые три числа, написанные через

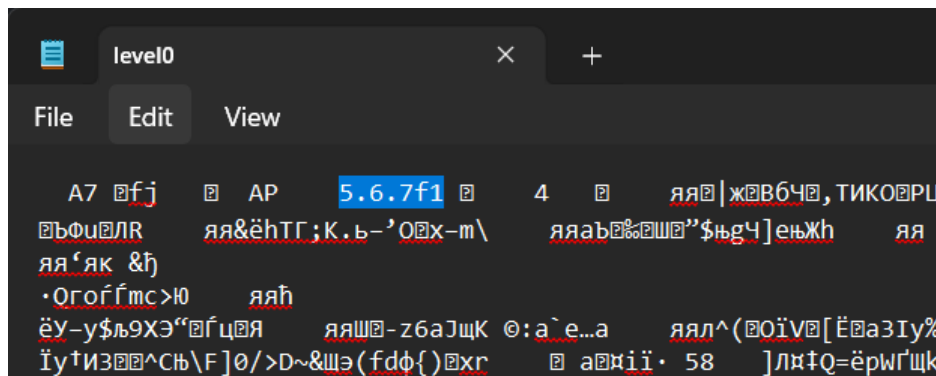
Точку.



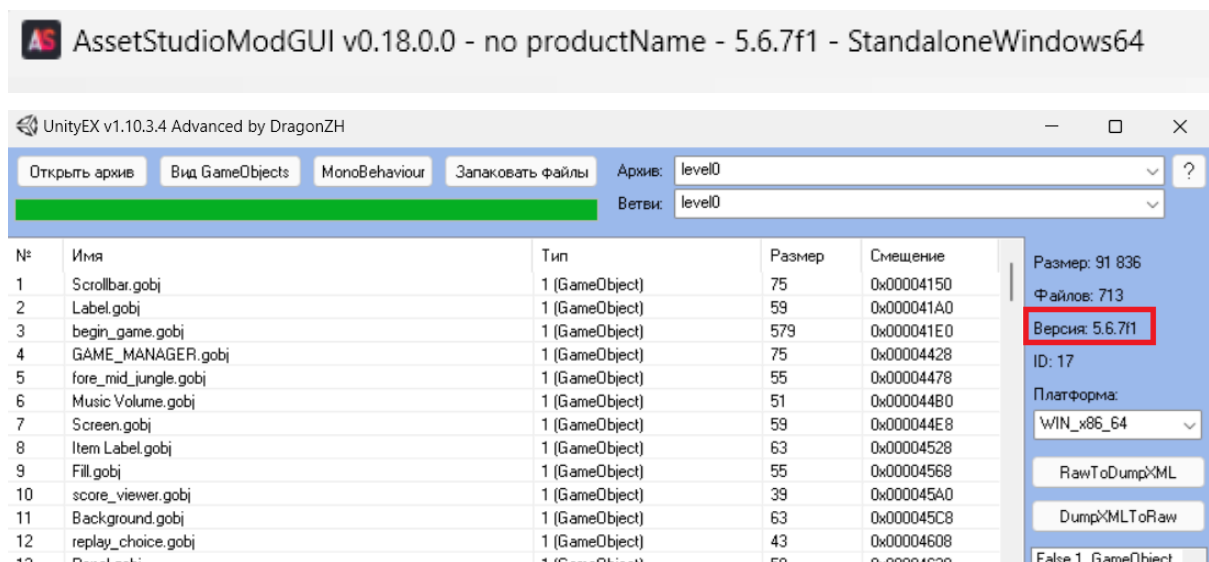
Если у вас есть hex-редактор, вы можете открыть в нём любой юнити файл (.bundle, .assets, level) и проверить, что написано в шапке. По надёжности это второй способ, так как некоторые разработчики вырезают версию движка из заголовков для усложнения жизни моддеров.



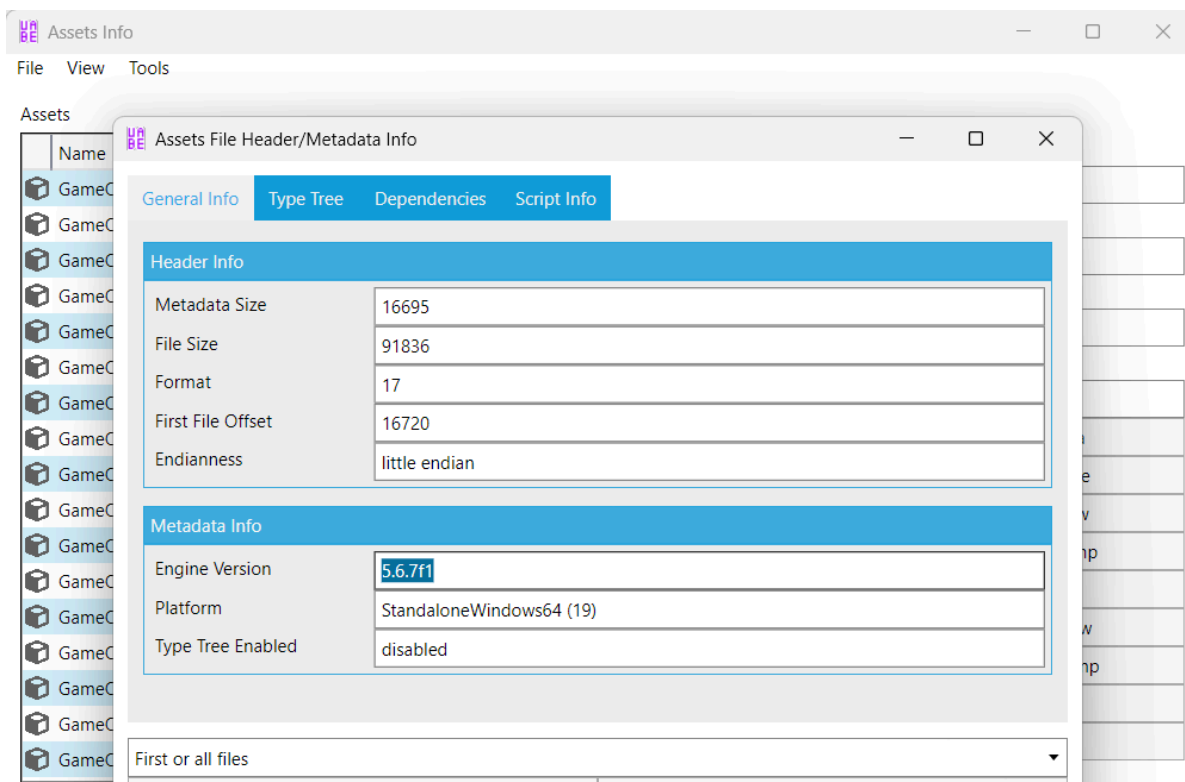
И то же самое, если открыть файл в блокноте:



Посмотреть версию движка (опять же, если она не была вырезана из заголовков) можно и с помощью **AssetStudioGUI/UnityEX**, после открытия там какого-либо юнити архива:



В UABEA через Tools → Info (F4):



Как скачать любую версию юнити движка

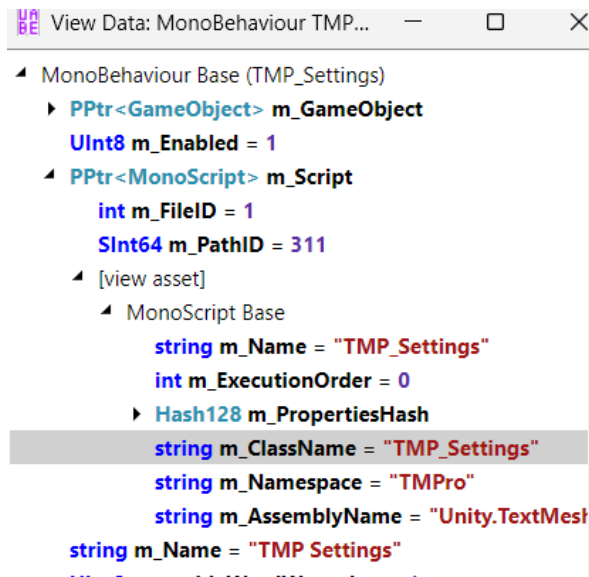
- Перейдите на страницу [Unity download archive](#)
- Найдите интересующую версию и нажмите кликабельную ссылку *See all* в последнем столбце строки.
- Выберите свою операционную систему для загрузки установщика.

Для удобства рекомендую установить [Unity Hub](#) – официальную программу для управления движками и проектами (кажется, для её использования придется зарегистрироваться).

Как узнать имена скриптов для экспорта MonoBehaviour

1. Откройте юнити файлы при помощи [UABEA](#).
2. Найдите нужные **MonoBehaviour**. Воспользуйтесь фильтром (**Ctrl + E**) или поиском (**Ctrl + F**) для ускорения процесса.

3. Нажмите кнопку **View Data** для просмотра свойств.
4. Раскройте ветки **PPtr <MonoScript>** и посмотрите, что написано в строке **m_ClassName**.
5. Запомните название и укажите в опции **--classes** для извлечения патчером.



Как использовать UABEA

Подготовка программы:

[Скачайте UABEA](#), распакуйте в любое место, запустите файл **UABEAvalonia.exe**.

Убедитесь, что установлены все необходимые зависимости, указанные на странице релиза:

Requirements:

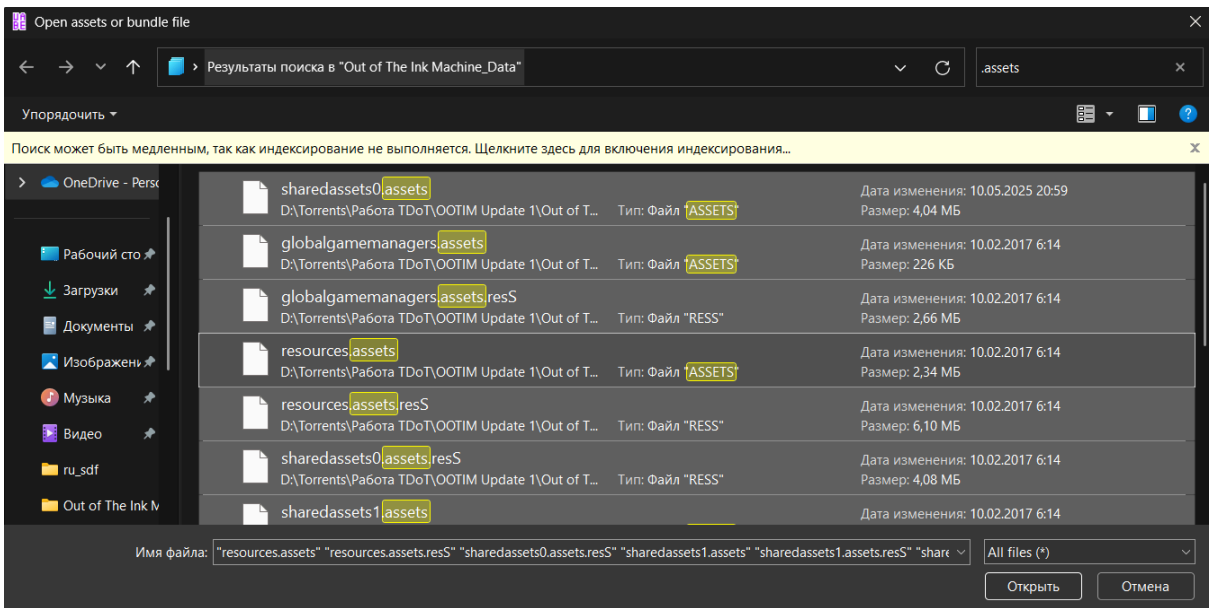
- [.NET Runtime 6.0](#)
- [VS C++ Redistributable \(on Windows\)](#)
- At least glibc 2.29 (on Linux). If you're on a distro not using glibc (e.g. musl), you'll need to compile UABEA yourself.

Открытие юнити-файлов:

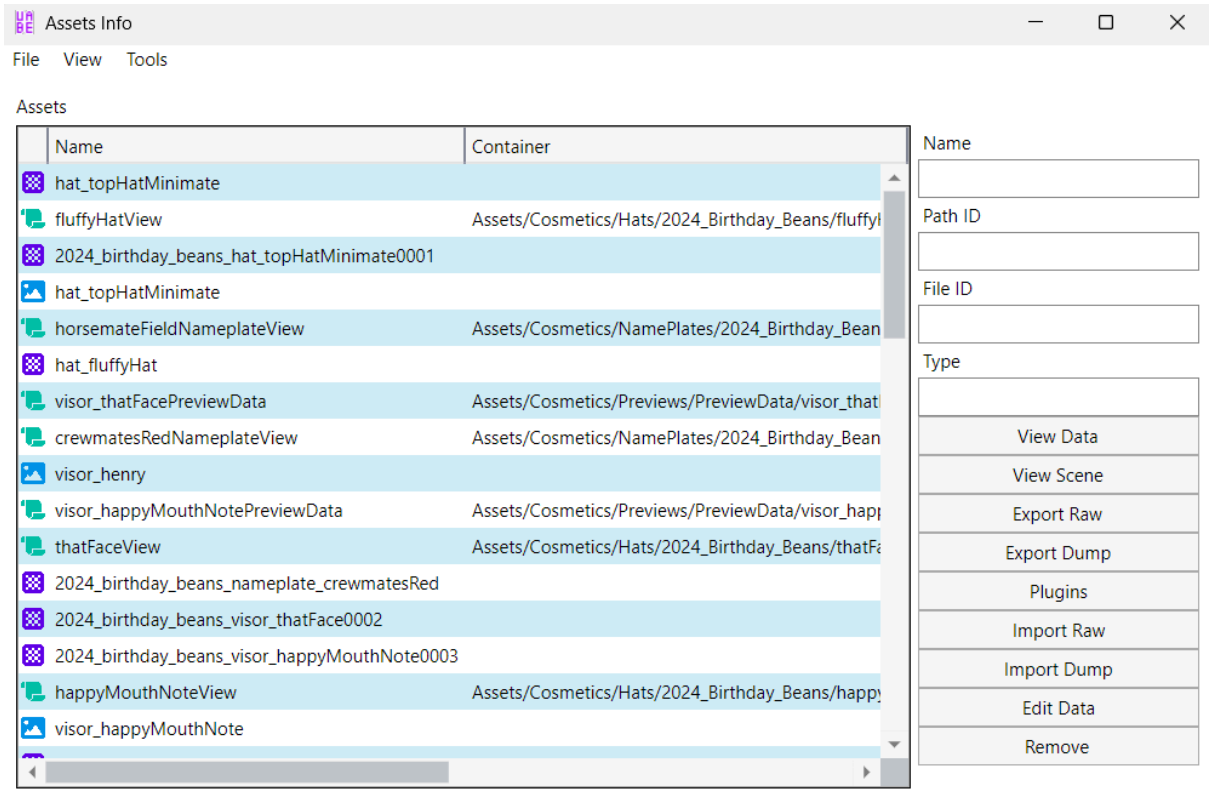
File → Open

Дальше вы переходите в папку **Data** и выбираете любой из следующих файлов: *level**, **.assets*, **.bundle*, *data.unity3d*. Чтобы выделить несколько файлов, нажмите кнопку Ctrl или

воспользуйтесь поисковой строкой Проводника. Не выделяйте все файлы в папке Data!, иначе **UABEA** выдаст ошибку **"This doesn't seem to be an assets file or bundle"**.

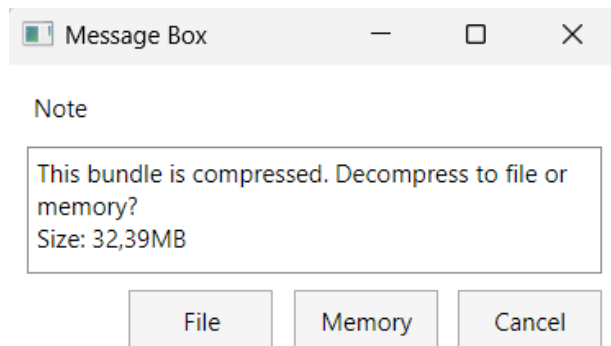


Перед вами появится окно Assets Info, где можно отфильтровать/отсортировать объекты, просмотреть их свойства, отредактировать или экспортировать.

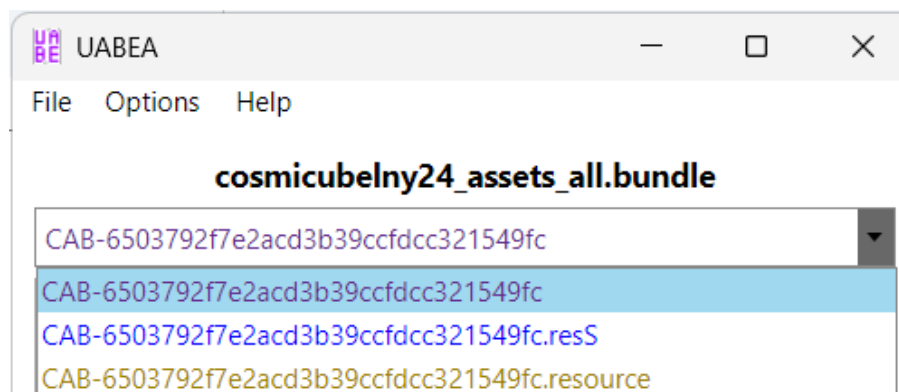
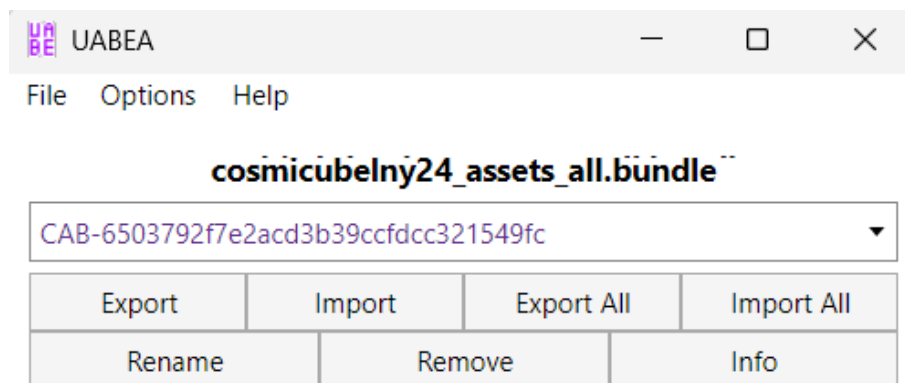


Работа с бандлами:

После открытия любого бандла появляется окно с вопросом, разжать его содержимое в оперативную память или на диск. Я всегда выбираю Memory, но если вес очень большой, жмите File.



Откроется окно редактирования бандла. Собственно говоря, что такое бандл – это файл, который содержит один и более архивов с их ресурсами. Здесь вы можете раскрыть список и увидеть из каких CAB-архивов он состоит.



➤ Объяснение кнопок:

- Export / Import – экспортировать / заменить выбранный архив
- Export All / Import All – экспортировать все архивы / заменить все архивы

- Rename / Remove – переименовать / удалить выбранный архив
- Info – просмотреть внутренние объекты выбранного архива
Примечание: .resS и .resource не содержат объектов, надо выбрать архив либо без расширений, либо оканчивающийся на .sharedAssets.

После нажатия на кнопку Info, появится окно Assets Info, где можно изучать и редактировать объекты.

➤ Сохранение бандла:

- В окне Assets Info щелкнуть File → Save, для сохранения изменений в архиве.
- Затем вам необходимо его закрыть и в окне управления бандлами тоже нажать File → Save, для внесения изменений уже в сам бандл, который является его родительским файлом.

➤ Как работать с большим количеством бандлов:

Открывать поштучно каждый бандл и работать с его архивами по отдельности – это просто пытка. Благо, в UABEA есть консольный режим, который нам поможет быстро подменить архивы в бандлах.

```

UABE AVALONIA
WARNING: Command line support VERY EARLY
There is a high chance of stuff breaking
Use at your own risk
  UABEAvalonia batchexportbundle <directory>
  UABEAvalonia batchimportbundle <directory>
  UABEAvalonia applyemip <emip file> <directory>
Bundle import/export arguments:
  -keepnames writes out to the exact file name in the bundle.
    Normally, file names are prepended with the bundle's name.
    Note: these names are not compatible with batchimport.
  -kd keep .decomp files. When UABEA opens compressed bundles,
    they are decompressed into .decomp files. If you want to
    decompress bundles, you can use this flag to keep them
    without deleting them.
  -fd overwrite old .decomp files.
  -md decompress into memory. Doesn't write .decomp files.
    -kd and -fd won't do anything with this flag set.

```

В проводнике откройте папку UABEA и выполните одну из команд:

UABEAvalonia batchexportbundle <directory>

UABEAvalonia batchimportbundle <directory>

<directory> – папка с бандлами, а не *Data*. Бандлы обычно лежат по пути “*GameName_Data\StreamingAssets\AssetBundles*”.

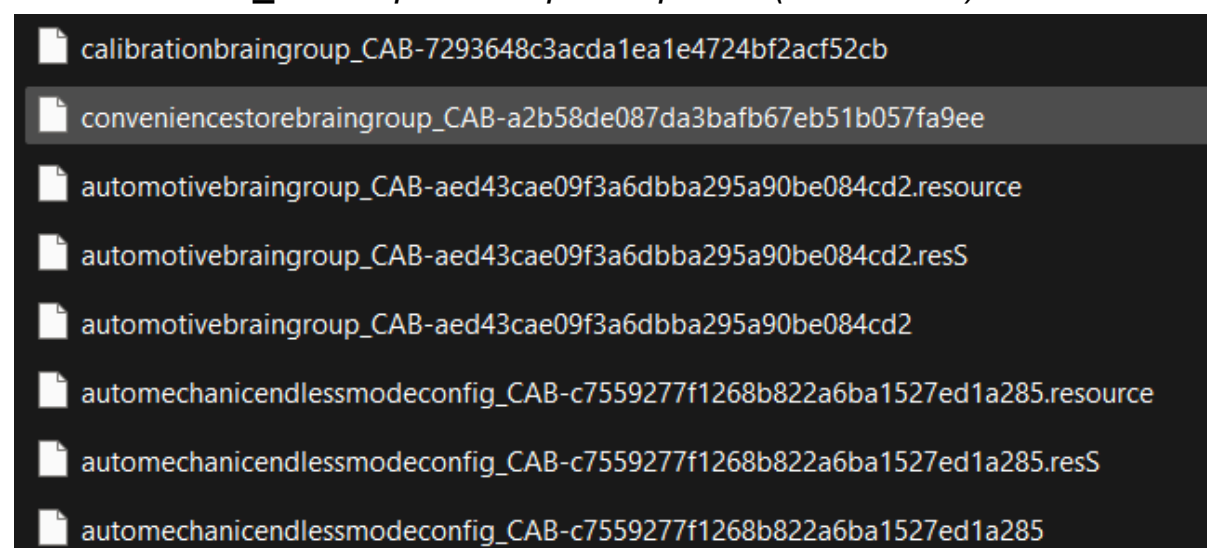
Пример полной команды:

UABEAvalonia batchexportbundle

“D:\Game\Game_Data\StreamingAssets\AssetBundles”

По итогу рядом с бандлами появятся файлы с именем вида:

<имя бандла>_<имя архива>.<расширение (если есть)>



Путь экспорта в консольной команде указать нельзя.

Отсортируйте файлы по столбцу *Дата изменения*, чтобы извлеченные архивы оказались в самом верху списка – это поможет вам не запутаться. Теперь вы можете через UABEA открыть за раз сразу все архивы:

Имя	Дата изменения	Тип	Размер
calibrationbraingroup_CAB-7293648c3acda1ea1e4724bf2acf52cb	11.05.2025 22:21	Файл	5 КБ
conveniencestorebraingroup_CAB-a2b58de087da3bafb67eb51b057fa9ee	11.05.2025 22:21	Файл	33 377 КБ
automotivebraingroup_CAB-aed43cae09f3a6dbba295a90be084cd2.resource	11.05.2025 22:21	Файл "RESOURCE"	9 184 КБ
automotivebraingroup_CAB-aed43cae09f3a6dbba295a90be084cd2.resS	11.05.2025 22:21	Файл "RESS"	60 606 КБ
automotivebraingroup_CAB-aed43cae09f3a6dbba295a90be084cd2	11.05.2025 22:21	Файл	45 429 КБ
automechanicendlessmodeconfig_CAB-c7559277f1268b822a6ba1527ed1a285.resource	11.05.2025 22:21	Файл "RESOURCE"	109 928 КБ
automechanicendlessmodeconfig_CAB-c7559277f1268b822a6ba1527ed1a285.resS	11.05.2025 22:21	Файл "RESS"	57 344 КБ
automechanicendlessmodeconfig_CAB-c7559277f1268b822a6ba1527ed1a285	11.05.2025 22:21	Файл	36 563 КБ
conveniencestoreendlessmodeconfig	11.03.2025 23:41	Файл	206 225 КБ
automechanicendlessmodeconfig	11.03.2025 23:40	Файл	203 834 КБ
conveniencestorebraingroup	11.03.2025 23:40	Файл	97 506 КБ

Главное выделять таким образом, чтобы первый файл был валидным архивом (не ресурсом).

Архивы можно импортировать в бандлы двумя способами:

- Автоматически через консольную команду
- Вручную через графический интерфейс UABEA, нажав кнопку Import All (вам понадобится заранее отсортировать архивы по отдельным папкам)

Чтобы ускорить запуск через консоль, создайте bat-файл с такой структурой:

```
@echo off
```

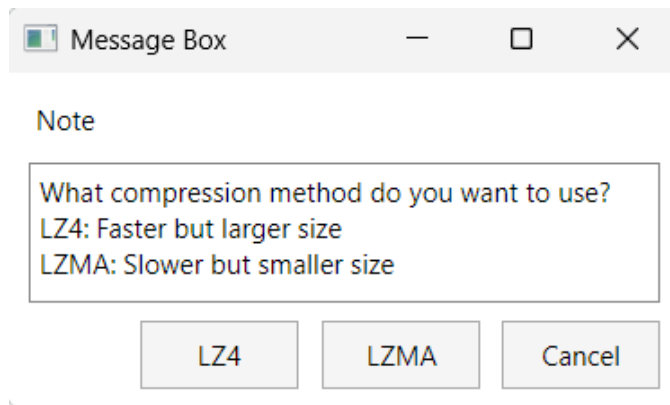
```
UABEAvalonia batchimportbundle
```

```
"D:\Game\Game_Data\StreamingAssets\AssetBundles"
```

```
pause
```

➤ Как сжать бандлы:

Возможно после импорта архивов в бандл вы заметите увеличение веса. Это потому что не применяется сжатие или оно слабое. Чтобы исправить ситуацию, откройте в UABEA бандл и перейдите в File → Compress. Далее укажите путь сохранения, выберите метод сжатия и подмените несжатый бандл сжатым.

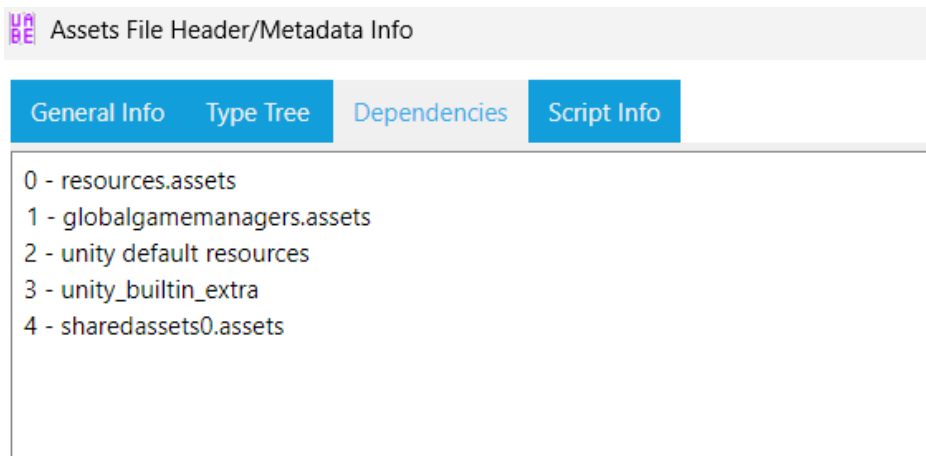


Обзор окна Assets Info

Таблица отображает список объектов из текущего и зависимых архивов.

Значение некоторых столбцов:

- File ID – айди родительского файла. 0 – из текущего архива (который открыли), 1 и больше – из зависимостей. Чтобы получить полное имя исходя из этого номера, откройте окно Tools → Dependencies (F6).

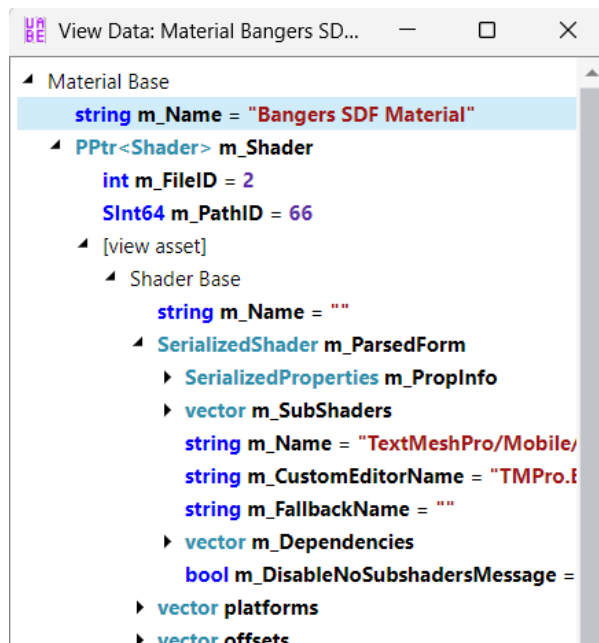


- Path ID – айди объекта внутри родительского файла.
- Size – байтовый размер данных объекта в сыром виде. Для аудио, видео и текстур он может показывать некорректные данные, потому что в архивах находятся лишь метаданные, а сам контент записывается в ресурс, отдельный файл.

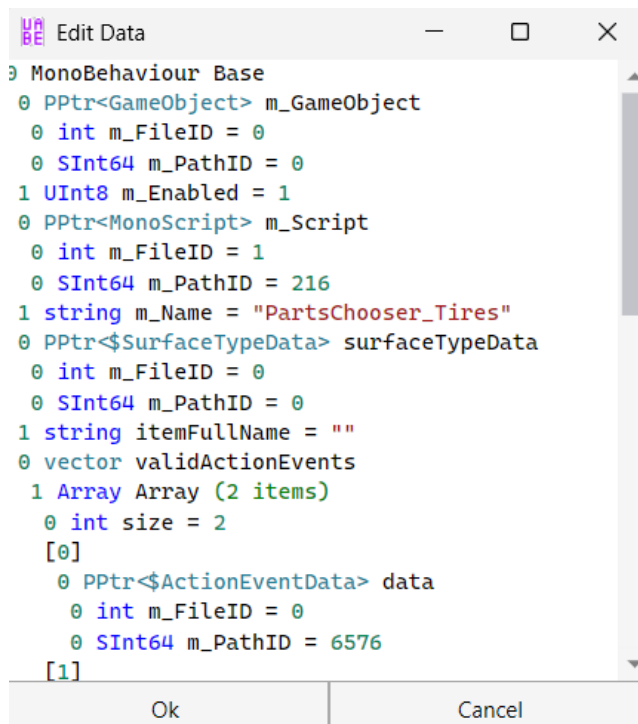
По нажатию на строку таблицы основная информация про объект переносится в правую половину окна. А также становятся доступны кнопки для операций, связанных с объектом.

Объяснение кнопок:

- View Data – открыть окно для просмотра свойств объекта.



- View Scene – показать сцену, где присутствует выбранный объект. Если программа не найдет его ни на одной сцене, то выдаст ошибку.
- Export Raw / Import Raw – экспортировать / импортировать в двоичном виде.
- Export Dump / Import Dump – экспортировать / импортировать в виде дампа формата txt/json (дамп – это raw данные, которые распарсили для удобного просмотра).
- Plugins – используется, когда для модифицирования контента выбранного объекта требуется дополнительная обработка. На данный момент плагины есть для типов TextAsset, Texture2D, AudioClip, Font.
- Edit Data – отредактировать свойства объекта через текстовый дамп, на месте (без его предварительного извлечения).



- Remove – удалить выбранный объект.

Внимание: экспортированные файлы (дампы, текстуры, текстовики и т.д.) нельзя переименовывать, иначе UABEA не поймет как паковать. В имени содержится информация про родительский архив и PathID.

Иногда замена текстур проходит вроде бы успешно, но игра потом вылетает на старте. Из того, что я могу посоветовать – либо паковать несколько раз пока не получится, либо воспользоваться новой версией – UABEANext. Если всё равно не будет работать, опишите проблему на дискорд сервере разработчика.

Меню View:

- *Search by name (Ctrl+F)* – найти объект по части имени. Допускается использовать wildcard символ * для подстановки любого количества любых символов, включая их отсутствие.
- *Continue search (F3)* – продолжить поиск с теми же параметрами, переходя к следующему совпадению.
- *Go to asset (Ctrl+G)* – перейти к объекту по FileID и PathID.
- *Filter (Ctrl+E)* – позволяет применить фильтры к таблице, чтобы отображать или скрывать объекты определённых типов.

➤ Изучение сцены:

Сцена в Unity — это пространство, в котором размещаются все игровые объекты и задаётся их начальное расположение, поведение и взаимодействие. Проще говоря, сцена — это контейнер игрового уровня, меню или любого другого визуального экрана в проекте.

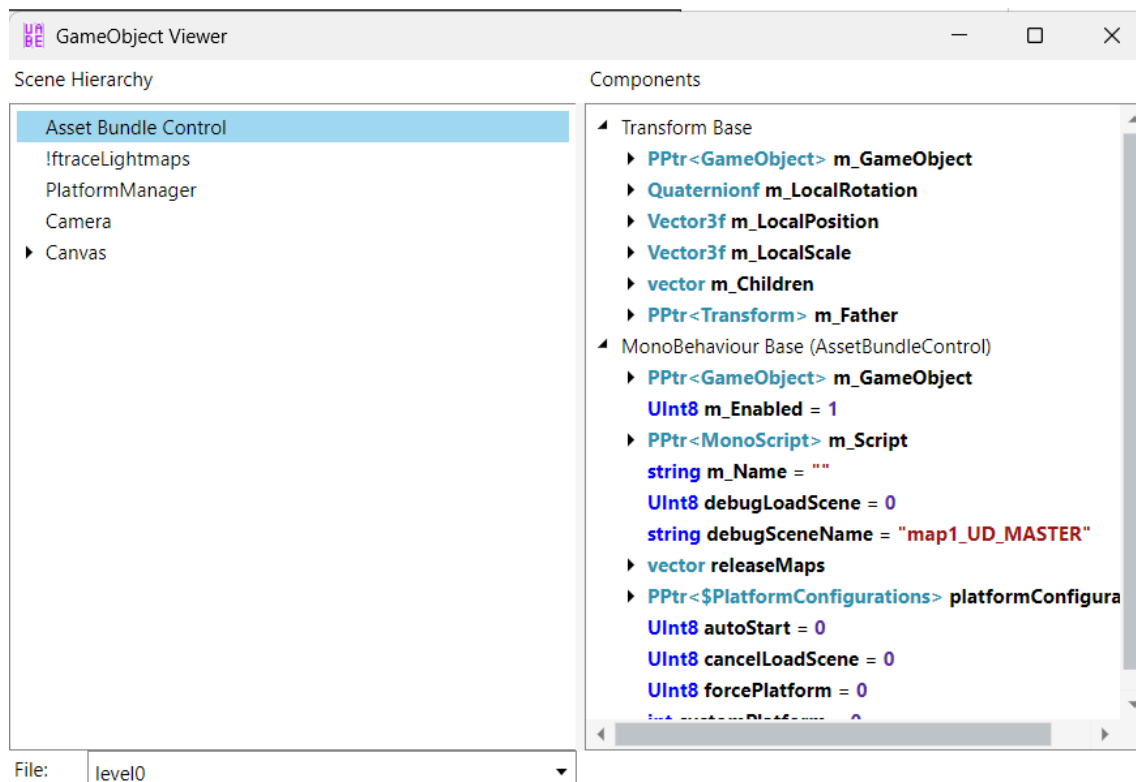
После загрузки сцены вы работаете с иерархией объектов типа **GameObject**. Это универсальные контейнеры, к которым можно добавлять любые компоненты: визуальные, физические, звуковые, пользовательские. Они могут быть как пустыми (например, для организации структуры), так и представлять полноценный игровой объект — персонажа, камеру, кнопку интерфейса или источник света.

Каждый **GameObject** обязательно содержит компонент **Transform**, который отвечает за его положение, поворот и масштаб в пространстве. Остальные компоненты добавляются по необходимости, например: **MeshRenderer** — для отображения модели, **Collider** — для физического взаимодействия, **AudioSource** — для воспроизведения звуков. Скрипты на C#, унаследованные от **MonoBehaviour**, также добавляются как компоненты и управляют поведением объектов.

Таким образом, сцена в Unity — это структурированная иерархия **GameObject** с компонентами, которые в совокупности создают внешний вид, механику и логику игры.

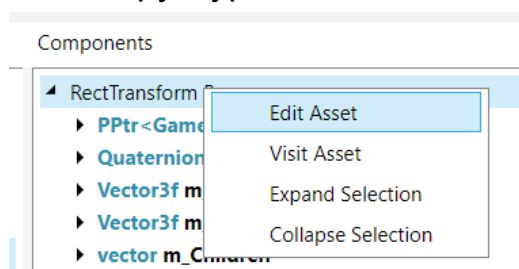
Практическая часть:

Для просмотра сцены выберите *Tools → Scene Hierarchy (F8)*. Перед вами появится небольшое окно. То, что вы видите слева — это список объектов типа **GameObject**. После выделения игрового объекта, его компоненты высветятся в правой части окна.



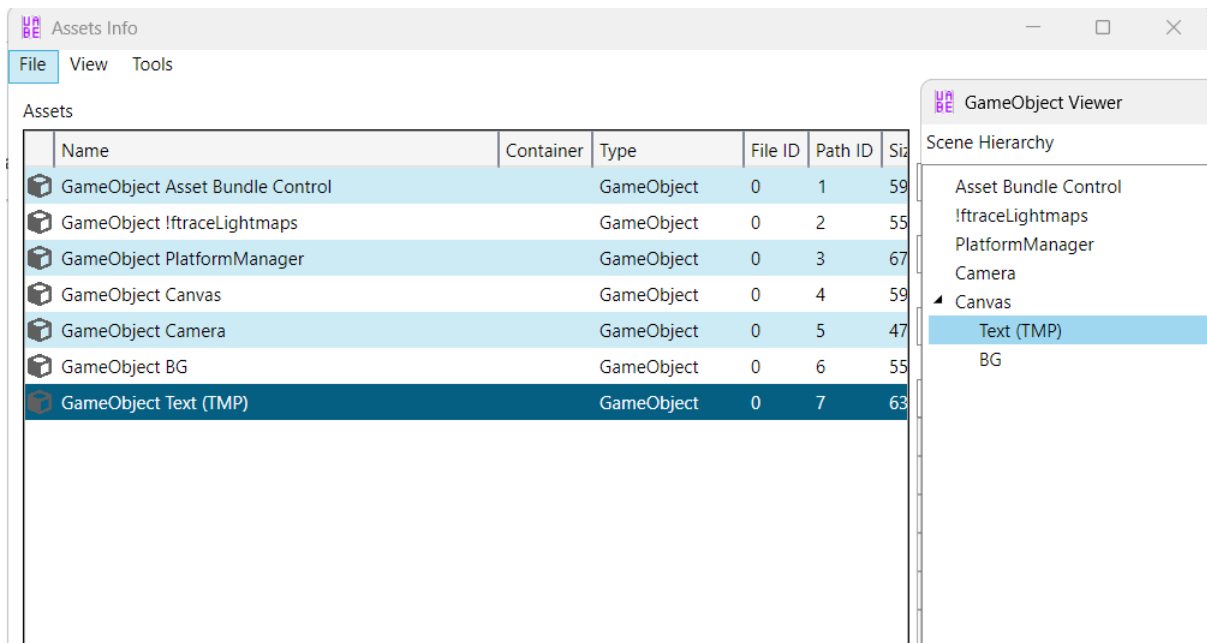
Щелкнув правой кнопкой мыши на любом компоненте, вы получите доступ к следующим действиям:

- *Edit Asset* – позволяет отредактировать текстовый дамп компонента напрямую, без необходимости экспорта.
- *Visit Asset* – выделяет связанный объект в таблице окна “Assets Info”.
- *Expand Selection / Collapse* – раскрывает или сворачивает все вложенные ветки компонента, упрощая навигацию по его структуре.

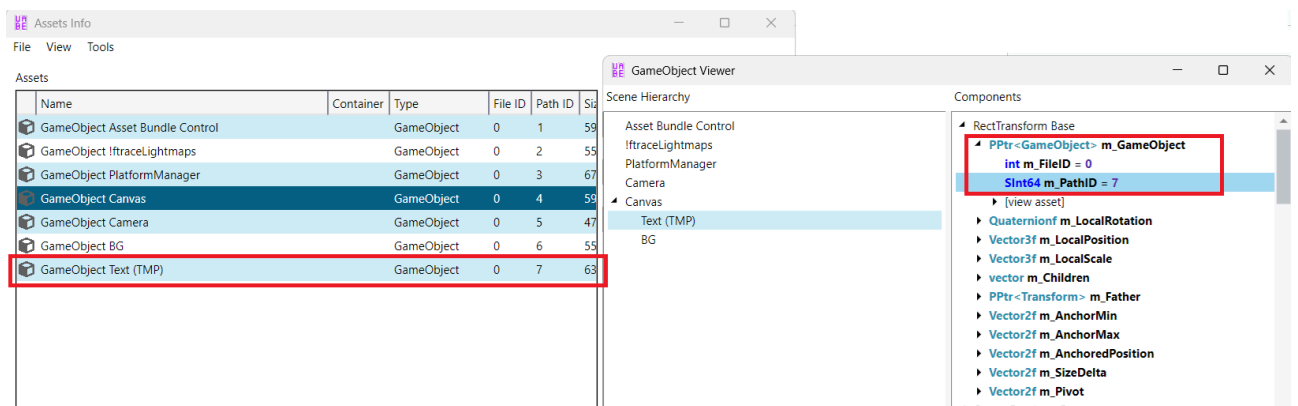


В каких файлах лежат сцены: *level<номер>*, *ассетбандлы*, *бандлы*.

Аналогично вы можете просматривать каждый игровой объект по отдельности, применив фильтр **GameObject** в окне Assets Info.



Если в дампе какого-то объекта поле `m_GameObject` содержит ненулевое значение, значит данный объект является компонентом и прикреплен к соответствующему `GameObject`.



Как в UABEA импортировать всё за раз

Массовый импорт дампов:

- При наличии бандлов, их надо сперва распаковать.
- Открыть в UABEA все .assets файлы.
- Нажать Ctrl+A, чтобы выделить все объекты.

⚠ Если объектов очень много, программа может зависнуть — в этом случае лучше заранее отфильтровать только нужные типы в окне фильтров.

- Нажать кнопку Import dump.
- По запросу выбрать папку с дампами.
- Нажать File → Save,
- Если использовались CAB-архивы — их нужно запаковать обратно в бандлы.

Для текстур похожим образом:

- В фильтрах оставить только тип **Texture2D**.
- Выделить все ассеты сочетанием Ctrl+A.
- Plugins → Export PNG

И точно также импорт, нажав Import PNG

Единственное, при массовом импорте нельзя выбрать другой формат, текстуры будут паковаться в формате, который был изначально.

Самые распространенные форматы на Windows:

DXT5 - сжатие с потерями качества

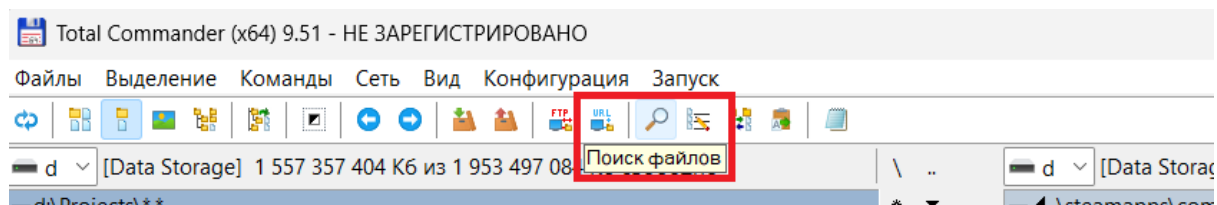
BC7 - сжатие без потерь

Смотрите также:

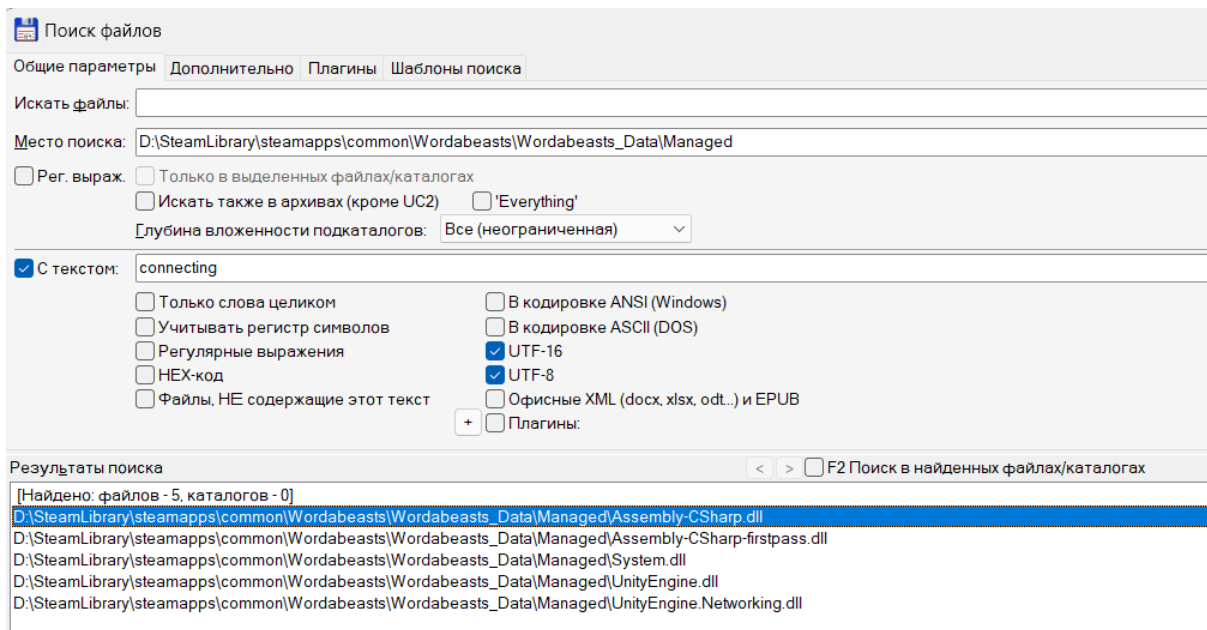
[UABEA – решение ошибки “Asset failed to deserialize”](#)

Как искать текст в бинарниках (TotalCommander)

Нажмите кнопку лупы.

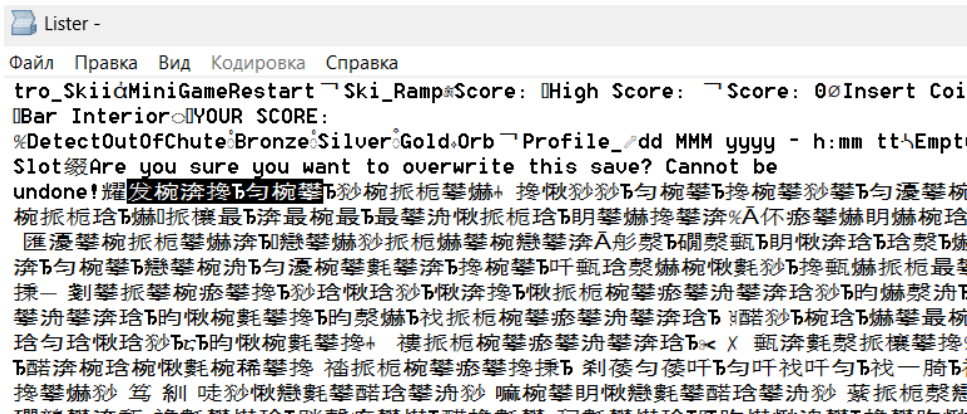


Введите место поиска и поисковую строку. Не забудьте поставить галочку возле пунктов **UTF-8** и **UTF-16**. Файлы, где обнаружится указанный текст, будут заноситься в область Результаты поиска.



Чтобы перепроверить факт наличия нужного текста в найденном файле:

- Задержите на нем правой кнопкой мыши до появления контекстного меню, затем выберите **Просмотр (Lister)**.
- Во вкладке **Вид** переключитесь на **UTF-16**.
- При помощи сочетания клавиш **Ctrl+F** осуществите поиск фрагмента строки, затем переключите вид на **Только текст**.

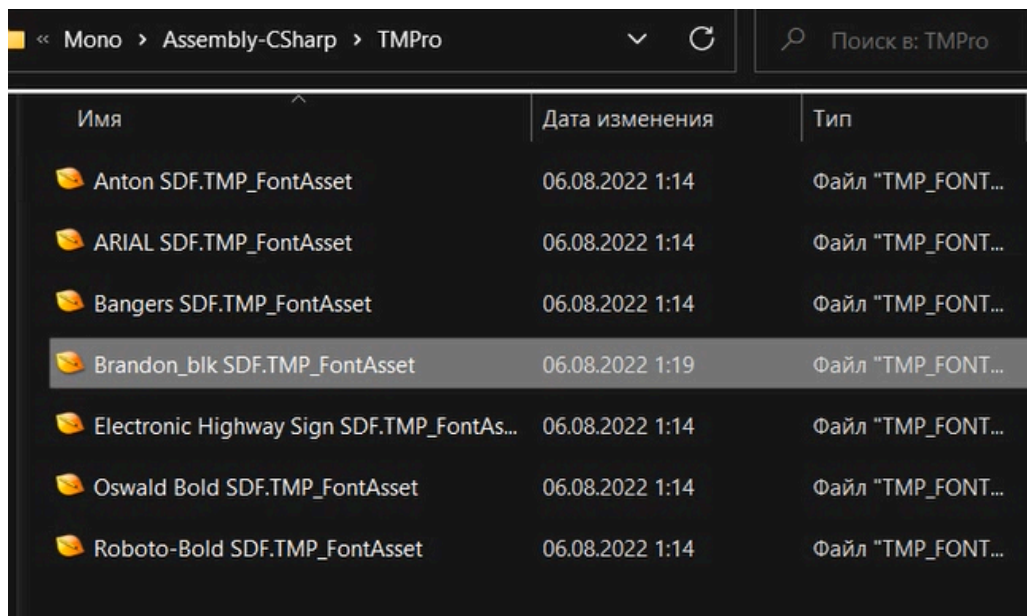


текстовик, поменять расширение на .bat и вставить в него следующий текст:

(Батник и UnityEX надо поместить в игровую папку Data)

```
@echo off
color a
for %%a in (resources.assets) do UnityEX.exe export "%%a"
-mb_new -t *SDF.TMP_FontAsset
for /l %%i in (0,1,90) do if exist sharedassets%%i.assets
UnityEX.exe export "sharedassets%%i.assets" -mb_new -t
*SDF.TMP_FontAsset
pause
```

Запустить батник.



The screenshot shows a file explorer window with the path <code>< Mono > Assembly-CSharp > TMPro</code>. The search bar contains <code>Поиск в: TMPro</code>. The table below lists the files found in this directory.

Имя	Дата изменения	Тип
Anton SDF.TMP_FontAsset	06.08.2022 1:14	Файл "TMP_FONT..."
ARIAL SDF.TMP_FontAsset	06.08.2022 1:14	Файл "TMP_FONT..."
Bangers SDF.TMP_FontAsset	06.08.2022 1:14	Файл "TMP_FONT..."
Brandon_blk SDF.TMP_FontAsset	06.08.2022 1:19	Файл "TMP_FONT..."
Electronic Highway Sign SDF.TMP_FontAs...	06.08.2022 1:14	Файл "TMP_FONT..."
Oswald Bold SDF.TMP_FontAsset	06.08.2022 1:14	Файл "TMP_FONT..."
Roboto-Bold SDF.TMP_FontAsset	06.08.2022 1:14	Файл "TMP_FONT..."

- Извлеченные файлы (с расширением .TMP_FontAsset) перемещаю в папку **files_unity** утилиты **fntxml_to_unityfnt**. Далее я из этих метрик экспортирую данные, чтобы посмотреть размер шрифта и список символов.

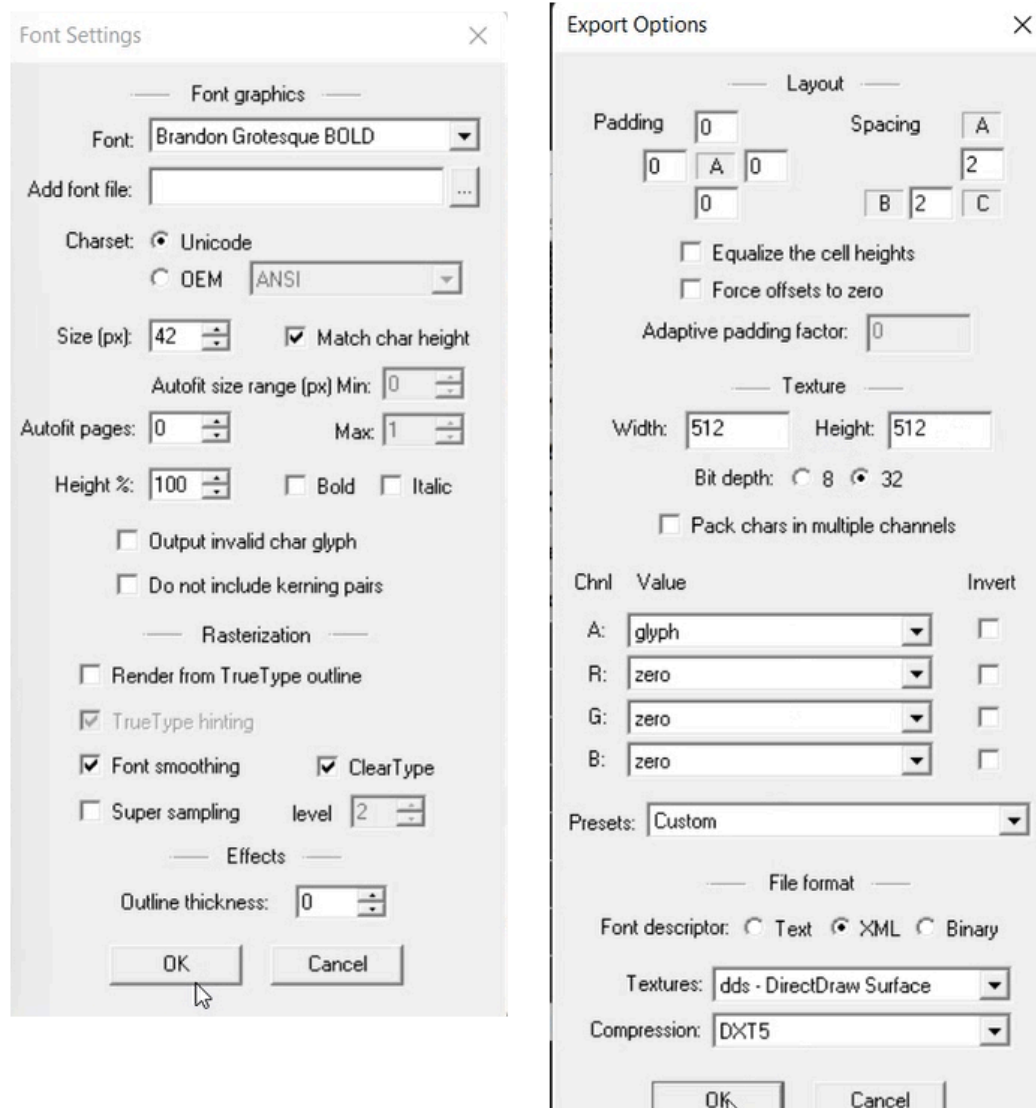
Вам достаточно использовать только два батника

export_UnityToFNT_auto_nchnl.bat – снятие координат с бинарного юнити файла в папку **files_fnt** (если в этой папке был fnt файл, он перезапишется!).

import_UnityToFNT_auto_nchnl.bat – импортирование координат из fnt файла в бинарник юнити.

Другие батники от этой утилиты вроде не нужны. Здесь автоматически определяется смещение данных. Также при импорте/экспорте надо смотреть, чтоб не было ошибок в командной строке, иначе это означает, что со шрифтом проблемы и в общем его лучше просто скипнуть.

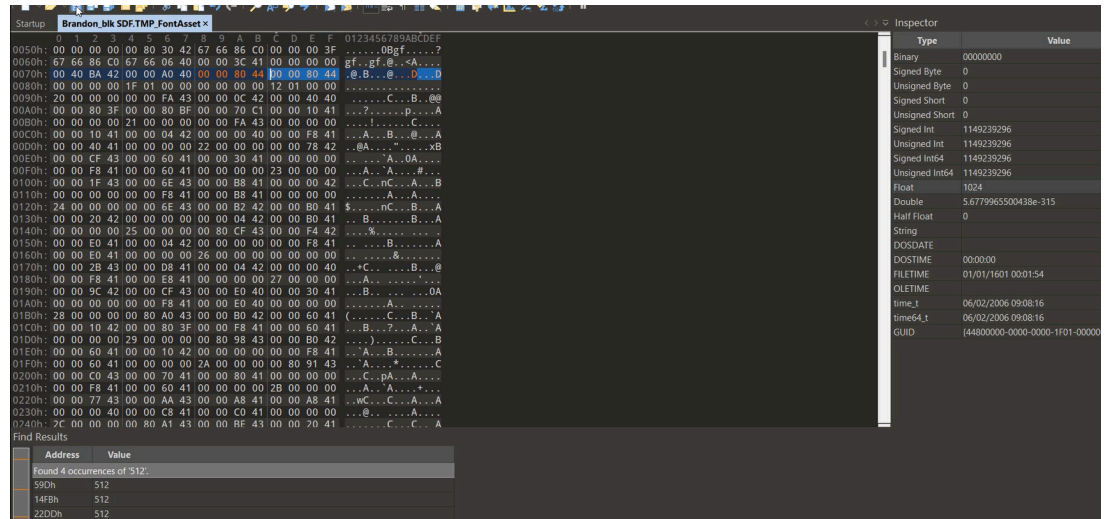
- Открываю и настраиваю **BMFont**:
 1. FontSettings:
 - Выбрать шрифт (возможно придется его установить, чтобы отобразился в списке).
 - Charset – Unicode
 - Поставить галочку возле Match char height
 - Возле Bold/Italic если надо, и если выбранный шрифт имеет отдельные файлы с такими стилями. Легко определить: если в ячейках вместо букв квадраты – значит выбранный стиль не поддерживается шрифтом.
 - Размер я беру из fnt, который мы получали путем экспорта.
 - Другие настройки трогать не надо.
 2. Export Options:
 - Bit depth: 32 Bit
 - Format: XML
 - Texture: dds
 - Compression: DXT5.
 - Поменять Width и Height на значения такие же, как у оригинального атласа.
 3. Настройка символов:
 - Убедитесь, что в BMFont выбраны все символы из файла .fnt. Номера символов отображаются при наведении курсора. Я делаю именно так, чтоб не возникло проблем, если не хватит букв, которые изначально шрифт содержал.



Настройки BMFont

- Настройки советую сохранить в файл.
- Сохраняю полученный шрифт, перезаписывая оригинальный fnt.
- Если утилита сгенерировала более одной текстуры для шрифта:
 - В настройках экспорта увеличиваю размеры атласа пока все символы не поместятся на одной картинке. Советую вписывать одинаковые значения для удобства.
 - Теперь необходимо изменить в бинарных метриках информацию о новом размере. Допустим изначально атлас был 512x512. Открываю файл TMP_FontAsset в любом hex редакторе и ищу значение 512 во float.

Обычно ширина и высота – это первые два найденных значения (они и расположены рядом). Через поле float меняю их на размеры атласа, сгенерированного с помощью BMFont, и сохраняю.



- Импорт и адаптация:
 - Запускаю батник **import_UnityToFNT_auto_nchnl** для импорта. Важно: у fnt и TMP_FontAsset должны быть одинаковые имена не считая расширения.
 - Через графический интерфейс **UnityEX** экспортирую оригинальные атласы, подменяю атласы и бинарные метрики на свои в папке **Unity_Assets_Files**. Пакую, предварительно подтянув имена кнопкой **MonoBehaviour**.
 - Готово. Можно зайти в игру и протестировать.

Информация для сглаживания символов на атласе.

Плагин:

<https://catlikecoding.com/unity/documentation/CCDistanceMapGenerator/>

Файлом: <https://yadi.sk/d/S9x2LqCe3HHYjQ>

Чтобы работал, нужно добавить в юнити это

<https://yadi.sk/d/ZNB04inH3HHYrj>

Unity 2017+: создание SDF и автоматическая адаптация

Все необходимые файлы вы найдете [здесь](#)

Перед тем, как продолжить, убедитесь, что у вас установлен пайтон.

1. Извлеките шрифты из игровых файлов:

`Patcher unpack -t SDF`

2. Скачайте из [Unity архива](#) движок той же версии, что и игра ([Как узнать версию движка, на которой создана игра](#)). Ближайшие версии тоже могут подойти.

3. Сделайте сборку:

- Создайте новый 3D проект.
- Подготовьте текстовый файл с символами. Тут два варианта:
 - Скачать [Chars.txt](#), где прописаны все основные спец символы, английские и русские буквы.
 - Собрать символы со всех SDF шрифтов. Для этого из папки **Patcher_Assets** скопируйте файлы, содержащие **@TMP_FontAsset**, в папку **MonoBehaviour** утилиты [SDF_chars_checker](#). Запустите нужный батник и скрипт создаст файл **Chars_new.txt** (возможно вам понадобится отредактировать батник, указав правильные ключи извлечения).
- Перенесите в проект векторные шрифты, содержащие кириллицу, а также текстовик с символами.
- Выберите **Window** → **TextMeshPro** → **Font Asset Creator** (для Unity 2017 вам надо предварительно установить данную версию [TMPPro](#), закинув в корень проекта файлы из архива. Для Unity 2018 (где нет встроенного плагина) должен подойти [TMPPro](#) 1.0-1.5 версии. Подробное объяснение как установить/обновить плагин и подобрать нужную версию: [Плагин TextMeshPro](#)).

- Если это первый запуск, появится небольшое окно, где надо нажать верхнюю кнопку для подгрузки необходимых файлов, и потом закрыть его.
- Укажите следующие настройки:
 - **Source Font File:** файл ttf
 - **Sampling Point Size:** **autosizing** (либо как в оригинале, но увеличивая размер атласа, пока символы не поместятся на текстуре)
 - **Padding:** как в оригинале. Если размер атласа большой и у вас недостаточно мощный компьютер для генерации шрифта, попробуйте, например, указать 5. Учтите, что использование небольшого Padding при большом атласе хоть и может ускорить генерацию шрифта, но может также и сильно ухудшить качество отображения из-за недостаточного места для сглаживания (спасибо Solicen за информацию).
 - **Packing Method:** **Optimum**
 - **Atlas Resolution:** как в оригинале, или, если символы не помещаются на атласе (о чем вам сообщат в окошке с логом), то увеличить.
 - **Character Set:** **From File**. Здесь нужно выбрать текстовик с символами.
 - **Render Mode:** **SDF16** или как в оригинале (смотрите ниже).

Дополнительная информация:

- Для пиксельных шрифтов используйте рендер **RASTER_HINTED**, а размер шрифта укажите кратным четырем (либо **Auto Sizing**).
- Информацию про Padding и размер атласа можно посмотреть в одном из следующих полей дампа: **m_fontInfo** (для Unity 5), **m_CreationSettings** (более современные Unity). Либо она может находиться в отдельных полях: **m_AtlasWidth**, **m_AtlasHeight**, **m_AtlasPadding**.

- Чтобы узнать, какой рендеринг использовался в оригинале, откройте дамп и обратите внимание на поле `renderMode` или `m_AtlasRenderMode`.

Возможные значения:

SMOOTH_HINTED = 4121,
SMOOTH = 4117,
COLOR_HINTED = 69656,
COLOR = 69652,
RASTER_HINTED = 4122,
RASTER = 4118,
SDF = 4134,
SDF8 = 8230,
SDF16 = 16422,
SDF32 = 32806,
SDFAA_HINTED = 4169,
SDFAA = 4165

Также в старых версиях могут быть:

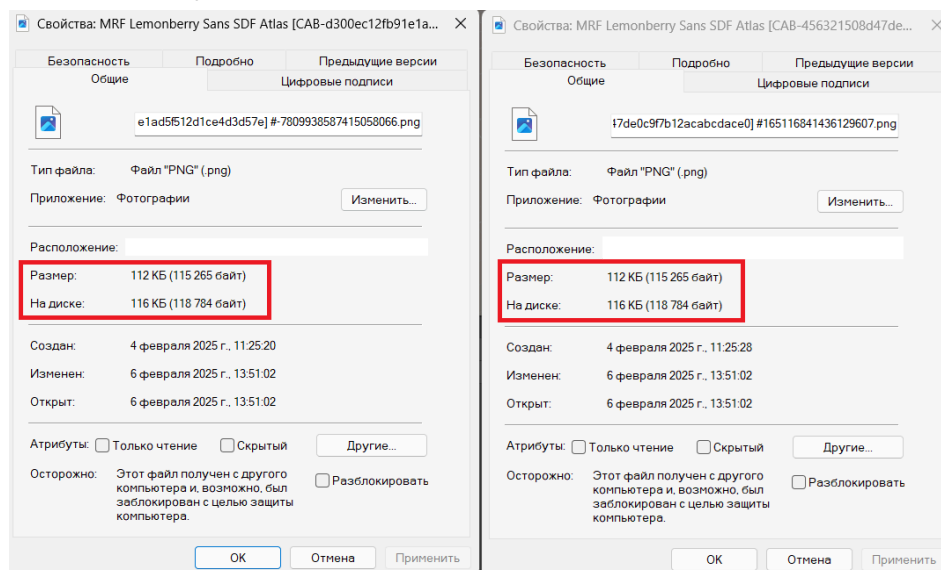
0: SMOOTH_HINTED;
1: SMOOTH;
2: RASTER_HINTED;
3: RASTER;
6: SDF16;
7: SDF32;

([Источник](#), автор: art-kandy)

- Нажмите кнопку для генерации и когда попросят выбрать путь, укажите имя файла точно как в оригинале. Если у вас дамп называется `Bangers SDF @TMP_FontAsset [sharedassets1.assets] #671.json`, значит имя ассета укажите `Bangers SDF.asset` (проще говоря вы копируете то, что идет до знака @). После чего создайте другие SDF под замену.

Учтите, что некоторые SDF на самом деле могут в игре не использоваться, а другие были добавлены для поддержки отдельных языков. Так, NotoSans зачастую содержит только иероглифы (о чем можно понять, взглянув на атлас), значит его менять не надо. Есть и дефолтные – LiberationSans, RobotoRegular, которые поставляются вместе с плагином TMLPro как примеры, разработчик их мог не удалять из проекта, но и никак не задействовать.

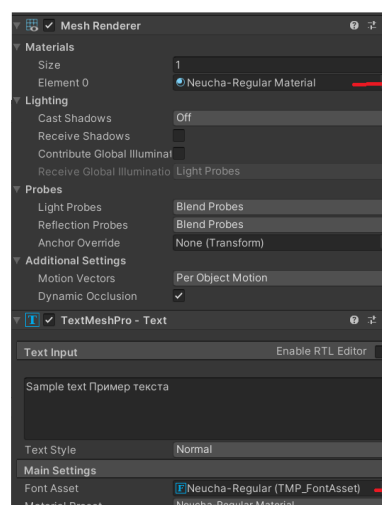
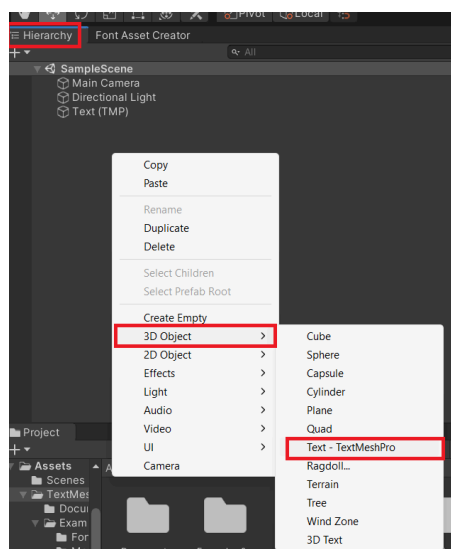
Иногда можно встретить идентичные SDF с разными материалами, в таком случае надо сравнивать размер атласов в байтах, чтоб в этом убедиться. И далее на движке генерировать несколько одинаковых шрифтов с соответствующими именами.



📁 MRF Lemonberry Sans SDF	06.02.2025 13:51	Папка с файлами
📁 MRF Lemonberry Sans SDF Full	06.02.2025 13:51	Папка с файлами
📁 MRF Lemonberry Sans SDFb	06.02.2025 13:51	Папка с файлами

Ну а для проверки на использование вы можете запаковать атлас, перекрашенный в черный квадрат. Если в игре пропадут символы – значит это тот самый шрифт.

- Создайте на сцене объект TMPro для каждого шрифта. Имя можете задать любое.



Выбрать материал
созданного sdf шрифта

Выбрать Font Asset
созданного SDF шрифта

- Обязательно сохраните проект (Ctrl + S). У некоторых людей возникает проблема, что если этого не сделать, то папка сборки будет пустая.
 - Соберите проект: **File** → **Build And Run** (Ctrl + B)
- Перейдите в папку сборки и распакуйте содержимое архива [SDF_adapter](#) рядом с папкой Data. В папку **ORIGINAL_FONTS** скопируйте оригинальные файлы шрифтов (атласы и дампы). Запустите батник **export_n_adapt**, в результате чего автоматически экспортируются шрифты из вашей сборки, затем скрипт перенесет информацию из одного дампа в другой (адаптирует), и сохранит результат в папке **OUTPUT**.
 - Перенесите папку **OUTPUT** в папку вашей игры и выполните команду для запаковки:
Patcher pack "OUTPUT"
 - Результат будет в папке **Patcher_Result**.

Если у оригинальных шрифтов дампы **MonoBehaviour** и атласы имеют разные имена (до знака @, или при его отсутствии – до [), то вам понадобится в папке сборки вручную копировать атлас из **Patcher_Assets** в **OUTPUT** и переименовывать под оригинал.

Смотрите также:

[Возможные проблемы и их решение](#)

Unity 2017+: замена через UABEA

Подробные руководства из **Steam**, где показана замена через **UABEA**:

[Заміна TMP-шрифтів у іграх на Unity](#)

[Виявлення та заміна шрифту у Unity](#)

*Для автоматического перевода вставьте ссылку в гугл переводчик.

➤ SDF адаптер для UABEA

Инструкция по использованию:

1. Скачать [пайтон скрипт](#).
2. Поместить **MonoBehaviour** дампы и текстурные атласы: сделанные на движке – в папку **ru_sdf**, оригинальные – в **original_sdf**.
3. Запустить скрипт. Он адаптирует шрифты и положит результат в новую папку под названием **ru_adapted_sdf**.
4. Проверить, чтобы в папке адаптированных шрифтов было столько же файлов, сколько и в ru папке. Если что-то не перенеслось, доделать вручную. Потом адаптированные шрифты запаковать в юнити архивы.

Напоминание:

- Импорт текстур в UABEA: **Plugins** → **Import Texture2D**
- Импорт **MonoBehaviour** дампов: **Import Dump**
- Для массового импорта выделить несколько ассетов

Возможные проблемы и их решение

1. Полоски вокруг букв или поврежденная тень
В **MonoBehaviour** дампе **TextMeshPro** отключить **m_enableExtraPadding** (сделать равным нулю).

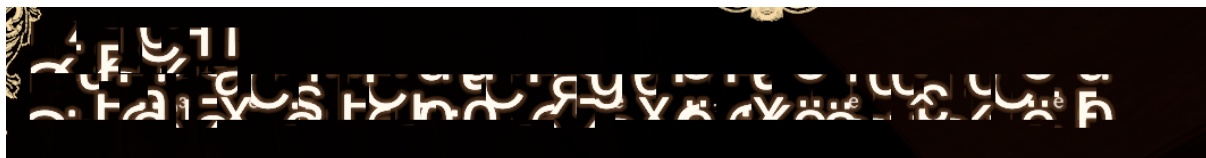
Может помочь также пересоздание **SDF** с правильными настройками:

- **Sample Point Size: auto size**
- **Atlas Resolution:** как в оригинале или, при необходимости, увеличить.
- **Padding:** как в оригинале. Если размер атласа большой и у вас недостаточно мощный компьютер для генерации шрифта, попробуйте, например, указать 5.

Прочие варианты:

- Вообще убрать тень (не рекомендую). Для этого оригинальный шейдер надо заменить шейдером из русской сборки юнити, то есть вытащить дамп, возможно подменить там важную информацию типа айди и заменить ею оригинал.

2. Буквы превратились в “кашу”



Возможно в юнити архиве был подменен атлас, но остался оригинальный **MonoBehaviour**, либо наоборот.

3. Буквы не отображаются вообще

- Если вы уверены что всё сделали правильно, после запаковки убедились через **UABEA/AssetStudio**, что замена прошла успешно и не выдается никаких ошибок, а также корректно проведена адаптация, вероятно, игра использует старую версию плагина **TextMeshPro**, а не ту, которая была установлена по умолчанию на вашем движке юнити. Решение проблемы описано здесь: [Плагин TextMeshPro](#) (вкратце: извлечь структуру своего шрифта, сравнивать с другими структурами для нахождения максимально близкой, установить на юнити плагин **TMPPro** в соответствии с подходящей структурой,

сделать SDF шрифты и извлечь, подкорректировать дампы **MonoBehaviour** при необходимости).

- b. Вы отредактировали текст в ассете, который был извлечен из бандла? Возможно catalog.json/bin не дает бандлу загрузиться после его модификации. Решение описано здесь: [Патчинг catalog](#).
- c. Если вы меняли ассеты через **UABEA**, возможно она криво запаковала файлы. Как это проверить? Открыть модифицированные и оригинальные юнити файлы в любой другой программе, затем посмотреть не выдается ли ошибок.

Варианты решения проблемы:

- 1. Попробовать **UABEANext**;
- 2. Паковать, пока не заработает.

4. Вместо новых шрифтов отображается Ариал

Значит не были отредактированы айди в дампе **MonoBehaviour** после создания на юнити. Соответственно, при вставке в другую игру они уже будут некорректные и игра вместо одного шрифта отображает запасной (Fallback).

5. Квадраты вместо букв

Некорректные айди в **MonoBehaviour** метрике, либо отсутствие символов в шрифте (если не все буквы стали квадратами).

6. Мыльные буквы (как будто низкое качество)

Проверьте корректность размеров атласов, они должны быть не меньше оригинальных.

7. Вместо одних букв отображаются другие

Периодически встречается такая проблема, что в ttf файле глифы помещают не туда. Решение: отредактировать шрифт с помощью FontCreator или чем-то подобным – найти символ и скопировать в правильную ячейку. Чтобы узнать, куда копировать, вам пригодится [таблица символов в юникоде](#).

8. Текст отображается, но с кучей проблем?

Замыливание некоторых букв, некорректная тень, жирность и т.д. Если вы видите такое количество проблем, подмените также SDF материал на извлеченный из собственной сборки. Не забудьте перенести в свой дамп поля `m_Name` и `m_Shader` из оригинала!

9. Прочие проблемы

➤ Автоматическое переключение на другой шрифт: если игра поддерживает несколько языков, то в объектах

`MonoBehaviour` может быть указана кастомная конфигурация относительно того, для какого языка какой шрифт применять. Поищите через AssetStudio по ключевым словам типа Text, Localization, English и так далее. Возможно, это обычный запасной шрифт, указанный в поле

`m_FallbackFontAssetTable`. Принцип системы Fallback прост: если в текущем шрифте нет нужных символов (или он содержит некорректные айди?), используется следующий — и так до конца списка. Либо переключение шрифта может быть прописано в игровом коде (читайте ниже).

➤ Странности со шрифтом (например удаляются пробелы или появляются лишние): вероятно костыль разработчика.

Декомпилировать библиотеку из папки **Managed** (обычно это **Assembly-CSharp.dll**), запустить текстовый поиск по имени шрифта, с которым связана проблема (взять из поля `m_Name` в его дампе). Таким образом вы найдете участок кода, где происходит модификация шрифта. И тут два варианта – либо сменить `m_Name` у шрифта, чтобы избежать модификации, либо удалить костыль в самом dll файле через **dnSpy**.

Остальные стандартные вещи вроде размера, отступов – редактируется через `MonoBehaviour` дамп соответствующего `TMP_FontAsset`.

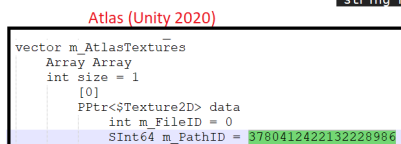
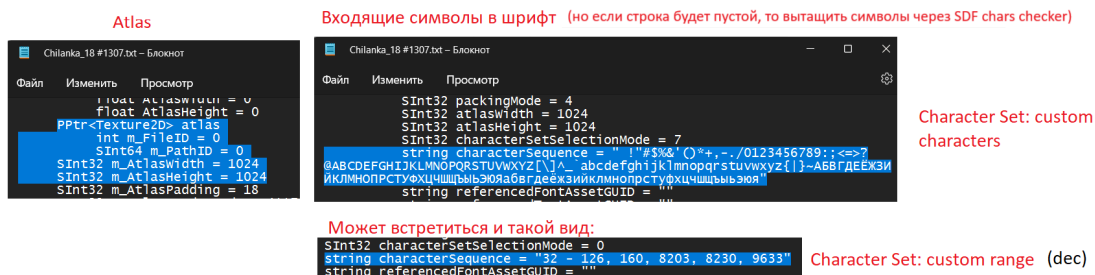
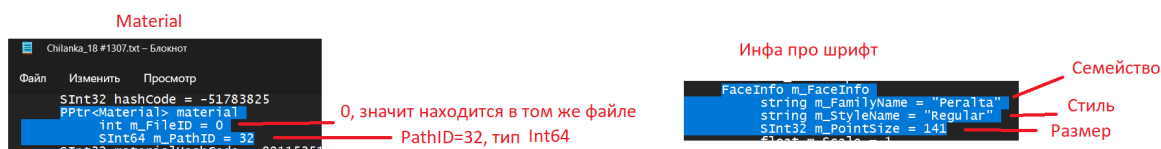
➤ Ничего не помогло исправить проблемы со шрифтом: если вы точно сделали всё возможное, но остались проблемы

отображения, есть маленькая вероятность, что сам плагин **TextMeshPro**, используемый игрой, содержит баги. С этим наверное ничего не поделаешь.

Общая информация по созданию SDF

➤ Основная информация:

- Атлас может быть 1 или несколько.
- **MonoBehaviour** дампа содержит базовую информацию и отладочную.
- **Material** содержит свойства типа жирность текста.



➤ Как проводить адаптацию после создания нового шрифта на движке и экспорта его дампа?

В разделе выше это делается автоматически, здесь же я расскажу про способы вручную. Некоторые поля могут отсутствовать в зависимости от версии юнити.

Способ №1. Перенос данных из нового дампа в оригинал (рекомендуется)

Что надо переносить:

- `m_GlyphTable`
- `m_CharacterTable`
- `m_UsedGlyphRects`
- `m_FreeGlyphRects`
- `m_KerningTable` (вроде не обязательно)
- `m_FaceInfo`
- `m_AtlasWidth`
- `m_AtlasHeight`
- `m_AtlasPadding`
- `m_AtlasRenderMode`

Для ~**Unity 5 – Unity 2018**:

- `m_fontInfo`
- `m_glyphInfoList`

Способ №2. Перенос данных из оригинала в новый дамп

Что надо переносить:

- `m_Name`
- `m_Script`
- `material`
- `atlas`
- `m_AtlasTextures` (на старых версиях юнити этого может не быть)
- `m_Material` (~Unity 6000+)

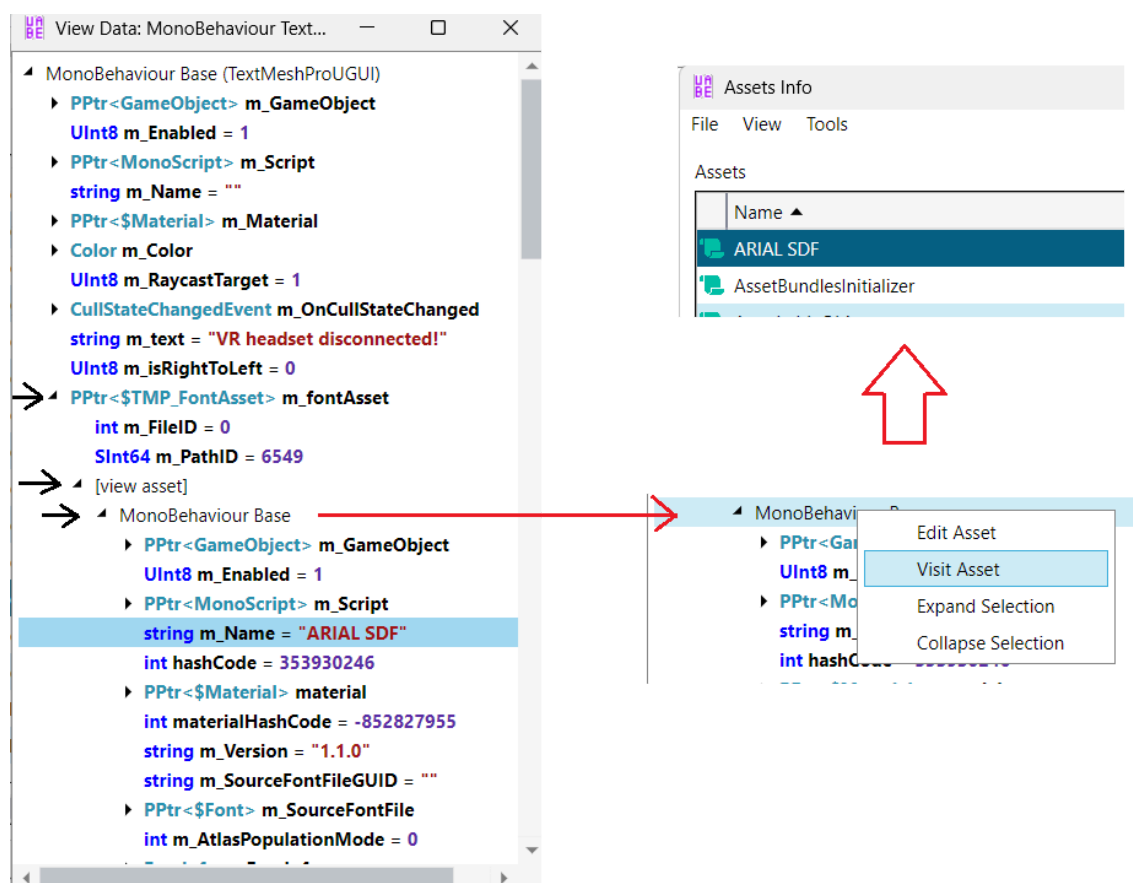
Если во время тестирования вместо букв квадраты – вероятно какой-то айди не перенесли из оригинала. Можно сверить дампы через **WinMerge**.

Почему первый способ лучше? Иногда в дампе созданного на движке шрифта отсутствуют некоторые поля из оригинала, что может привести к ошибкам из-за разной структуры файла. Вам придется дополнительно сравнивать дампы и исправлять их.

Переноса данные из своего дампа в оригинал вы обходите эту проблему.

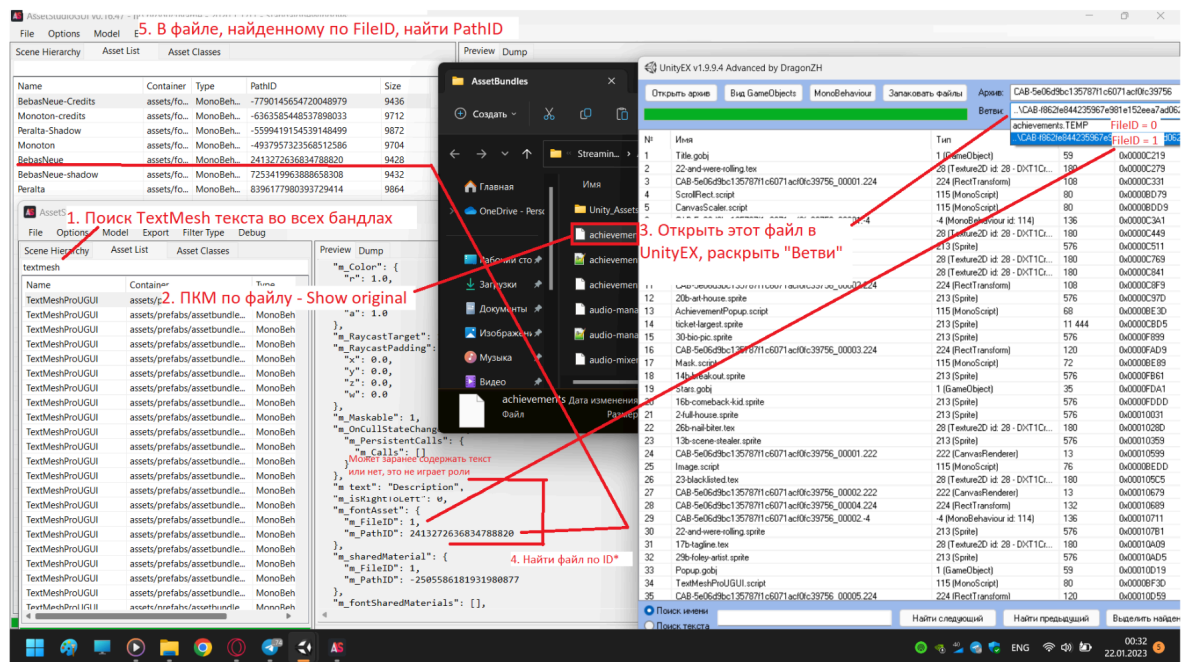
Как вычислить SDF шрифт

- Попробуйте пробить текст. Если он находится в объекте типа `@TextMeshProUGUI` или `@TextMeshPro`, айди шрифта будет указано в поле `m_fontAsset`.



UABEA позволяет быстро узнать имя шрифта и перейти к объекту `MonoBehaviour` по правой кнопке мыши для экспорта/импорта

Какой шрифт используется? Или же как найти по айди файл - в бандлах /в ассетах



А вот так делать не советую. Просто оставлено для информации.

- Отсеять атласы по внешнему виду. Оставшиеся заменить кириллическими, либо импортировать пустую картинку, чтобы проверить исчезнут ли буквы в игре.
- Если игра использует формат хранения текста в виде json (ключ-значение), то попробовать выйти по тексту ключа на @TextMeshProUGUI, а через него на SDF шрифт. Поиск по содержимому есть в патчере (команда search), возможно и в **AssetStudioMod-CLI** (консольная версия от aelurum). Пример команды для патчера: **Patcher search "somekey" --export**
- Если после запаковки кириллического текста игра переключилась на Arial или другой шрифт – узнайте как выглядел шрифт изначально и найдите его по атласу. Возможно вы просто забыли его подменить (или какой-нибудь из его дубликатов) или указали некорректные айди и поэтому игра не может его отобразить. Ещё немного информации на эту тему написано чуть выше в “Проблемах и возможных решениях”.

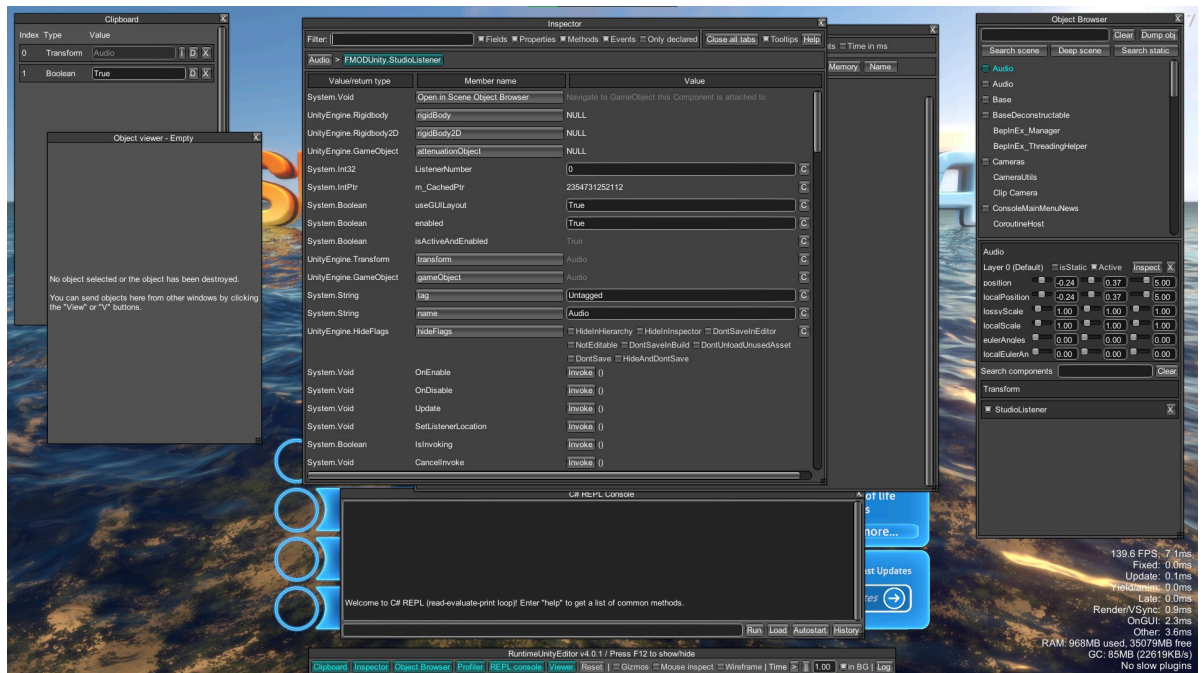
- Поставить плагин для изучения запущенной юнити игры. Можно по щелчку на строку текста узнать имя его SDF и прочую информацию. Пример таких плагинов:

[RuntimeUnityEditor](#)

[UnityExplorer](#)

Подробное руководство:

[UnityExplorer – изучение объектов запущенной игры](#)



Графический интерфейс плагина RuntimeUnityEditor

Плагин TextMeshPro

На старых версиях движка вы можете столкнуться с отсутствием встроенного плагина TMPPro, поскольку первое время он ставился из магазина и не был частью юнити. Что делать в такой ситуации?

“Все версии TMP можно скачать из гитхаба ([ссылка](#)), а также с самой юнити с vpn, примерно на Unity 2018-2019 можно было полный список видеть, а до включ. 2022 только установка ссылкой или распакованным package.json, на новых вообще не установить он вложенным пошёл в UI”, – dragonzh

На какой версии движка какие версии TMPPro доступны:

Unity 2018.2.20: TMLPro 0.1.2 — 1.3.0

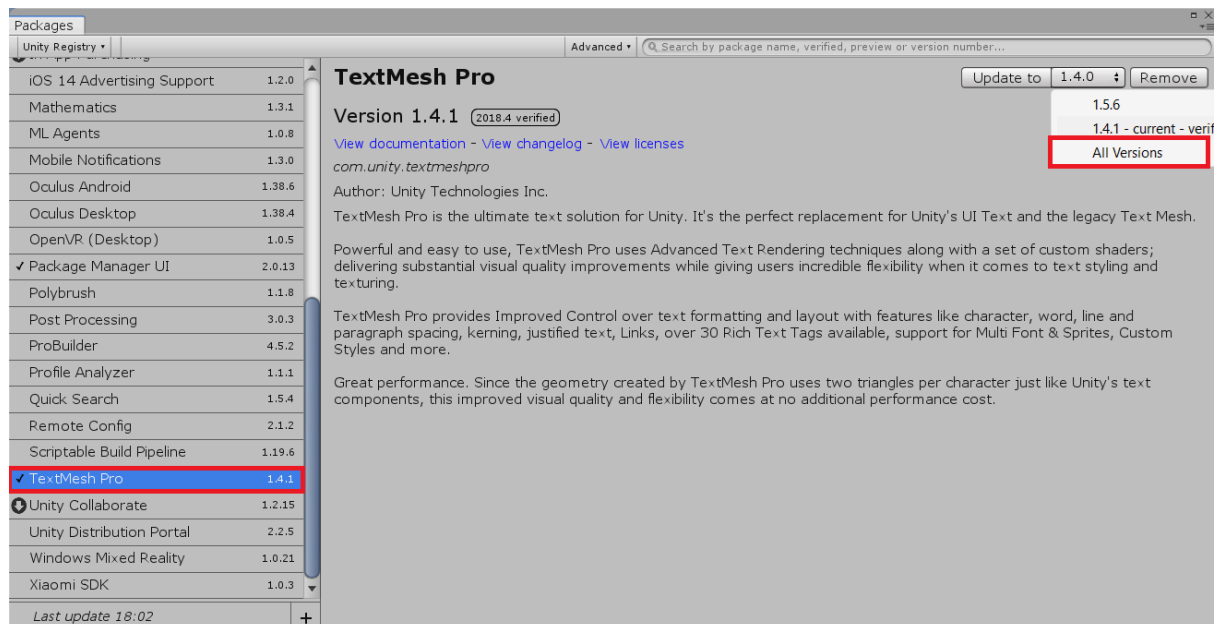
Unity 2018.4.36: TMLPro 0.1.2 — 1.6.0-preview1

Минимальная версия движка для разных версий плагина:

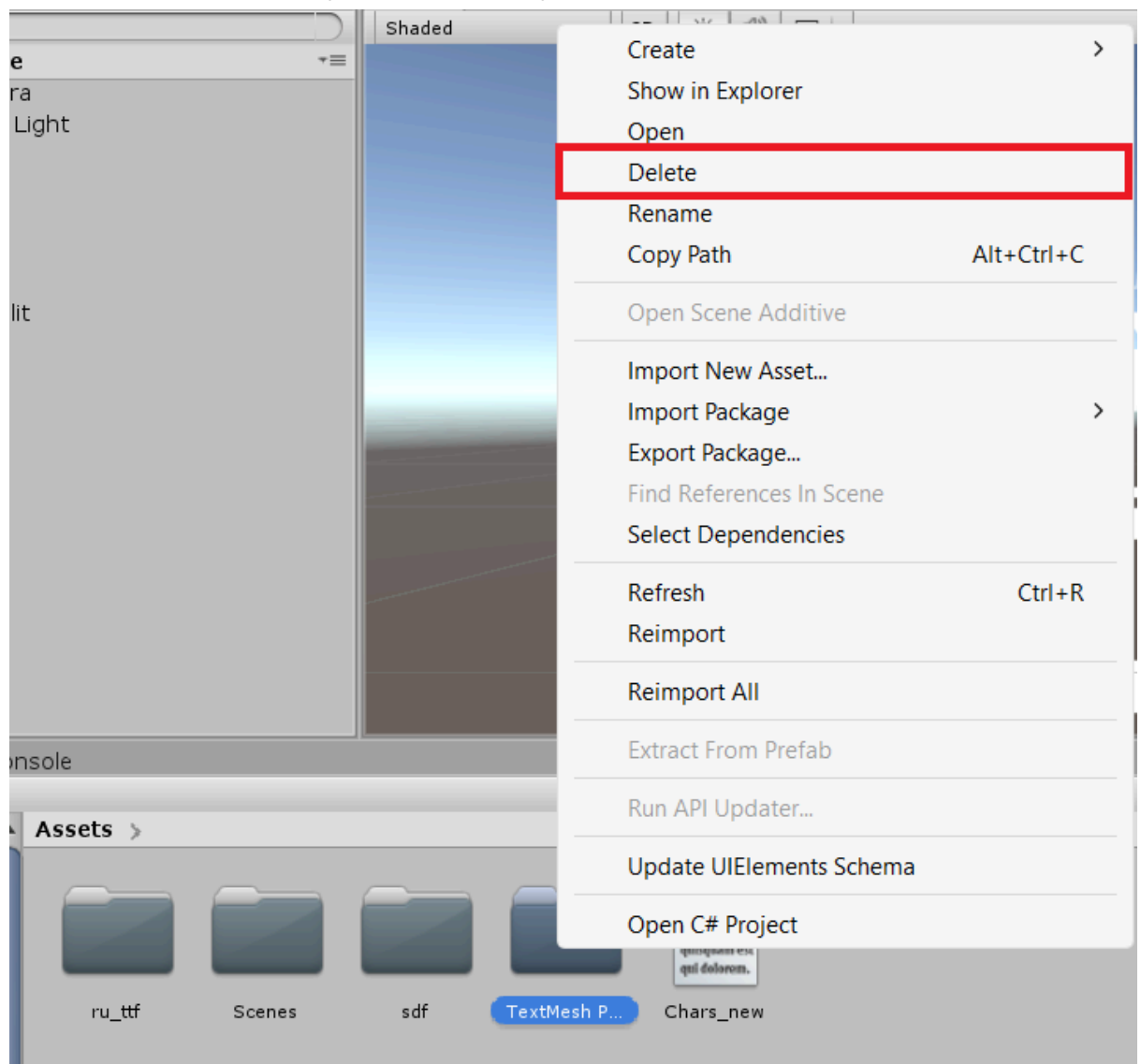
1.6	2018.4+	4.0	2022.1+
1.5	2018.4+	3.2	2020.3+
1.4	2018.3+	2.2	2019.4+
1.3	2018.1+	2.1	2019.1+
1.2	2018.1+	2.0	2019.1+
1.1	2018.1+		
1.0	2018.1+		

Unity 2018-2019. Как сменить версию плагина через Package Manager:

1. Window → Package Manager
2. В левой части окна выбрать TextMeshPro, а в правой – нужную версию, наведя стрелку мыши на пункт All Versions. Нажать Update to → Закрывать окно



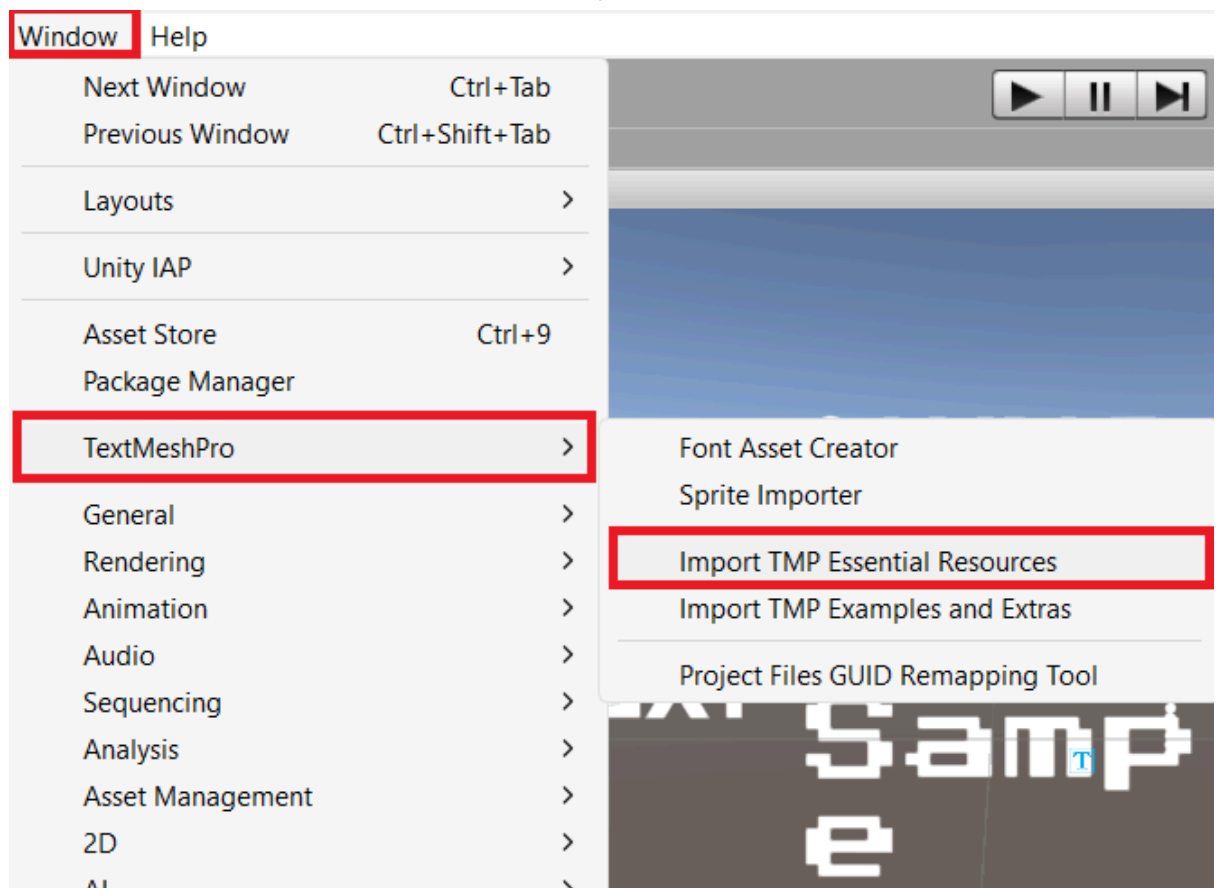
3. Перейти в Assets и удалить папку с плагином.



4. Переустановить плагин:

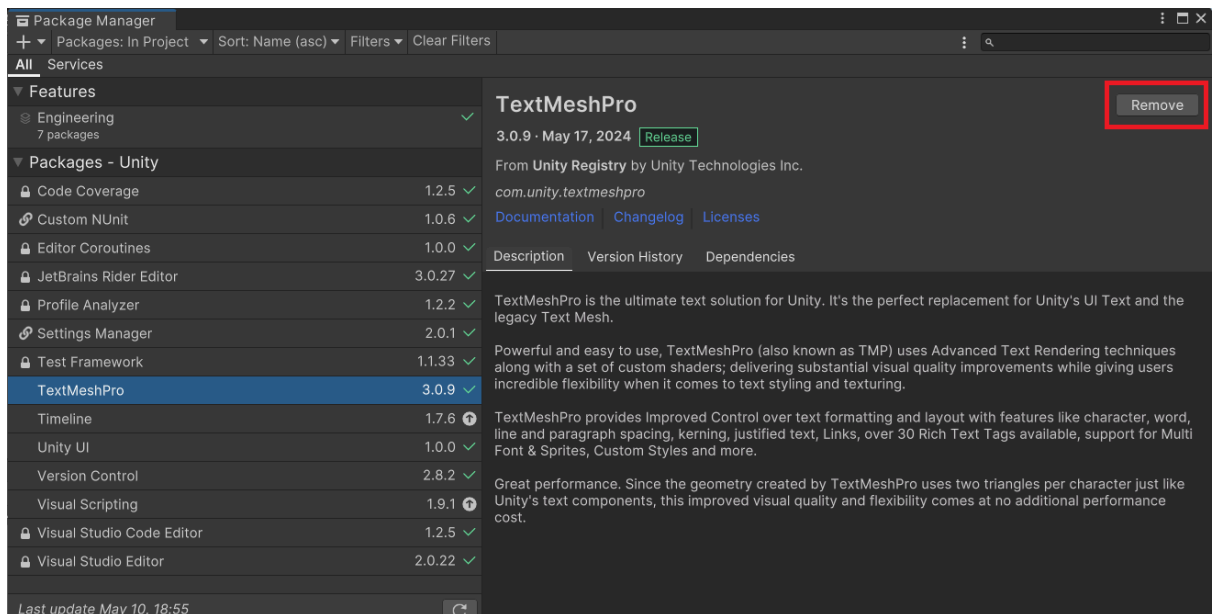
Window → TextMeshPro → Import TMP Essential Resources.

Затем в появившемся окне щелкнуть Import.

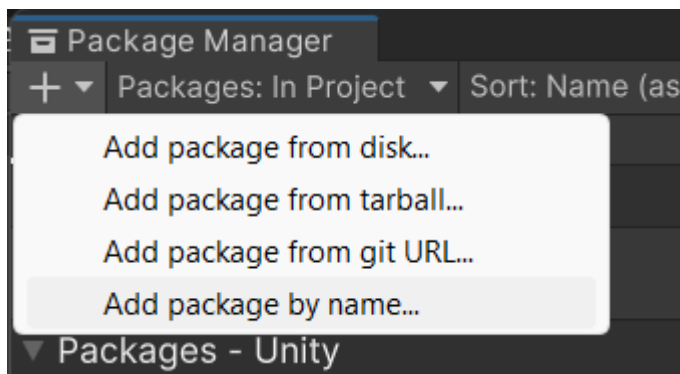


Unity 2020-2022. Как сменить версию плагина через Package Manager:

1. Window → Package Manager
2. В левой части окна выбрать TextMeshPro → нажать кнопку Remove в правом верхнем углу окна. Подтвердить удаление.



3. Нажать на плюсик, выбрать вариант добавления плагина.



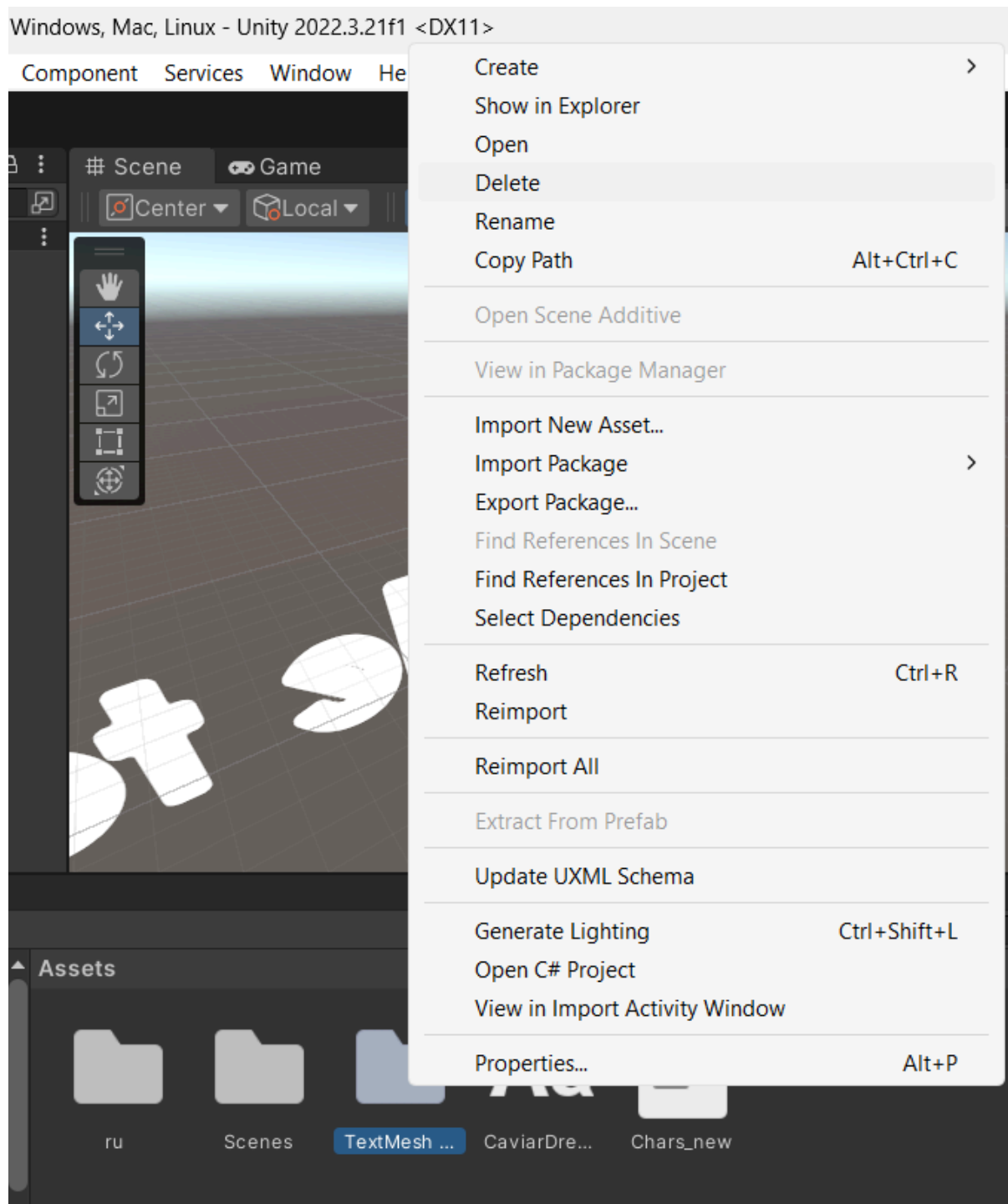
Например, для добавления по имени (Add package by name...), укажите следующие данные:

Name – com.unity.textmeshpro

Version – нужная версия в формате 1.0.26 ([список версий](#))

Нажать кнопку Add

4. Перейти в Assets и удалить папку плагина, если она там есть.



5. Переустановить плагин:

Window → TextMeshPro → Import TMP Essential Resources.
Затем в появившемся окне щелкнуть Import.

Как подобрать правильную версию TMPPro:

1. Извлеките структуру **TMP_FontAsset** из файлов вашей игры.

Вам понадобится утилита **dumper_dll_class** от dragonzh,

скачать её можно [тут](#) в шапке по ссылке “*UnityEX_Soft*”.
Распакуйте утилиту в любое место. Затем создайте рядом с папкой небольшой батник:

```
@echo off
set /p VERSION=Enter Unity version (4 digits):
"dumper_dll_class\dumper_dll_class.exe" ".\Managed"
"Unity.TextMeshPro.dll" "TMPPro" "TMP_FontAsset"
"%VERSION%"
pause
```

Что делает данный файл? Он запрашивает версию движка, а затем извлекает структуру. Если версия 2022.3.21f1, вы можете ввести либо 2022.3.21, либо 2022.3.21.1 (буква f заменяется на точку).

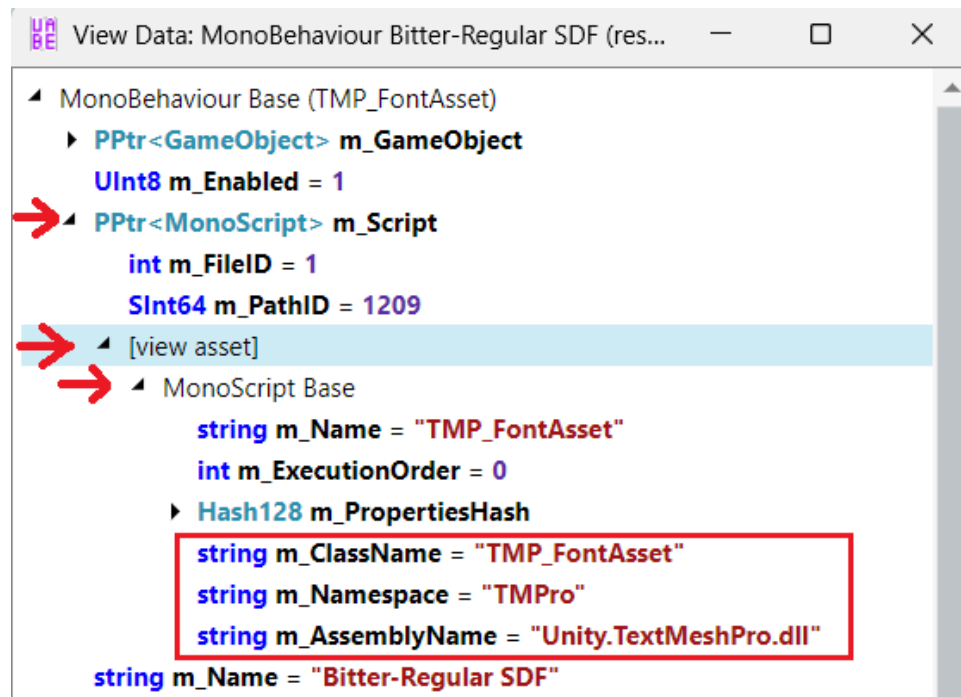
Рядом с батником нужно также положить папку Managed из Data, либо создать фиктивные библиотеки, если игра IL2Cpp ([подробнее](#)).

Формат команды:

```
dumper_dll_class.exe {PathManaged}
{m_AssemblyName} {m_Namespace} {m_ClassName}
{VersionUnity}
```

Возможно потребуется подкорректировать батник. Узнать имя сборки, пространства и класса можно при помощи UABEA:

- Открыть юнити файл, где лежат SDF шрифты, найти **MonoBehaviour** объект с разметкой (обычно имеет SDF в имени). Нажать **View Data** и раскрыть блок **m_Script**.



После запуска батника появится текстовик со структурой.

2. Сравните файлы структур.

[Вот тут](#) я извлек структуры некоторых версий TextMeshPro. Просто сравниваете их со своей структурой, чтобы найти максимально похожую. Это будет удобнее делать при помощи специальных сайтов и программ (я например использую **WinMergeU**). Затем вам понадобится установить движок юнити и ту версию плагина, которая вам подошла исходя из структуры.

D:\Утилиты для моддинга\Unity\dumper_dll_class\Job Simulator - TMP_FontAsset.txt	D:\Утилиты для моддинга\Unity\dumper_dll_class\1.4.1 - TMP_FontAsset.txt
0 SInt64 m_PathID	0 SInt64 m_PathID
0 int materialHashCode	0 int materialHashCode
1 string m_Version	1 string m_Version
1 string m_SourceFontFileGUID	1 string m_SourceFontFileGUID
0 PPtr<\$Font> m_SourceFontFile	0 PPtr<\$Font> m_SourceFontFile
0 int m_FileID	0 int m_FileID
0 SInt64 m_PathID	0 SInt64 m_PathID
0 int m_AtlasPopulationMode	0 int m_AtlasPopulationMode
0 FaceInfo m_FaceInfo	0 FaceInfo m_FaceInfo
1 string m_FamilyName	0 int m_FaceIndex
1 string m_StyleName	1 string m_FamilyName
0 int m_PointSize	1 string m_StyleName
0 float m_Scale	0 int m_PointSize
0 float m_LineHeight	0 float m_Scale
0 float m_AscentLine	0 float m_LineHeight
0 float m_CapLine	0 float m_AscentLine
	0 float m_CapLine

3. Сделайте шрифты.

Создайте SDF на движке как обычно. Извлеките из сборки

свои SDF, сделайте перенос значений либо из своего дампа в оригинальный, либо наоборот. Если в предыдущем шаге структура немного отличалась, вам надо подогнать её под оригинал, например, удалив некоторые поля в дампе или преобразовав типы данных.

Правка SDF шрифта (@TMP_FontAsset)

1. Как изменить размер шрифта?

В дампе изменить `m_PointSize`: чем больше значение, тем меньше текст, и наоборот. Подбирать экспериментальным путем и проверять в игре.

Если вы хотите изменить размер отдельных надписей, а не всего шрифта, то смотрите ниже – правка `MonoBehaviour` текста.

2. Как изменить отступ по высоте?

В дампе править `m_LineHeight`.

3. Как изменить отступ между буквами?

Возможно поможет изменение в дампе значения `normalSpacingOffset`, `boldSpacing`

Правка SDF материала

1. Исправление ширины тени



Отредактировать `_OutlineWidth`.

2. Исправление засвечивания текста

Я так рада, что все почти закончилось. Поздравляю, милая.

Должно помочь уменьшение жирности текста (`_WeightNormal`, `_WeightBold`) или чего-то начинающегося с `_Glow`.

Правка `MonoBehaviour` текста (@TextMeshPro)

1. Как изменить размер строки?

Править свойство `m_fontSize` в дампе, где прописана нужная надпись.

В случае, когда `m_enableAutoSizing` равен 1 (включено автомасштабирование по минимальному-максимальному размеру), вам понадобится отредактировать значения `m_fontSizeMin` и `m_fontSizeMax` соответственно.

Иногда автосайз делает всё только хуже, в таком случае приходится регулировать `m_fontSize` вручную.

2. Как изменить отступ между строками?

В дампе нужных надписей править одно из значений: `m_HorizontalAlignment`, `m_VerticalAlignment`.

Теги для юнити строк `TextMeshPro`, сработают также для legacy text (`Font`), но возможно не все: [официальная документация Unity](#).

Пример использования тега:

Тут <size=20> большой </size> текст

(Размер шрифта будет увеличен только для одного слова)

Хитрости

- **Ускорение правок**

Исправлять поля дампов и перезапускать игру для тестирования может быть довольно утомительно, особенно когда текст криво отображается во многих местах. Что делать в таком случае? Как я писал ранее, есть плагины, позволяющие изучить свойства объектов со сцены и протестировать их изменение.

Подробный tutorial:

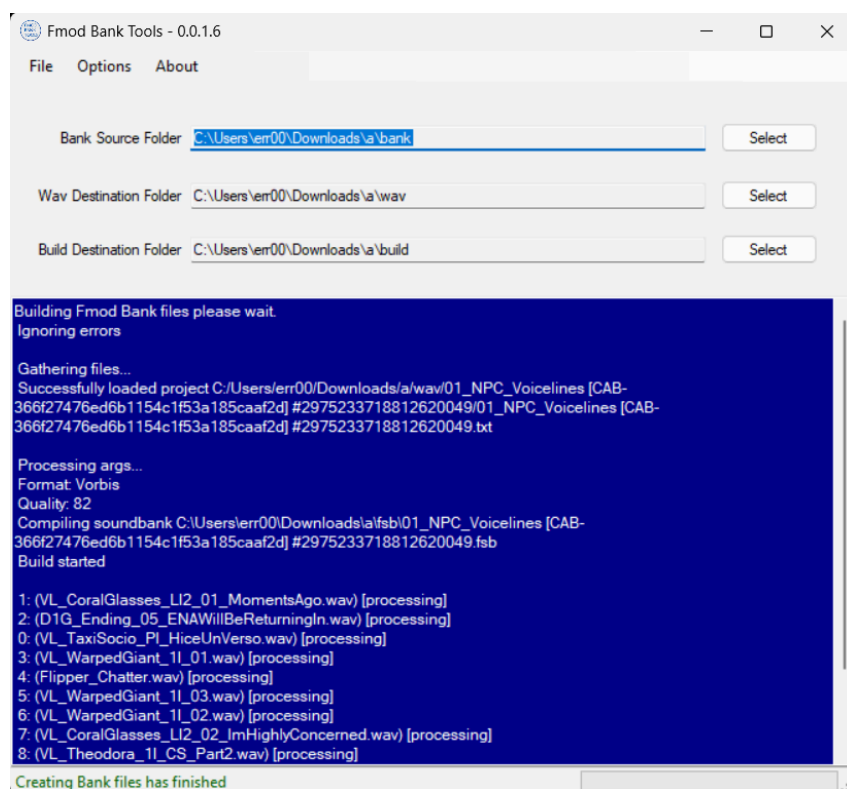
[UnityExplorer – изучение объектов запущенной игры](#)

🎵 Аудио и видео замена

Fmod bank – распаковка/пересборка

Понадобится утилита [Fmod Bank Tools](#) ([копия Google диск](#))

(Опенсорсная альтернатива, которая работает на порядок быстрее, но не кодирует аудио в wav, а извлекает в оригинальном формате – [FenixProFmod](#))



Интерфейс Fmod Bank Tools

Важно:

Утилиту надо распаковать по пути, не содержащему кириллицы.

Описание использования программы:

- Поместите **.bank** файлы, из которых хотите извлечь аудио, в папку **bank**. После извлечения вы найдете файлы **.wav** в папке **wav**, внутри которой будет другая папка с таким же именем, как у bank файла.
- Чтобы пересобрать банки, замените wav-файлы своими. При необходимости отредактируйте txt-файл, указав новые имена wav-файлов. После этого нажмите **File** → **Build**. Готовые файлы банков появятся в папке **build**.
- При замене wav-файлов убедитесь, что они имеют следующий формат:
 - Тип: WAV
 - Формат: PCM
 - Глубина битов: 16 бит

- Частота дискретизации: такая же, как у извлеченных wav, обычно 44 кГц или 48 кГц.

Вот батник (**fix_wav.bat**), который приведет WAV-файлы к правильному формату с помощью FFmpeg:

```
@echo off
setlocal enabledelayedexpansion

:: Проверка наличия FFmpeg
where ffmpeg >nul 2>nul
if %errorlevel% neq 0 (
    echo FFmpeg не найден! Убедитесь, что он установлен
    и добавлен в PATH.
    pause
    exit /b
)

:: Создаем папку "fixed" для исправленных файлов
mkdir fixed 2>nul

:: Обработываем все WAV-файлы в текущей папке
for %%F in (*.wav) do (
    echo Обработываю: %%F
    ffmpeg -i "%%F" -ar 44100 -acodec pcm_s16le
    "fixed\%%F"
)

echo Готово! Исправленные файлы находятся в папке
"fixed".
pause
```

Как использовать:

1. Убедитесь, что FFmpeg установлен и добавлен в PATH.
2. Поместите **fix_wav.bat** в папку с WAV-файлами.
3. Запустите **fix_wav.bat**, и исправленные файлы появятся в папке **fixed**.

Этот батник сохраняет частоту дискретизации такой же, как у исходного файла, если она уже 44/48 кГц. Если хотите, чтобы он принудительно ставил 48 кГц, замените `-ar 44100` на `-ar 48000`.

AudioClip/VideoClip: запаковка через UABEA + автоконвертация

Заранее хочу предупредить, что данный способ не подойдёт для бандлов. Там скорее всего сработает первый способ, за исключением того, что `m_Source` надо указывать в другом формате: `archive:/CAB-ee65fb0ba3e2ec61b1507faf89a44e59/CAB-ee65fb0ba3e2ec61b1507faf89a44e59.resource`

Способ №1. Через движок приблизительной версии, что и игра

Вот видео-тutorиал, где всё подробно рассказано и показано:

 [UABE | Modify Unity Audio Files \(v2.2 Stable D\) Part 2](#)

Краткий пересказ действий:

- Создать юнити проект, импортировать аудио/видео, сделать сборку.
- При помощи **UABEA** извлечь из сборки текстовые дампы ваших `AudioClip` и `VideoClip`. В каждом файле отредактировать поле `m_Source`, указав любое другое имя. Допустим у вас было `sharedassets0.resource`, вы можете заменить на **RuDubbing.resource**.
- В папке сборки переименовать ресурсный файл в соответствии с предыдущим пунктом (в моем случае с `sharedassets0.resource` на **RuDubbing.resource**). После чего перенести его в папку игры рядом с другими assets файлами.
- Открыть оригинальные файлы игры через **UABEA** и вручную импортировать дампы ранее извлеченные из сборки и отредактированные. Массовый импорт тут не пройдет, поскольку имена дампов содержат другие айди.
- Не забудьте сохранить изменения.

Зачем нужно переименовывать ресурс?

Чтобы ваши аудио и видео вытягивались из отдельного ресурса, а всё остальное – из оригинального.

Способ №2. Без движка, но возможно займет больше времени

Шаг 1. Для начала вам необходимо подготовить все файлы:

1. Аудио перекодировать в fsb5 при помощи [FSB5.Converter](#)
`FSB5.Converter.exe -a audio_path`

Массовая конвертация:

- Создайте папку **audio** рядом с программой.
 - Поместите в неё свои **.wav** файлы.
 - Скачайте [батник](#), поместите рядом с программой и запустите.
 - После завершения появятся файлы с расширением **.fsb**
2. Видео перекодировать в оригинальный формат. Полезные ссылки на данную тему:
 - [VP8 кодировщик](#), сохраняющий прозрачность
 - [Пайтон скрипт](#), чтобы собрать информацию обо всех видео-кодеках для видео из текущей папки (инструкция по использованию написана в самом скрипте)
 - [Команды ffmpeg для перекодировки видео вручную](#)
 - [Проблемы, связанные с заменой видео](#)

Автоматическая конвертация [*похоже, требуется доработка скрипта*]:

- Скачайте [утилиту](#).
- В **original_video** поместите оригинальные видео, а в **new_video** – свои. Имена у файлов должны быть одинаковыми (включая расширения), а также проверьте, чтобы совпадали ширина и высота.
- Запустите пайтон скрипт, затем он проверит формат оригинального видео и применит его к вашему, сохранив в папке **output**.

Примечание: ffmpeg должен быть скачан и добавлен в системную переменную `Path`. Также понадобится скачать модуль: `pip install python-ffmpeg`

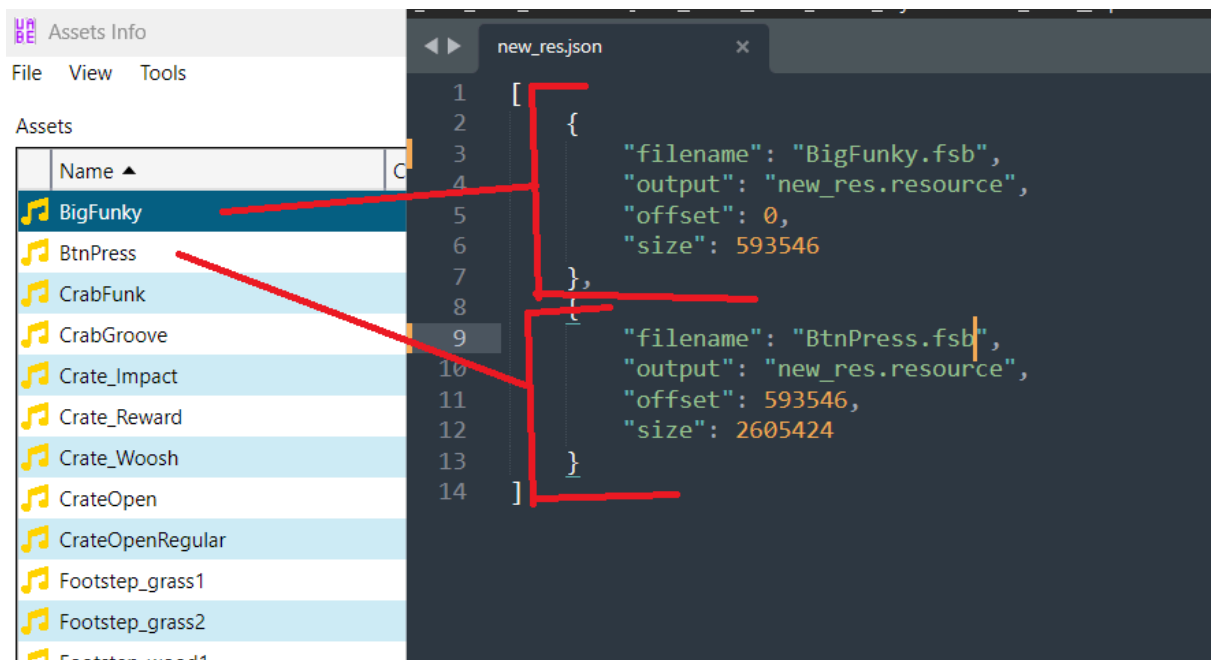
Шаг 2. Записать в двоичный файл:

- Полученные fsb аудио и перекодированные видео поместите в папку **Content** утилиты [resource_packer](#), запустите батник для записи в новый файл или дозаписи в существующий.
- В папке появится двоичный файл (resource) и json с данными замены.

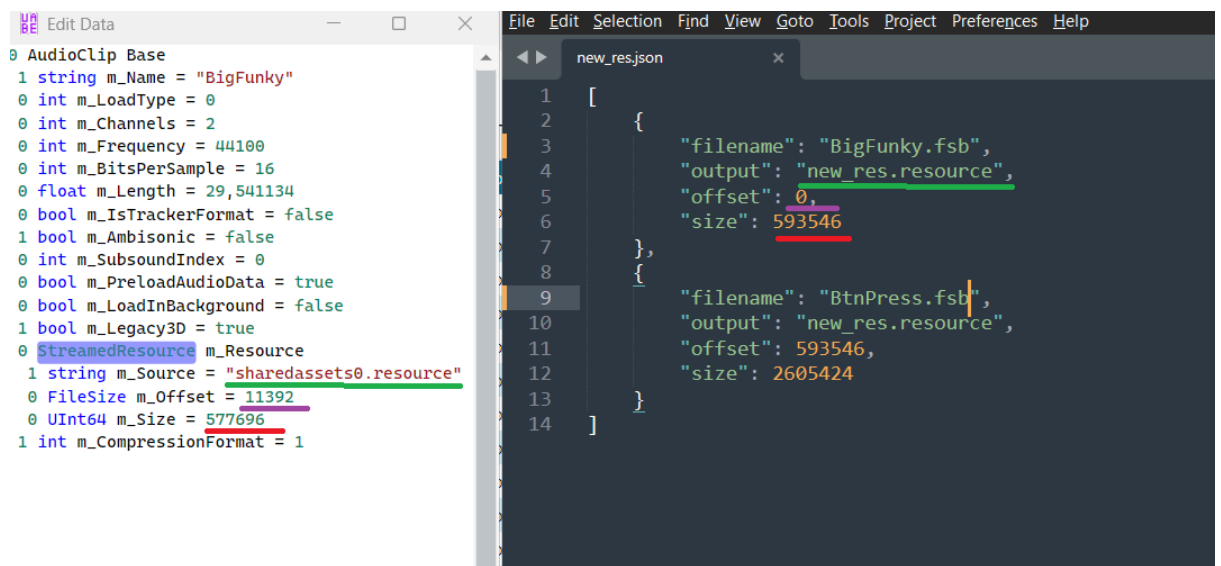
Шаг 3. Патчинг дампов

Вручную:

- Откройте json любым удобным редактором, а в **UABEA** – юнити архивы, содержащие информацию о аудио ([AudioClip](#)) или видео ([VideoClip](#)), которые вы хотите подменить. Для удобства вы можете отсортировать столбцы или применить фильтры в программе.



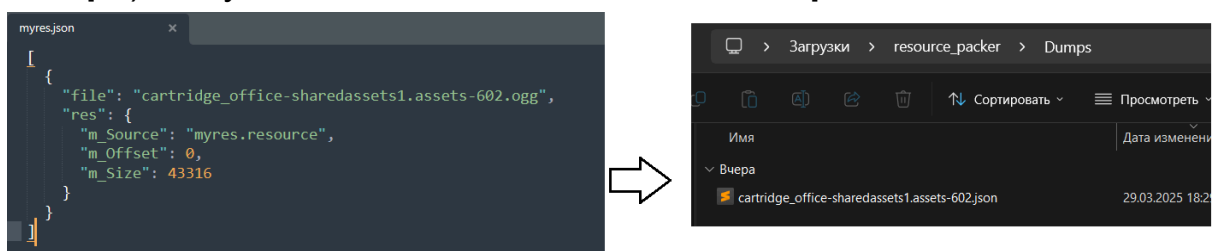
- Поочередно открываете свойства объектов (**Edit Data**) и редактируете блок `StreamedResource`, перенося туда информацию из json.



- По завершению редактирования файла, не забудьте его сохранить: **File** → **Save (Ctrl+S)**.
- И самый последний шаг – файл **myres.resource** перенесите в папку **Data** вашей игры.

Автоматически:

- В папку **Dumps** поместить json дампы оригинальных **AudioClip** и **VideoClip**, запустить батник для патчинга дампов. Он осуществит перенос данных, ориентируясь по именам как показано на скриншоте (имя берется из ключа *file*, расширение заменяется на json и файл ищется в папке **Dumps**). Результат появится в папке **NewDumps**.



VideoClip: решение проблем замены

1. Исключение ffmpeg: “Не удастся найти указанный файл” при запаковке видео патчером:

```

Name: mv_11-
Type: VideoClip
Script: None
PathID: 9215919011183476036
Source: movie\CAB-bce0a639cb2bd429155a356006be46ff

[ERR] Regular file import failed:
Traceback (most recent call last):
  File "core\ObjectHandler.py", line 234, in _handle_import
  File "classes\VideoClip.py", line 18, in import_
  File "patches\VideoClip.py", line 40, in _VideoClip_set_video
  File "ffmpeg\probe.py", line 20, in probe
  File "subprocess.py", line 966, in __init__
  File "subprocess.py", line 1435, in _execute_child
FileNotFoundError: [WinError 2] Не удается найти указанный файл

```

Даже если вы просто заменяете видео без перекодировки, патчер всё равно использует ffmpeg для того, чтобы прочитывать основную информацию о видео, такую как ширина, высота, наличие видеопотока. Отсутствие ffmpeg вызовет исключение.

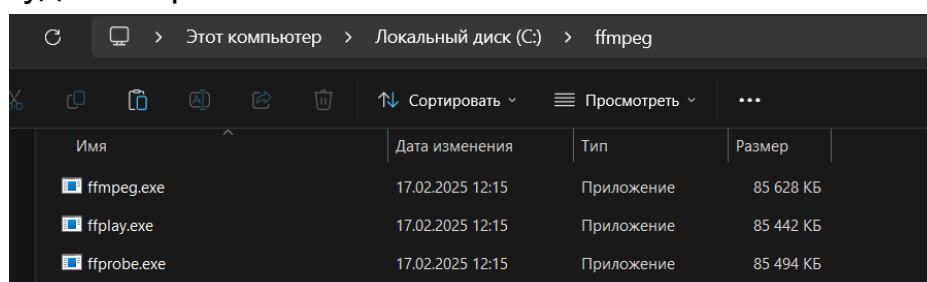
FFmpeg — это мощная библиотека и утилита для работы с мультимедиа. Она позволяет конвертировать, сжимать, редактировать и стримить аудио- и видеофайлы, поддерживая множество форматов.

Как решить эту проблему:

- Перейдите по [ссылке](#) и скачайте **ffmpeg-git-essentials.7z**



- Откройте архив, перейдите в папку **bin**, где будет лежать три exe. На системном диске создайте папку **ffmpeg** и скопируйте туда эти файлы.



- Нажмите **Win + X** и выберите **Система** или найдите в Пуске **Этот компьютер → Свойства**.
- Нажмите **Дополнительные параметры системы**.
- Во вкладке **Дополнительно** нажмите **Переменные среды**.
- В нижнем разделе **Системные переменные** найдите переменную **Path** и нажмите **Изменить**.
- Нажмите **Создать** и введите **C:\ffmpeg**
- Нажмите **ОК** для сохранения изменений.
- Перезапустите компьютер.
- Попробуйте снова запаковать видео патчером.

2. Если видео и аудио кодеки совпадают с оригинальными (что можно проверить утилитой **MediaInfo**), но после перепакровки у видео отсутствует звук во время игры, то возможно поможет данная команда исправить ваше видео:

```
ffmpeg -i input.webm -c:v copy -c:a copy output.webm
```

Команда копирует потоки в новый корректный контейнер. Далее видео нужно паковать патчером без опции **--transcode**.

VideoClip: команды ffmpeg для перекодировки видео

1. WMV (**.asf** контейнер)

```
ffmpeg -i input_video -c:v wmv2 -c:a wma2 output.asf
```

2. VP9 (**.webm** контейнер)

```
ffmpeg -i input_video -c:v libvpx-vp9 -c:a libvorbis output.webm
```

3. VP8 (**.webm** контейнер)

```
ffmpeg -i input_video -c:v libvpx -c:a libvorbis output.webm
```

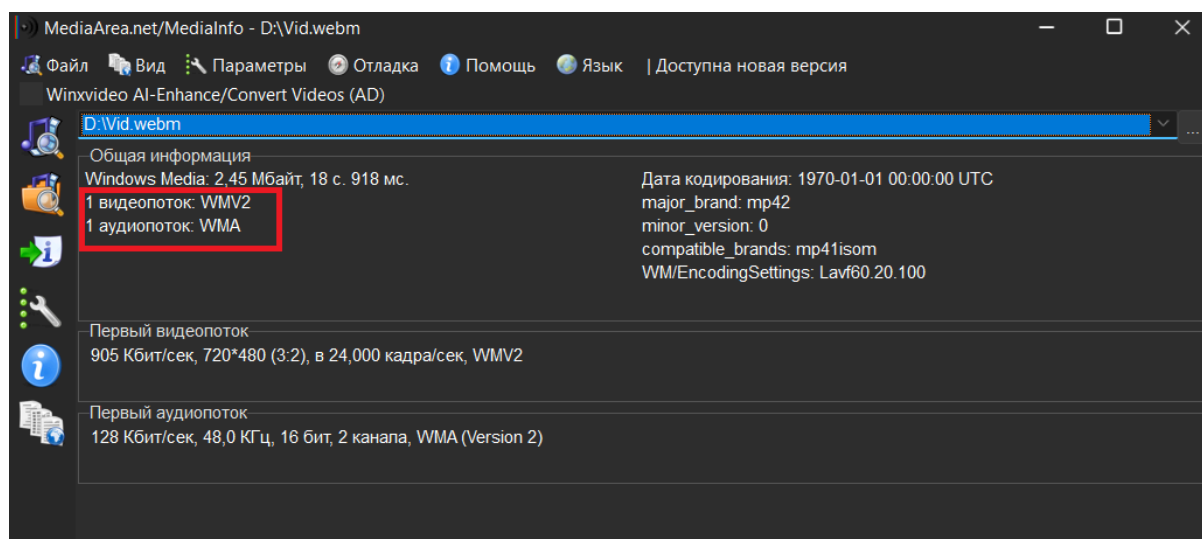
4. MP4 (**.mp4** контейнер)

```
ffmpeg -i input_video -c:v libx264 -c:a aac  
output.mp4
```

Пояснения к командам

- **-c:v** указывает видеокодек, а **-c:a** — аудиокодек.
- Эти команды обеспечивают минимально необходимые параметры для использования указанных кодеков.

Чтобы узнать, какой формат вам нужен, откройте извлеченное из игры видео в программе **MediaInfo** и обратите внимание на поле **видеопоток**:



Плагины VerInEx

XUnity.AutoTranslator – автоперевод юнити игр

XUnity Auto Translator — это плагин для перевода текста в играх на Unity в реальном времени. Он перехватывает текстовые строки, отправляет их в онлайн-переводчики (Google, DeepL и др.) или

использует локальные словари, а затем заменяет оригинальный текст переведённым.

Ещё раз, этот плагин извлекает текст и текстуры по ходу игры, с возможностью автоперевода. Это означает, что вам нужно будет пройти игру вдоль и поперек для извлечения всех игровых строк. Полученную текстовую базу можно также править вручную через текстовый файл и предоставлять другим пользователям в качестве русификатора. Однако, опять же, переводить таким способом не слишком удобно и мало гибкости. Советую только если вы не знаете английский в достаточной мере и решили поиграть с автопереводом, для чего, собственно, этот плагин и предназначался. Если некоторые строки остались без перевода, значит они задаются через код. Как вариант, пробить строку через **dnSpy** и добавить её вручную в текстовую базу **XUnity.Autotranslator**.

Принцип работы:

Строки, которые подаются на вывод в объекты типа TextMeshPro, перехватываются, и заносятся в _AutoGeneratedTranslations.txt (формат: оригинал=перевод). Это могут быть как строки, прописываемые в объектах MonoBehaviour, так и задаваемые через код.

В соответствии с выбранным переводчиком, отправляется запрос на перевод извлеченных строк. Затем он прописывается в вышеупомянутом текстовике и в игре текст оригинала подменяется на перевод.

Недостаток такого подхода в том, что если текст задается динамически (т.е. всячески комбинируется), то его будет сложно подменить плагином. Например: “статическая строка” + значение переменной, которое может быть целым числом от 1 до 100. Чтобы такую строку перевести плагином, вам понадобилось бы прописать все 100 вариантов в текстовой базе.

Установка через BepInEx:

1. Перейдите на страницу [релизов](#) и скачайте **BepInEx** под свою разрядность системы, то есть файл *BepInEx_win_xXX_X.X.X.X.zip*.

Как узнать разрядность на Windows 11:

Параметры → Система → О системе → обратить внимание на поле Тип Системы.

2. Распакуйте содержимое архива в корневую папку игры, рядом с файлом .exe.
3. Запустите игру один раз, чтобы создались папки plugins, config, patchers и другие.
4. Скачайте [плагин XUnity.AutoTranslator](#) для **BepInEx** и распакуйте в корневую папку игры.
5. Запустите игру, чтобы плагин создал файл конфигурации по следующему пути:

BepInEx\config\AutoTranslatorConfig.ini

Установка через ReiPatcher:

1. На странице релизов скачайте *XUnity.AutoTranslator-ReiPatcher-X.X.X.zip*
2. Распакуйте содержимое в корневую папку игры и запустите *SetupReiPatcherAndAutoTranslator.exe*.
3. Запустите созданный программой ярлык {GameExeName} (*Patch and Run*).
4. После первого запуска игры программа должна создать *{GameDirectory}\AutoTranslator\Config.ini*.
5. Настроить файл конфигурации.
6. Игра готова. Игру можно запускать через основной exe.

Что редактировать в файле кофигурации:

- *Language=ru* (на какой язык переводить)

- `FromLanguage=en` (язык в игре)
- `MaxCharactersPerTranslation=500`
- `IgnoreWhitespaceInDialogue=False`
- `IgnoreWhitespaceInNGUI=False`
- `OverrideFont=` (отвечает за замену шрифтов. Пример: `OverrideFont=Corbel Bold Italic`)
- `OverrideFontTextMeshPro=` (отвечает за замену шрифтов в играх с TextMeshPro)
- `Service=GoogleTranslate` — сервис перевода (можно изменить на `DeepL`, `Papago` и др.)
- `EnableSubstitution=true` — использование локального словаря.

Горячие клавиши:

- `ALT + 0`: включить интерфейс **XUnity AutoTranslator**.
- `ALT + T`: включить/выключить переведённую версию текстов.
- `ALT + R`: перезагрузить файлы переводов. Полезно, если вы изменяете текстовые и текстурные файлы на лету. Не гарантируется работа для всех текстур.
- `ALT + U`: ручной захват. Захват по умолчанию не всегда подхватывает текст. Плагин попытается сделать поиск вручную.
- `ALT + F`: если настроен `OverrideFont`, будет переключаться между переопределённым шрифтом и шрифтом по умолчанию.
- `ALT + Q`: перезагрузите плагин, если он был выключен.

Решение проблем:

- Вместо текста кракозябры или его вообще нет:

Игра использует SDF без кириллицы. Есть решение:
`OverrideFontTextMeshPro` отвечает за внешние шрифты в файле *Config.ini*. Шрифты нужно делать самим или скачать с github файл *TMP_Font_AssetBundles.zip*. Там 2 шрифта, распаковать в корневую папку игры. В конфиге указать:
`OverrideFontTextMeshPro=arialuni_sdf_u2018` (пример)

Дополнительные возможности:

- Использование локального словаря: можно редактировать `Translation\en.txt` и добавлять переводы вручную.
- Автоматический перевод UI: можно включить параметр `EnableUITranslation=true` в конфиге.

(Частично информация позаимствована из ZoG)

- 👉 Для продвинутых настроек смотрите документацию на GitHub.
- 👉 Страница на форуме Zone of Games с разной полезной информацией касательно плагина – [ссылка](#).
- 👉 [Наглядная инструкция](#) по установке и базовой настройке плагина.

UnityExplorer – изучение объектов запущенной игры

Данный плагин позволяет вам находить объекты со сцены, тестировать изменение различных свойств и изучать их.

Поддерживает версии Unity от 5.2 до 2021+ (IL2CPP и Mono).
К сожалению, проект на гитхабе был архивирован в 2023 году, но плагин до сих пор остается полезным при работе с большим количеством игр.

Установка:

- Скачайте [BepInEx](#), разархивируйте в корень игры, рядом с файлом .exe.
- Скачайте [UnityExplorer](#) и распакуйте .dll файлы в папку **BepInEx\plugins**.
- Запустите игру – плагин загрузится автоматически.
- Чтобы показать/скрыть интерфейс плагина, нажмите **F7**. В настройках можно включить автоскрытие GUI при запуске игры.

Тutorial по установке на русском:

▶ Установка и использование Unity Explorer через BepInEx(все с...

Тutorials по установке и использованию плагина (основное):

▶ How to install and use the UnityExplorer for House Flipper

▶ Realistic ambient lights (making a sunset) using UnityExplorer

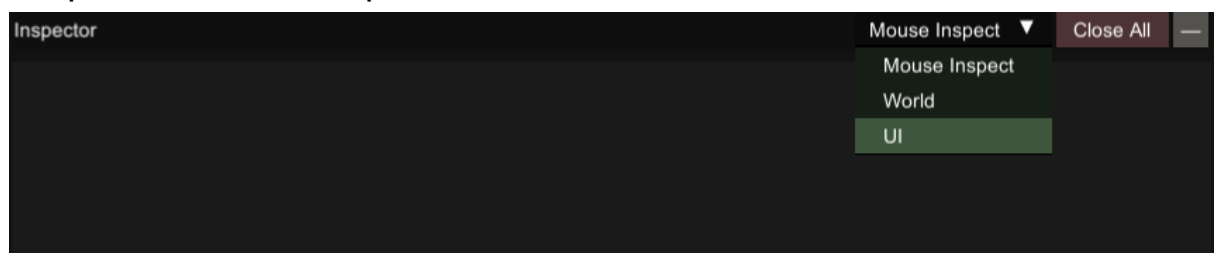
Тutorial – изучение свойств через **UnityExplorer** и написание простого **Harmony**-патча:

▶ UNITY MODDING WITH MELONLOADER PT. 2 (Changing values...

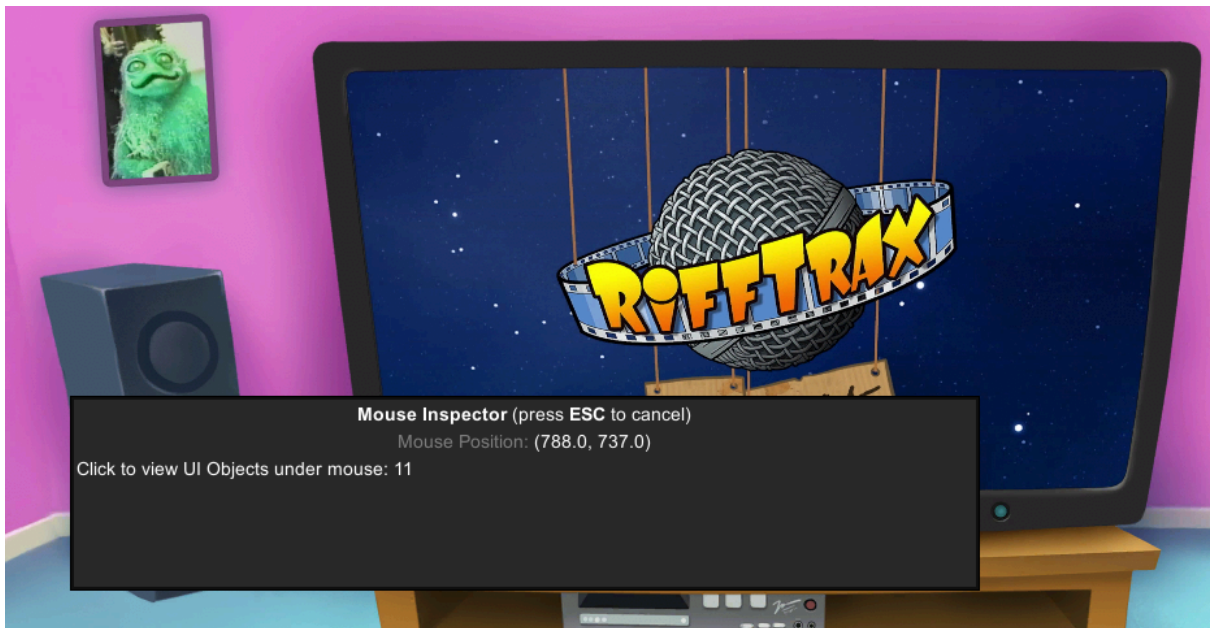
Примеры использования:

1. Узнать имя текстуры:

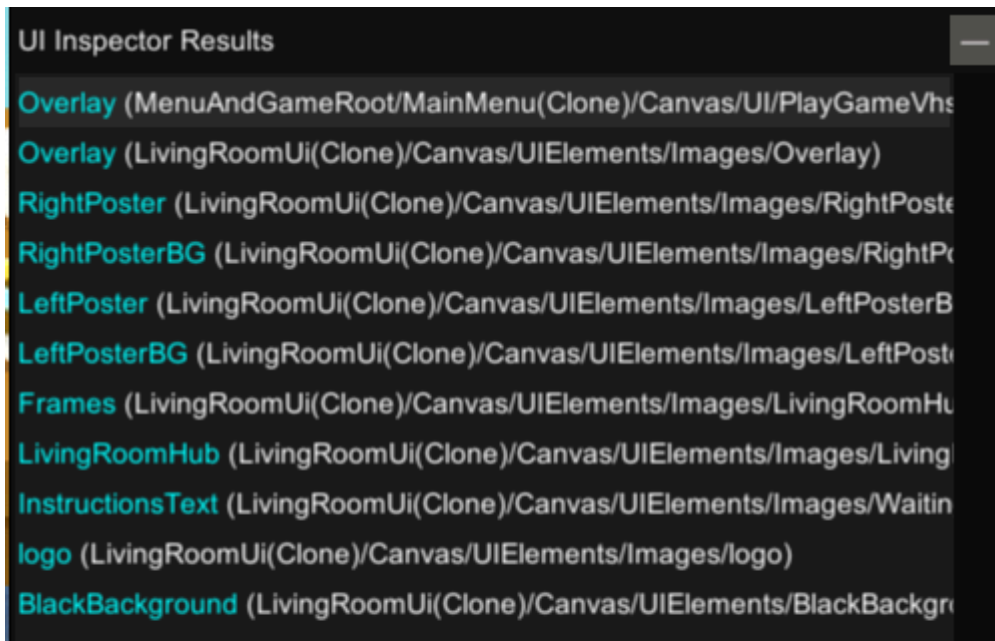
- Открыть UI инспектор.



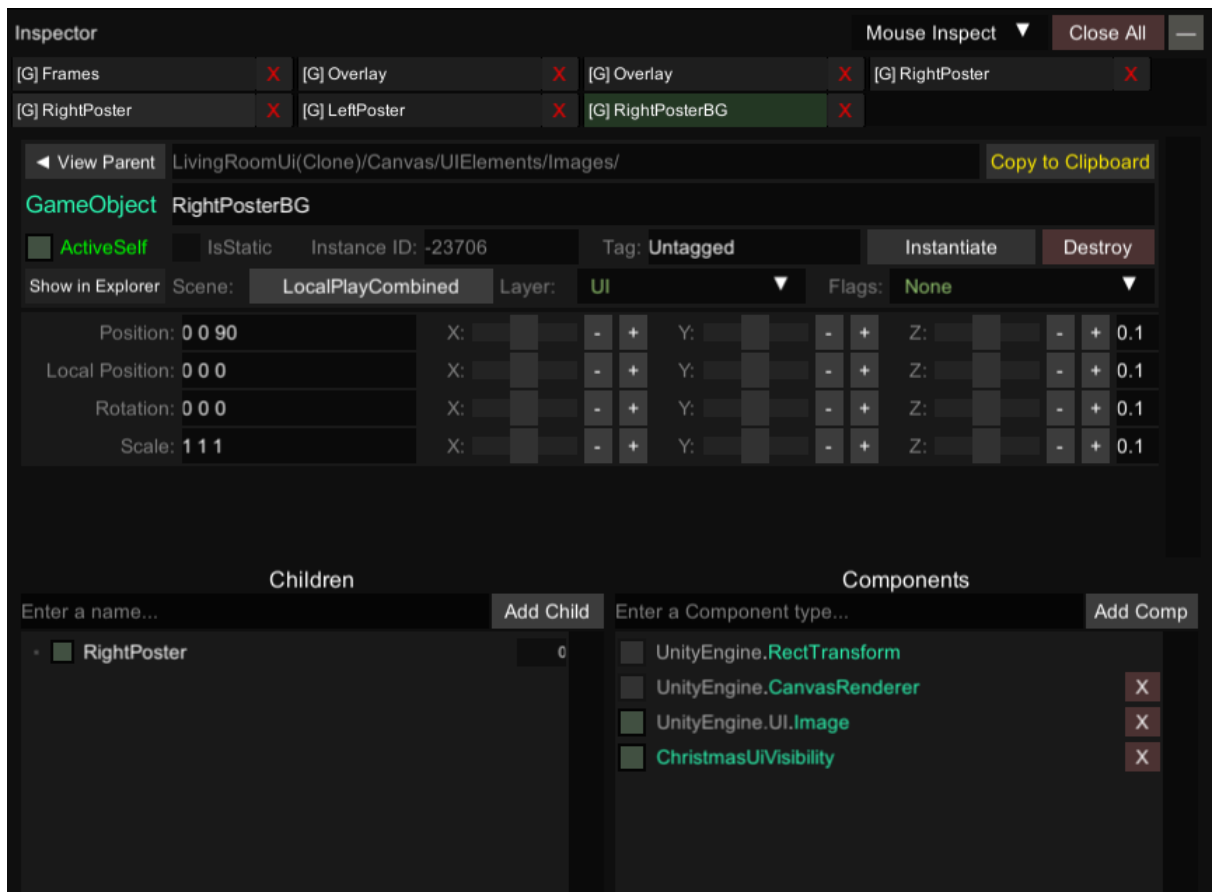
- Навести стрелку мыши на нужную картинку и нажать.



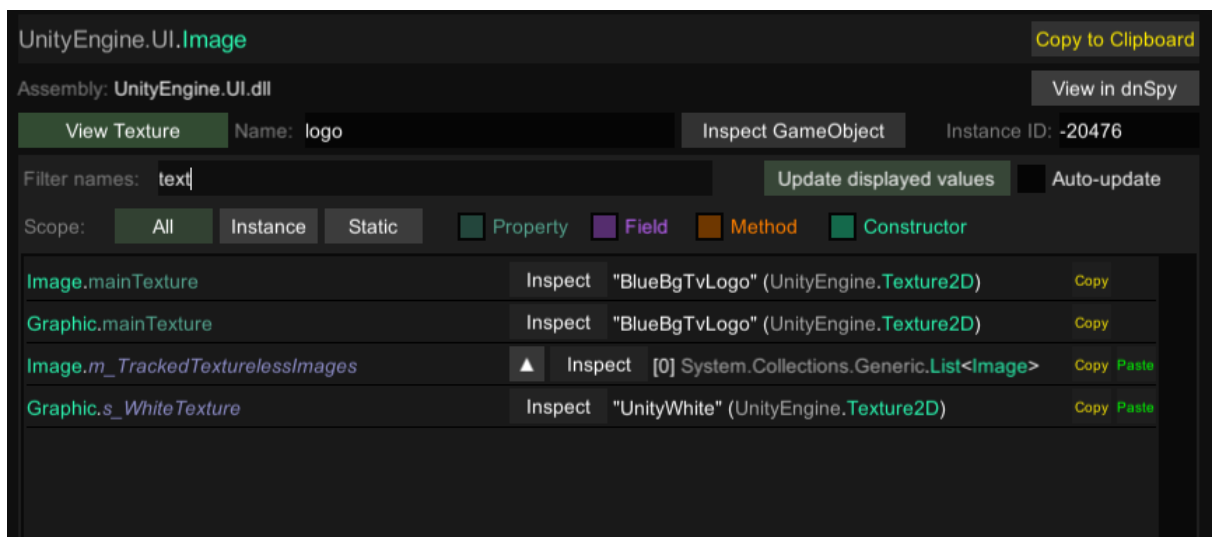
- Появится список UI объектов.



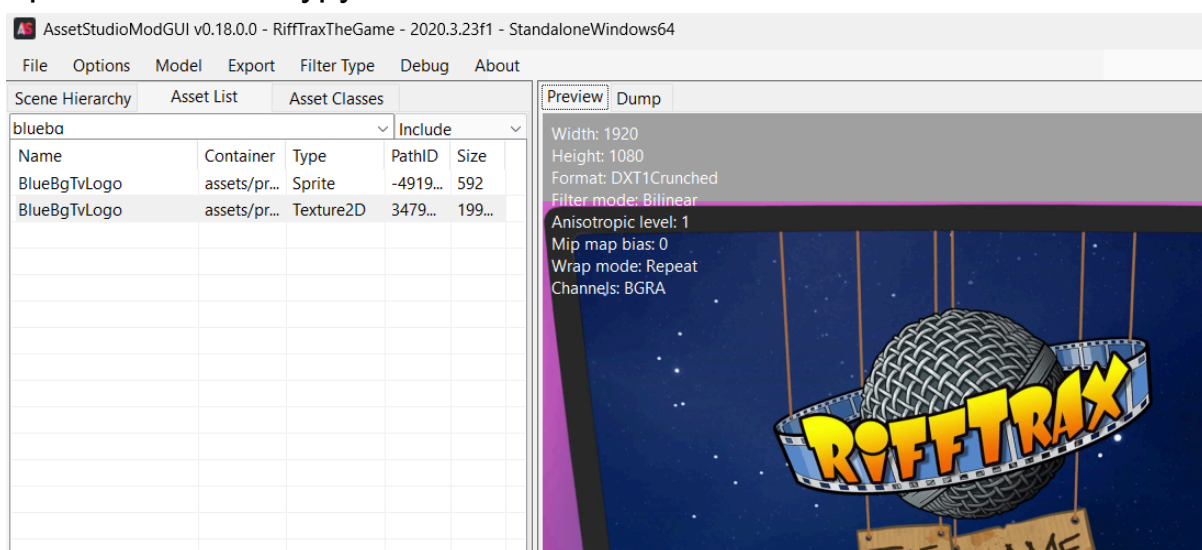
- Прокликиваем каждый объект в поиске нужного, ориентируясь по именам. При нажатии на строку списка, данные про объект заносятся в окно инспектора. Чтобы убедиться, что это правильный объект, в области **Components** снимаем галочку с **UnityEngine.UI.Image** – картинка должна исчезнуть (либо снять галочку с **ActiveSelf**, чтобы отключить весь **GameObject**). Если этого не произошло, переходим к следующему объекту, и так далее. Обратите внимание, что некоторые компоненты у объекта нельзя отключить.



- После нахождения нужного объекта, щелкаем на компонент `UnityEngine.UI.Image` в окне инспектора. Ищем значения `Image.mainTexture` (имя текстуры, из которой был взят спрайт), `Image.activeSprite` (имя текущего спрайта).

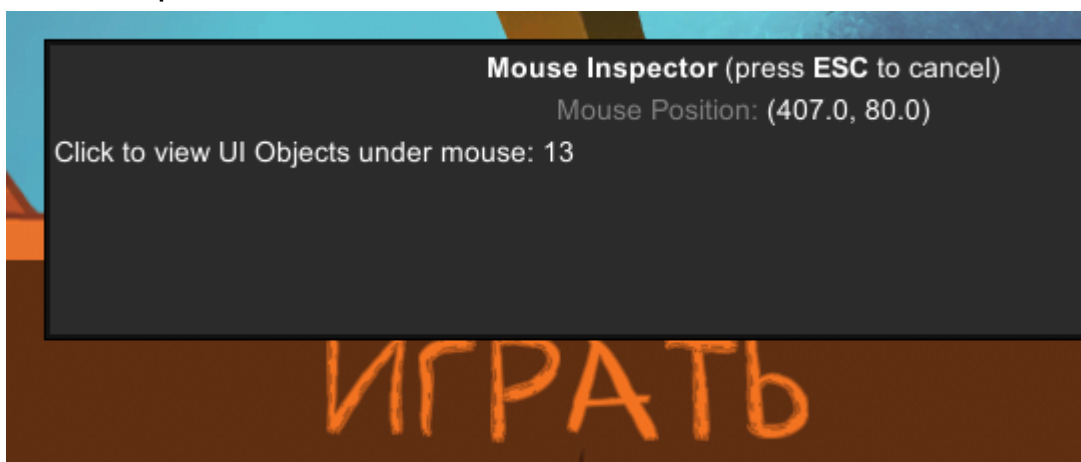


- Пробиваем текстуру по имени.

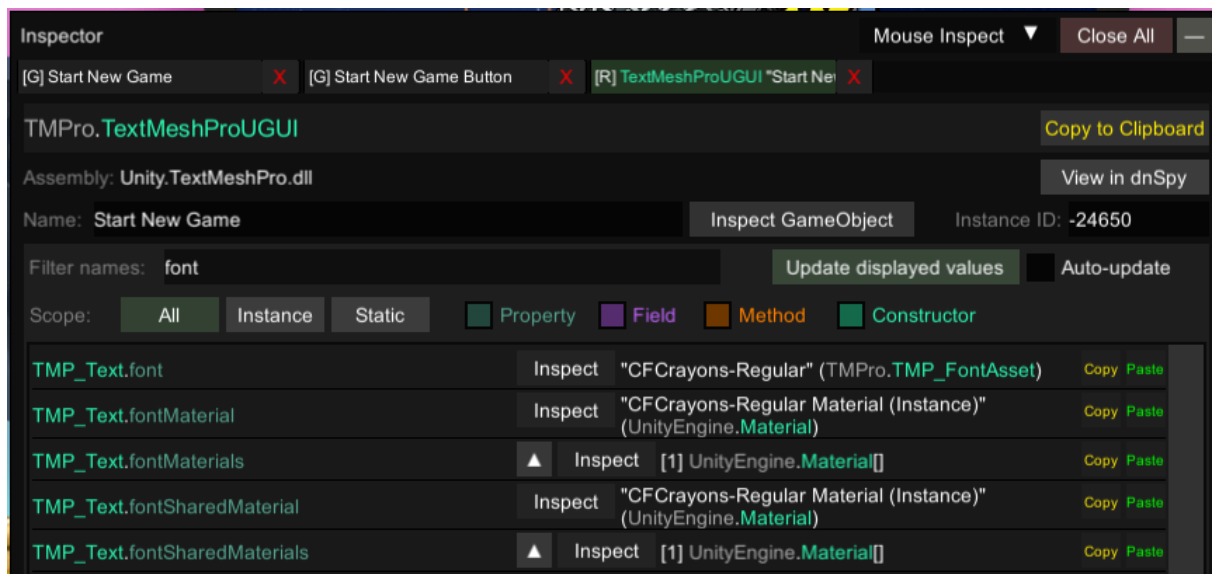


2. Узнать имя SDF шрифта:

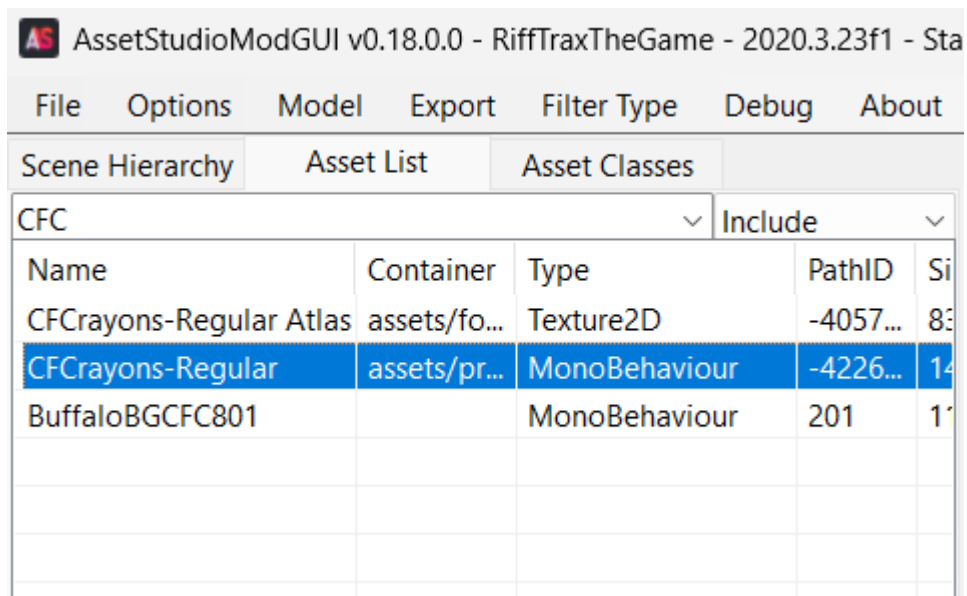
- Открываем UI-инспектор и нажимаем на какую-нибудь надпись. Важно: надо нажимать по центру надписи, иначе инспектор не захватит этот объект.



- Точно так же как и с картинками прокликиваем каждый объект из списка и проверяем, но на этот раз ищем нужный компонент `TMPPro.TextMeshProUGUI` – после снятия галочки надпись должна исчезнуть.
- После нахождения компонента, щелкаем по нему и ищем свойство `TMP_Text.font` – это и будет имя SDF (а точнее `MonoBehaviour` объекта).

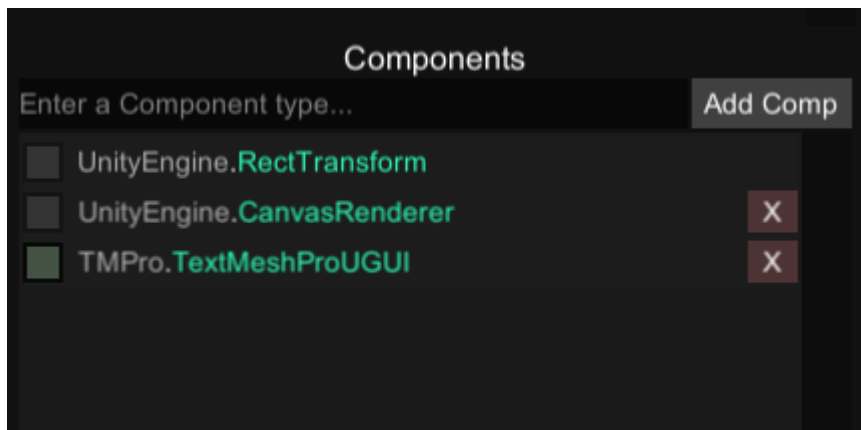


- Далее пробиваем любой удобной программой для экспорта и дальнейших манипуляций.



3. Исправление надписей:

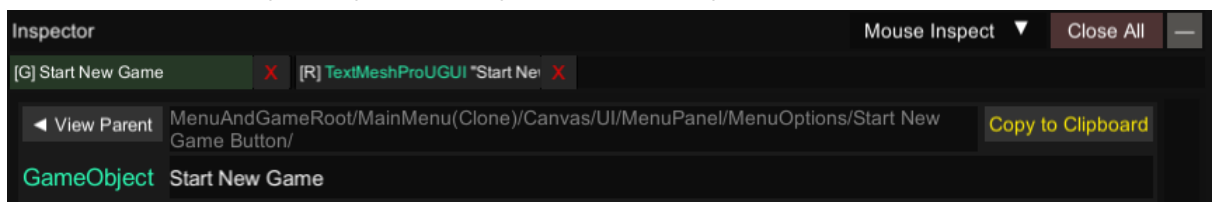
- Предположим, вы нашли объект, содержащий **TextMeshProUGUI**, который вы хотите подправить. Например вы перевели текст, но он не влез и переносится на следующую строку, либо вам нужно его немного сместить в сторону.



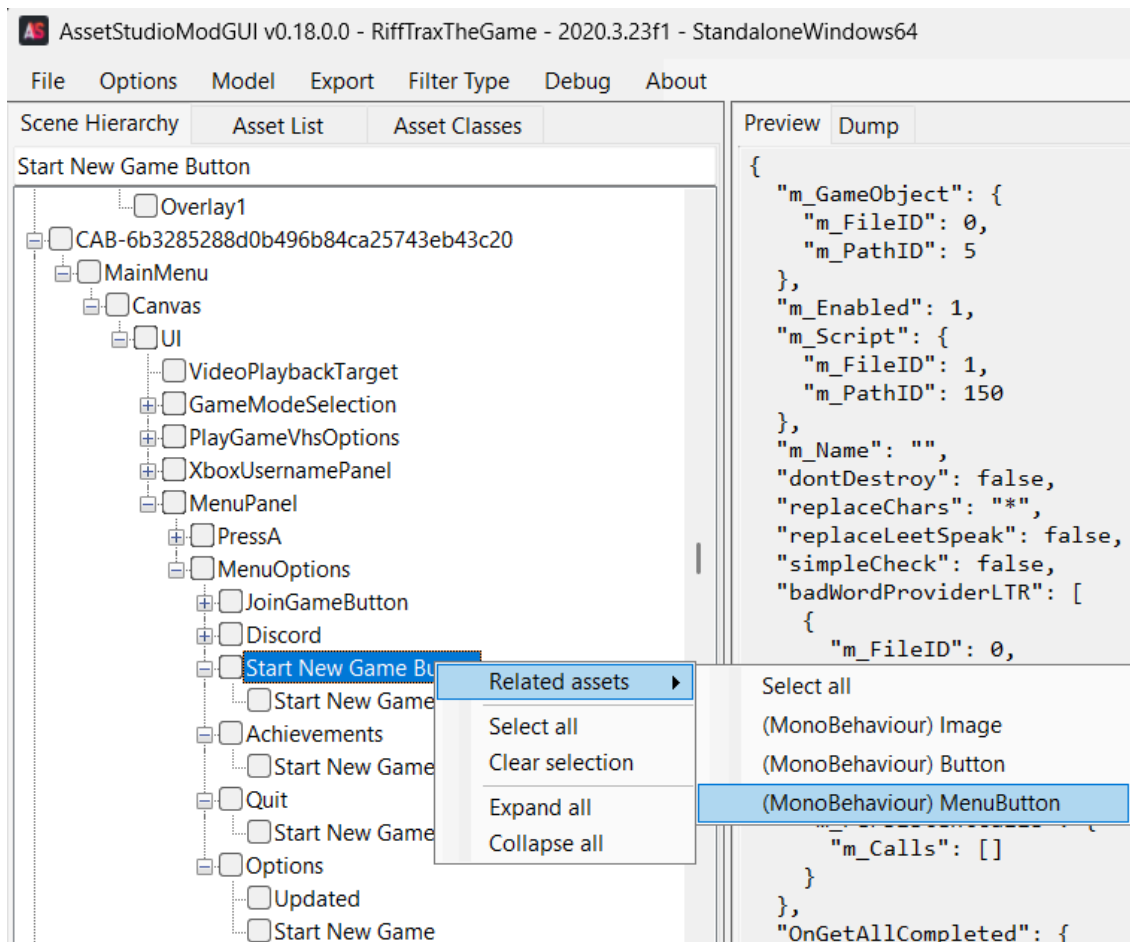
- Для изменения позиции на экране используйте компонент **RectTransform**, а для всего остального – **TextMeshProUGUI**. В окне инспектора найдите нужные свойства компонента и попробуйте их поменять до достижения нужных результатов. Далее вам нужно перенести эти исправления в соответствующий json дамп или написать плагин, где вы находите объект и меняете его свойства программно (используя **BepInEx/MelonLoader**).

Как выйти на GameObject в **UABEA/AssetStudio**:

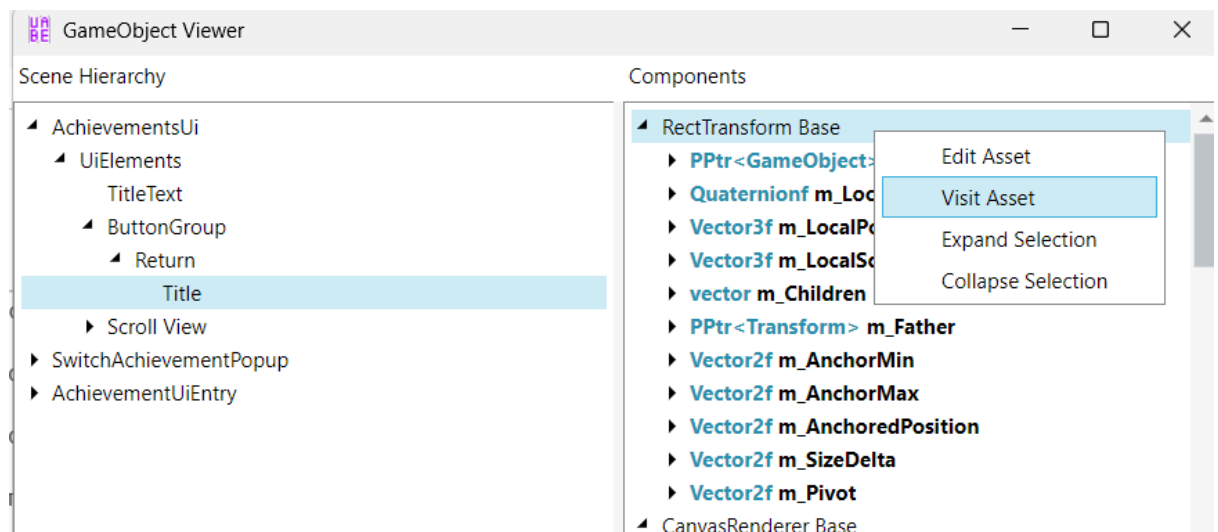
- В инспекторе будет указан путь к объекту.



- Через **AssetStudioMod** загрузите всю папку игры и на первой вкладке пробивайте объект. В строке можно прописать часть пути, переход к следующему обнаружению – по нажатию кнопки **Enter**. Дальше ПКМ по объекту → **Related assets**.

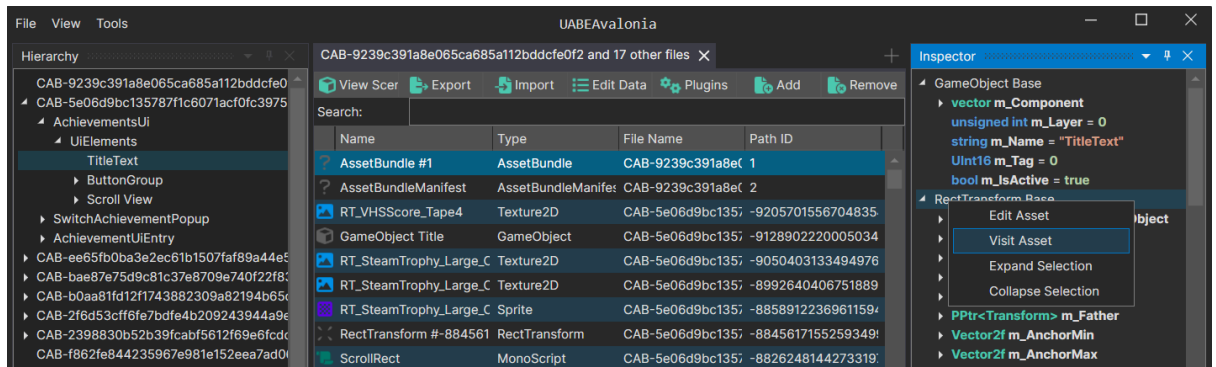


- В **UABEA** это сделано менее удобно: вам нужно открывать каждый бандл и затем окно Scene Hierarchy (F8), где нет автоматического разворачивания иерархии и отсутствует строка поиска. После нахождения компонента – ПКМ по нему → **Visit Asset**.



- **UABEANext**: добавить в **Workspace Explorer** бандлы и ассеты, с помощью сочетания **Ctrl+A** выделить все элементы

списка. перейти на вкладку **Hierarchy** и найти нужный объект по имени. Затем в инспекторе ПКМ по компоненту → **Visit Asset**.



- Примечание: вы можете перепроверить, что это действительно тот же самый компонент, что и в **UnityExplorer**, сравнив совпадение нескольких свойств, таких как размер шрифта.

TMPro_Translator – перевод строк TMPro

Это небольшой плагин для перевода **TextMeshProUGUI**. Он схож с *XUnity.AutoTranslator*, но гораздо проще в использовании.

Автор: Ambigram (на базе плагина *REPO_Translator*)

Совместимость: VerInEx 5.Mono / VerInEx 6.IL2Cpp

Скачать:  [TMPro_Translator](#)

История версий:

- v1 — начальная версия. Сохранение переведённых строк в XML файл.
- v2 — переход на формат JSON для хранения текста. При обнаружении файла в старом формате (XML), он автоматически конвертируется в новый формат (JSON).
- v3 – добавлена поддержка legacy Text (можно включить в настройках), умное сохранение: дозапись новой строки в файл вместо полной перезаписи, синхронизация базы с файлом.

Выбор подходящей версии VerInEx:

- Если в папке игры есть файл **GameAssembly.dll** — это IL2CPP-игра. Используйте **BepInEx 6** (архив с именем **BepInEx-Unity.IL2CPP-win...**).

Игра стала крашиться? Откройте логи в папке BepInEx. Наличие следующего текста означает, что ваша игра не поддерживается:

```
[Error :InteropManager] Failed to generate Il2Cpp
interop assemblies:
Cpp2IL.Core.Exceptions.LibCpp2ILInitializationException
: Fatal Exception initializing LibCpp2IL!
---> System.FormatException: Unsupported metadata
version found! We support 23-29, got 31
```

- Если **GameAssembly.dll** отсутствует — используйте **BepInEx 5** (архив **BepInEx_win_x64...**).

Установка:

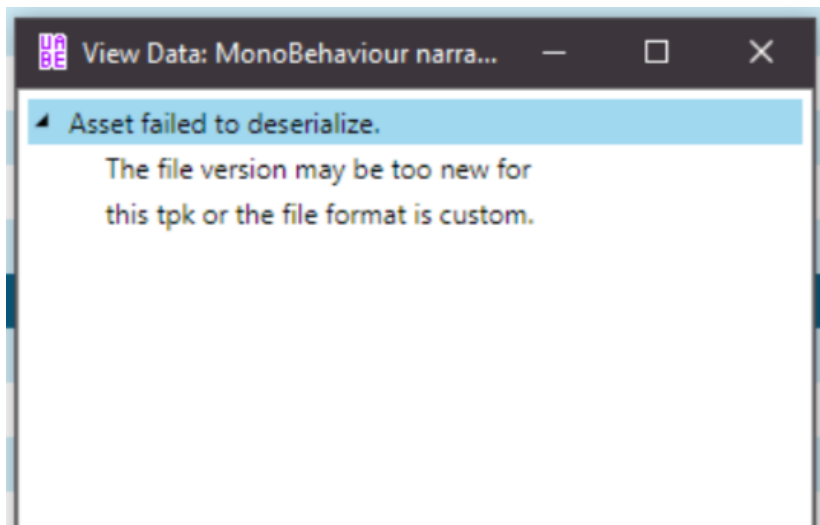
- Распакуйте содержимое архива **BepInEx** в корневую папку игры — туда, где находится исполняемый файл (**.exe**).
- Скопируйте плагин **TMPro_Translator.dll** в **BepInEx\plugins** (если папки нет, её нужно создать).
- Запустите игру. По пути **BepInEx\config** создастся файл конфигурации **com.github.qert2002.REPO_Translator.cfg**, в нем надо изменить *devMode* на *true* и перезапустить игру – это активизирует сбор текста в json по пути **BepInEx\plugin**.
- ⚠ Пока игра открыта, не переводите этот файл, потому что он перезаписывается.
- Закройте игру, переведите файл с текстом (а именно то, что написано после *"translate="*). При следующем запуске игры текст будет заменяться на перевод.
- В самом конце (перед публикацией русификатора), желательно отключить *devMode*.

Примечания:

- К сожалению это не сработает для динамически меняющихся строк, а также потребует сыграть в игру для сбора текстовой базы.

😊 Работа с форматами и решение проблем

UABEA – решение ошибки “Asset failed to deserialize”



О чем говорит эта ошибка?

- Если вы ранее использовали **IL2CppDumper**, переименовали папку на **Managed** и поместили в **Data** папку игры, отключите опцию *IL2Cpp* в **UABEA** и попробуйте снова: *Options* → *Enable/Disable Cpp2Il* → Должно появиться окно с текстом "Use Cpp2Il is set to: false".
- Возможно игра использует кастомную версию Unity, и поэтому **UABEA** не может прочитать данные ассета. Обычно в таком случае фанаты создают отдельные версии моддинговых программ, специально заточенные под конкретную игру и их внутреннюю структуру.
- Возможно версия файла не поддерживается данным tpk. Трк файл содержит структуры всевозможных стандартных ассетов юнити начиная с версии Unity 5 и используется для чтения тех ассетов, где структура в виде дерева типов не встроена.

Поэтому есть вероятность, что имеющийся tpk файл просто не содержит структур для данной версии юнити, особенно если он давно не обновлялся.

Как обновить Трк:

- Скачать последнюю версию tpk:
https://nightly.link/AssetRipper/Tpk/workflows/type_tree_tpk/master/uncompressed_file.zip
- Переименовать на "*classdata.tpk*", заменить оригинальный файл в корневой папке **UABEA**.

Решение ошибок "Stripped Unity version" и "version 0.0.0"

Некорректная шапка у бандлов вида **unityfs 5.x.x.0.0.0**

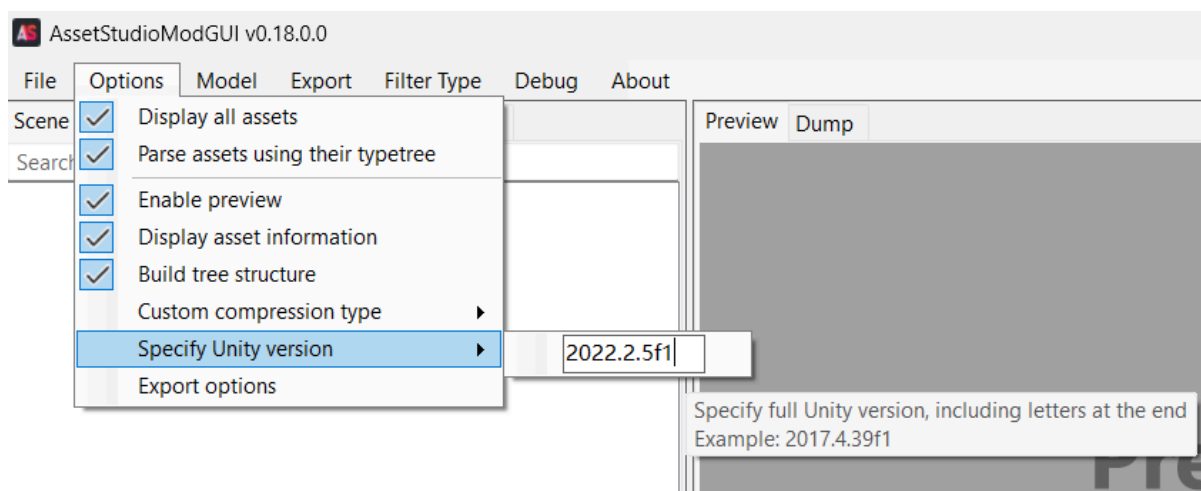
Чтобы такие прочесть нужно вручную указывать в программах моддинга корректную версию юнити.

Как это сделать в патчере:

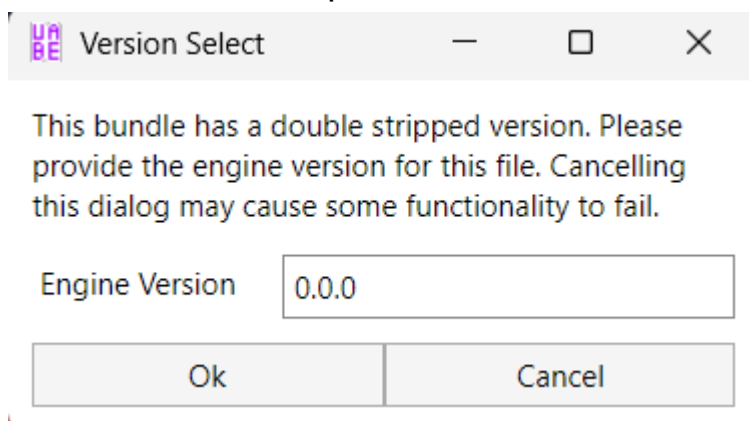
1. Откройте свойства исполнительного файла вашей игры и перейдите на вкладку **Подробности**, там вы и увидите версию движка в поле **Версия продукта**. Например: *2022.2.5f1*.
2. Пропишите дополнительную опцию с указанием версии:
--fallback_version "2022.2.5f1"
3. Попробуйте запустить команду и посмотрите, нет ли ошибок.

Также я находил [пайтон скрипт](#) для поиска версии при помощи парсинга **.assets** файлов, но особо не вникал, как он работает.

Как указать версию движка в программе **AssetStudioGUI**:



Окно для ввода версии в **UABEA**:



Также эту проблему можно исправить [пайтон скриптом](#):

- Предварительно установите модуль **UnityPy** командой `pip install UnityPy==1.10.18`
- Скрипт попросит ввести путь к игре и версию (формата `2020.3.47f1`) – вводить нужно без кавычек и пробелов.
- Скрипт прописывает в юнити файлах версию, чтобы избежать ошибок `"The unity files has been stripped"`.
- Результат будет в новой папке под названием `"InsertedVer_UnityFiles"`.
- К сожалению, вам придется делать это после каждого обновления игры, поэтому будет проще указывать версию ручную, как показано выше.

Другие способы, чтобы узнать версию:

[Как узнать версию движка, на которой создана игра](#)

Решение бесконечной загрузки и непрогружающихся ассетов (патчинг catalog.json)

У вас не загружается игра/загружается бесконечно, перестали работать кнопки либо не включаются диалоги, и тому подобные проблемы, когда что-то перестает работать без видимой причины после патчинга.

Если по пути `[имя]_Data\StreamingAssets\aa`, есть файл `catalog.json`, то именно из-за него блокируется дальнейшая загрузка игры, поскольку он содержит хэши оригинальных файлов. После подмены данных в юнити-архивах они соответственно уже будут другие. Решение – пропатчить `json`, обнулив хэши. Может попасться `catalog.bin` – это новый формат.

AdressablesTools (от nesrak1) – для catalog.json/bin:

1. Скачать утилиту [AdressablesTools](#).
2. В папку утилиты скопировать файл каталога.
3. Запустить команду:

```
Example patchcrc catalog.json
```

Либо

```
Example patchcrc catalog.bin
```

4. Файл должен обновиться, заменяем оригинальный файл каталога.
5. Проверяем, решилась ли проблема.

UnityCatalogPatcher (от dragonzh) – только для catalog.json:

1. Скачать [отсюда](#) утилиту.
2. Перетащить catalog.json на exe, либо воспользоваться командой:

```
UnityCatalogPatcher.exe путь/к/catalog.json
```

3. Файл должен обновиться, заменяем оригинальный файл каталога.
4. Проверяем, решилась ли проблема.

Принцип работы:

Программа находит тег `m_ExtraDataString`, распаковывает данные base64, затем все `m_Crc` обнуляются (либо если смотреть по инструкции dragonzh – числа заменяются пробелами). Потом всё запаковывается в обратном порядке. Так было в json каталоге.

Еще утилиты, которые могут помочь:

https://github.com/AXiX-official/Unity_CRC32_Bypass

P.S. Некоторые разработчики зачем-то кладут catalog в отдельный бандл 🧑

Теоретическая часть (написано ChatGPT):

Addressables – это система управления ресурсами (asset management system), предоставляемая Unity, которая позволяет эффективно управлять и загружать ресурсы в игре. Вместо того чтобы иметь множество отдельных файлов с ресурсами, Addressables позволяет упаковывать ресурсы в бандлы (bundles) и загружать их по требованию, когда они действительно нужны.

CRC (Cyclic Redundancy Check) – это метод контрольной суммы, используемый для проверки целостности данных. В контексте Addressables, CRC проверки применяются для обнаружения любых изменений в бандлах, которые могли возникнуть после их создания. Если бандл был изменен (например, модифицирован или поврежден), CRC проверка может сигнализировать об этом и предотвратить загрузку бандла.

Чтобы модифицировать или редактировать бандлы, созданные с использованием Addressables, необходимо выполнить очистку CRC проверок. Это означает удаление или обход проверок CRC, чтобы Unity не обнаруживала изменений и позволяла загрузку модифицированных бандлов.

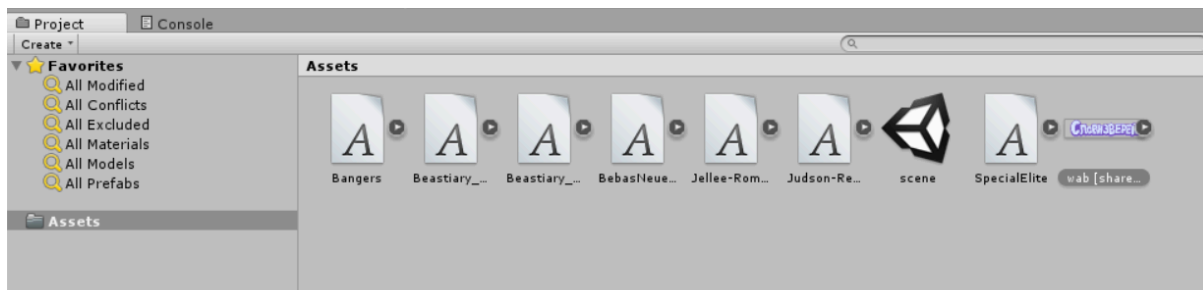
Исправление обрезанной текстуры

В Unity играх для оптимизации на спрайт накладывается mesh-сетка, которая ограничивает видимость текстуры по краям. Это исправимо, ниже смотрите пошаговое руководство.

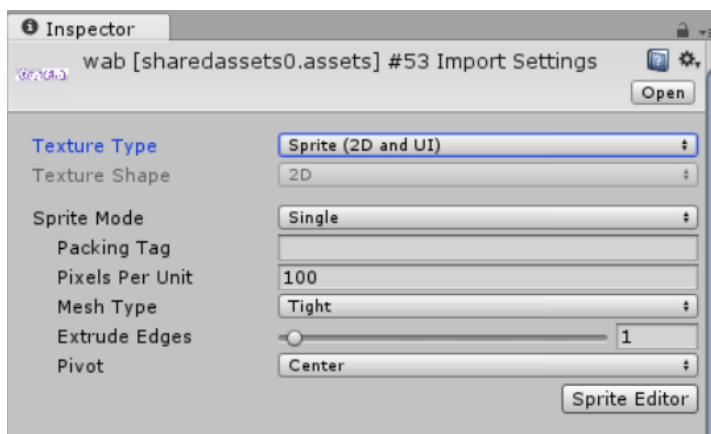


Способ №1. Через движок той же версии, что и игра ([Как узнать версию движка, на которой создана игра](#))

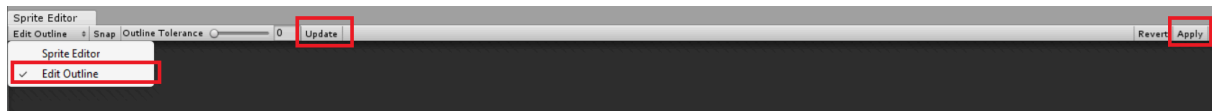
1. Добавьте текстуру в проект путем перетаскивания png файла на область с ассетами.



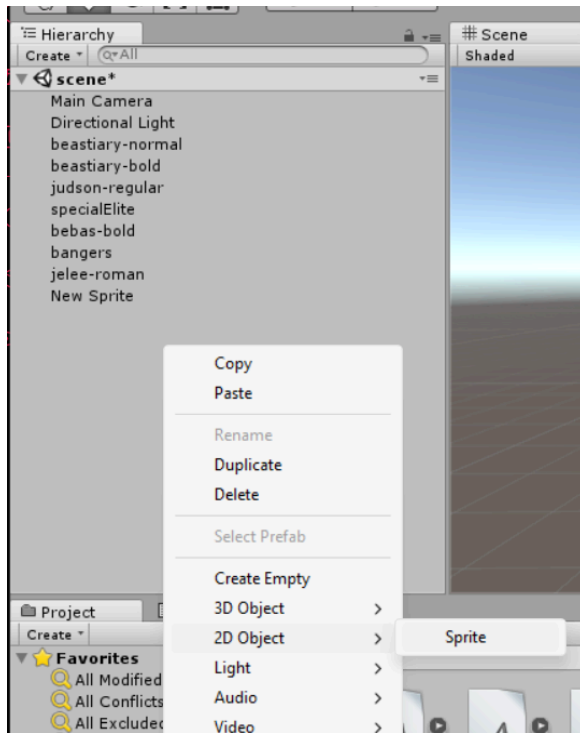
2. В инспекторе измените тип на **Sprite (2D and UI)**. Перейдите в **Sprite Editor**.



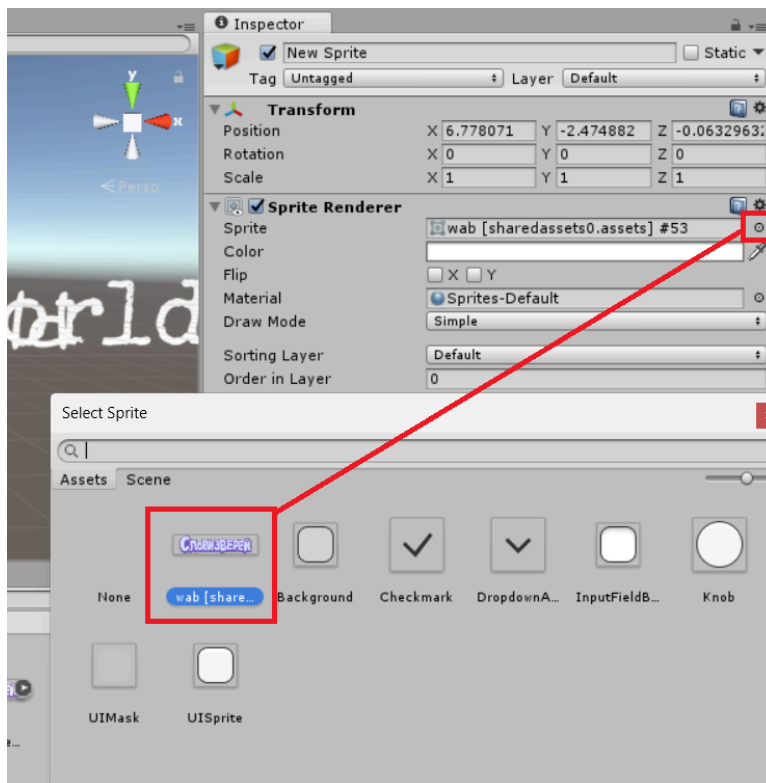
3. **Edit Outline** → **Update** → **Apply**.



4. Закройте окно. На сцене создайте новый 2D объект – **Sprite**, нажмите по появившейся строке **New Sprite** (можно переименовать).



5. В инспекторе, в строке напротив **Sprite** нажмите на значок шестеренки и выберите спрайт по двойному щелчку.



6. Соберите проект, нажав сочетание клавиш **Ctrl+B**. В первый раз движок запросит путь, куда сохранять сборку. Запомните его.
7. Перейдите в папку сборки. Запустите команду для извлечения дампа спрайта:

Patcher unpack -t Sprite -m dump

Затем найдите его по пути *Patcher_Assets/Sprite*. Имя должно начинаться как и у текстуры. Вам также надо извлечь оригинальный дамп спрайта. Для этого перейдите в папку игры и запустите вышеприведенную команду.

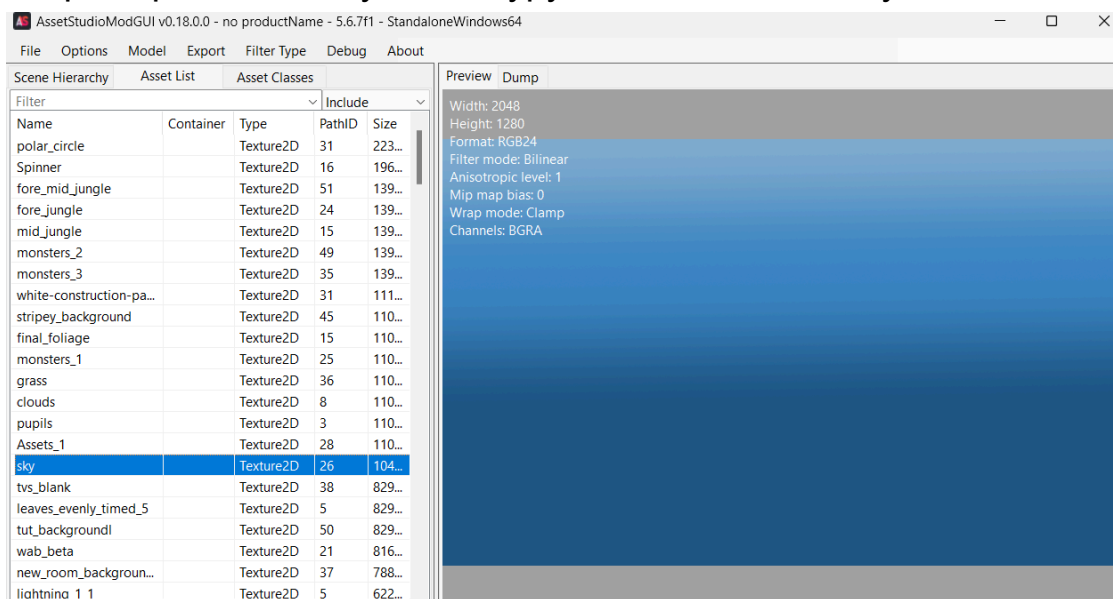
8. Адаптация спрайта. Откройте два дампа в удобном json редакторе. Из оригинального дампа перенесите в новый содержимое полей **m_Name** и **m_RD[texture]**. Новый дамп переименуйте под оригинал и можете паковать его.

P.S. Либо используйте **UABEA** для редактирования дампов:
Фильтр *Sprite* + кнопку *Export Dump / Import Dump*

Способ №2. Взять дамп спрайта другой текстуры

Это экспериментальный способ, он конечно может не сработать. Попробуйте, если вам не хочется качать движок.

1. Через **AssetStudioGUI** находим полноэкранную текстуру, которая не содержит прозрачных пикселей. Обычно в каждой игре такая имеется. Для облегчения поиска кликните столбец **Size** для сортировки по размеру от большего к меньшему. Я например нашел такую текстуру под названием sky.



2. Извлекаем патчером дампы спрайтов:
Patcher unpack -t Sprite -m dump
Ищем нужный дамп в папке с ассетами (в моем случае, **sky.dump.json**).
3. Открываем дамп спрайта, которая обрезается, и дамп-замену (**sky.dump.json**) в json редакторе. В дамп-замену переносим значения **m_Name** и **m_RD[texture]**. Возможно понадобится скопировать ещё какие-то поля. Переименовываем файл под оригинал, пакуем.

P.S. Либо используйте **UABEA** для редактирования дампов:
Фильтр *Sprite* + кнопку *Export Dump / Import Dump*

Почему я говорю взять именно спрайт полноразмерной текстуры: есть меши двух типов - тугая (tight) и полный квадрат (full rect). По умолчанию обычно используется тугая и именно она создаёт

проблемы, а вот для rgb текстур применяется полный квадрат, но тут важно брать от спрайта, который на весь экран, чтобы другой спрайт, который вы исправляете, отобразился весь.

Тutorial по поиску текста

1. Проверьте, нет ли текстовиков в папке **StreamingAssets**.
2. Извлеките текстовые ассеты одним из способов.
 - Через **UnityPatcher**:
`Patcher unpack --text`
 - Через **AssetStudioGUI**:
File → Load File / Load Folder
Filter Type → TextAsset
Export → Filtered Assets
 - Через **UABEA**:
File → Open
View → Filter → Deselect All → Поставить галочку рядом с TextAsset → Закрыть окно
Ctrl+A → Plugins → Export .txt
3. Вероятно, вы в них не найдёте весь игровой текст. Значит продолжаем поиск.

Способ №1. UnityPatcher + TotalCommander:

Создадим следующий текстовик под названием

`SearchPhrases.txt`, вписав свои поисковые фразы:

press enter

new game

go to safari

Пробиваем текст в ассетах командой `search`, попутно экспортируя их и составляя лог:

```
Patcher search SearchPhrases.txt --export --log
```

P.S. Для поиска с учетом регистра:

```
--case_sensitive
```

Также можно включить поиск по всем ассетам (помимо TextAsset и MonoBehaviour):

```
--entire_search
```

Если вы получаете массу ошибок, связанных с экспортом MonoBehaviour:

[Решение распространенных проблем](#)

По завершении работы программа выведет список уникальных имен скриптов из объектов MonoBehaviour, которые содержат указанный текст. Они также будут отображаться в именах извлеченных файлов после символа @.

Чтобы извлечь все объекты MonoBehaviour, укажите эти имена через пробел после опции `-c`:

```
Patcher unpack -c Text TextMeshProUGUI  
OtherClasses
```

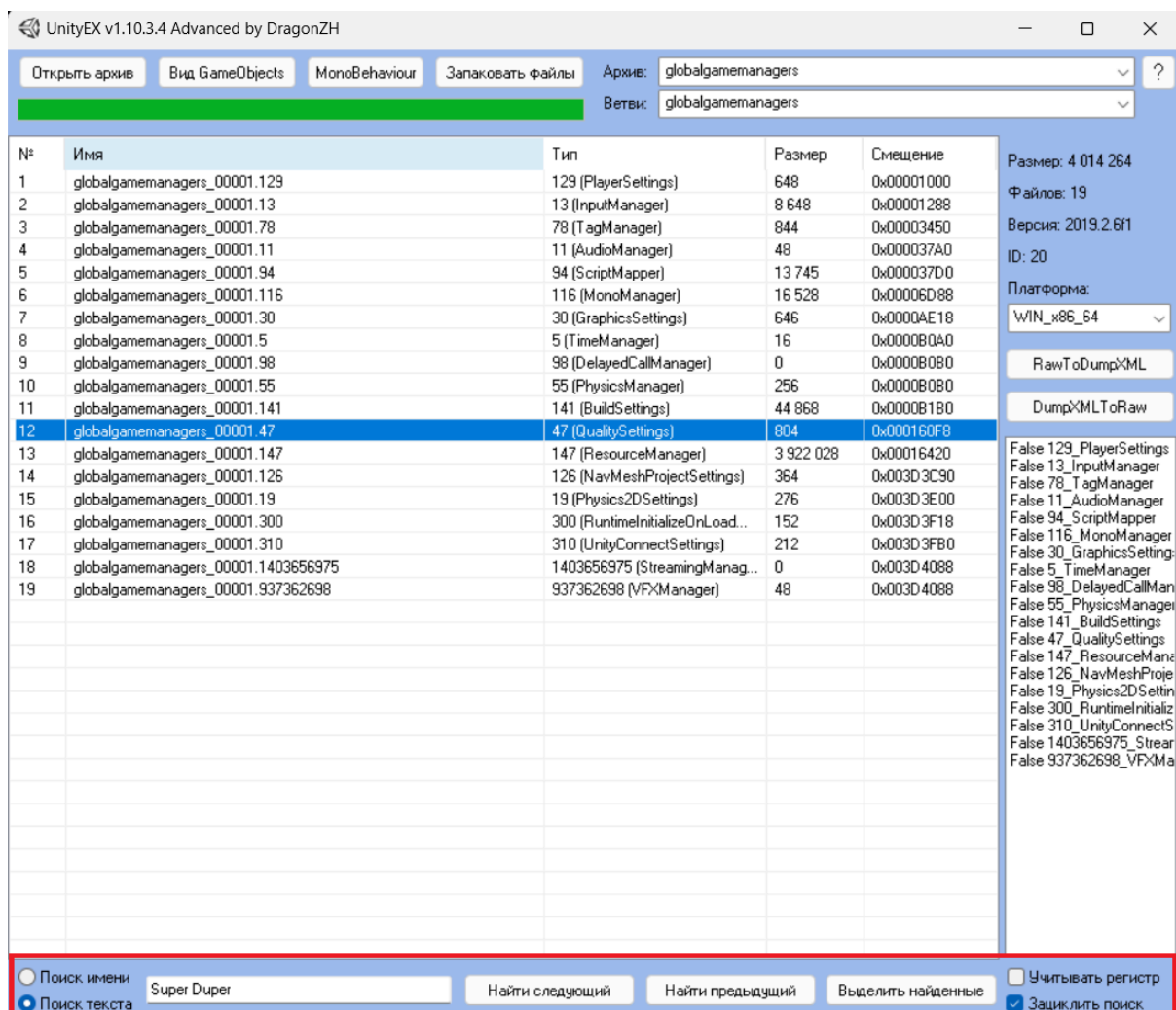
Вам понадобится дополнительно написать скрипт для извлечения и вставки текста в полученные json дампы, либо править их вручную.

Внимание: поиск командой `search` может быть неэффективным. Альтернативный способ – извлечь MonoBehaviour объекты как raw (`Patcher unpack --text --mb -m raw`) и выполнить поиск утилитой наподобие TotalCommander, где есть разные настройки для этого.

Способ №2. UnityEX + TotalCommander (оптимальный):

Пробить текст по **Data** папке утилитой **TotalCommander** ([гайд](#)), затем открыть найденный **.assets** файл с помощью **UnityEX** и воспользоваться функцией текстового поиска, чтобы выйти на объект, узнать его тип, имя, PathID (колонка PathID по умолчанию скрыта. Чтобы отобразить: ПКМ по таблице → поставить галочку на *PathID*). Для подтягивания нормальных имен у MonoBehaviour, нажмите соответствующую кнопку сверху.

Думаю, с бандлами двоичный поиск тотал командером не сработает, потому что они содержат сжатые юнити-архивы и для этого их нужно сперва распаковать (смотрите [тут](#) как быстро распаковать все бандлы при помощи консольного режима UABEA).



В красную рамку обведена область текстового поиска UnityEX.

По нажатию на кнопку “*Выделить найденные*” произойдет выделение всех найденных объектов. Затем можно их извлечь либо тут, либо в другой программе – зная PathID и имя родительского файла, найти объект не составит труда.

Способ №3. UABEA/AssetStudio + TotalCommander:

Действия схожи со вторым способом, но требует распаковки объектов.

1. Сперва пробить текст по **Data** папке утилитой **TotalCommander** ([гайд](#)). Например, он обнаружится в файле **resources.assets**.
2. Извлечь всё содержимое файла одним из способов.

- Через **AssetStudio**:
File → Load File
Export → Raw → All assets

- Через **UABEA**:
File → Open
Ctrl+A → Export Raw

3. При помощи **TotalCommander** открыть папку с извлеченными raw файлами и произвести поиск текста уже там. Процесс может затянуться, если файлов очень много.

4. Если всё равно не найдёте какой-то текст, попробуйте поискать его в папке **Managed** утилитой **TotalCommander** ([гайд](#)).

Отредактировать **.NET** сборки можно **dnSpy** или любой другой программой.

[DLL файлы: редактирование и перевод \(dnSpy\)](#)

5. Или при наличии в папке игры файла **global-metadata.dat** – в нем тоже может быть текст. Хотя, похоже, [данный файл может быть обфусцирован](#).

Программы для редактирования:

[Китайская утилита с графическим интерфейсом](#) (Github)

[Утилита от DragonZH консольная](#)

Полный список текстовых компонентов **MonoBehaviour** (взято из [данного архива](#)), правда, скорее всего не все они могут содержать текст:

Button, DialogueDatabase, Dropdown, TMP_InputField, InputField, LanguageSource, LanguageSourceAsset, LoadingTips, LocalizableText, LocalizationSO, LocalizedTextTable, LocalizeUITextMulti, StringTable, StringTableSO, Text, TextMesh, TextMeshPro, TextMeshProUGUI, TextTable, TMP_Dropdown, Toggle, UIStringData

Команда извлечения:

```
Patcher unpack -c Button DialogueDatabase Dropdown  
TMP_InputField InputField LanguageSource  
LanguageSourceAsset LoadingTips LocalizableText  
LocalizationSO LocalizedTextTable LocalizeUITextMulti  
StringTable StringTableSO Text TextMesh TextMeshPro  
TextMeshProUGUI TextTable TMP_Dropdown Toggle  
UIStringData
```

Если среди `MonoBehaviour` есть `TextMeshPro` или `TextMeshProUGUI` – значит используются SDF шрифты; если `TextMesh` или `Text` – значит Legacy Font (ttf/otf) (но не факт).

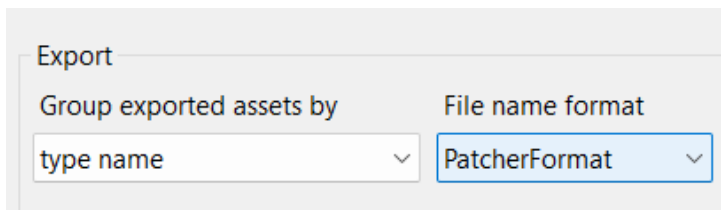
Совместимость патчера с AssetStudioGUI

Если вы ещё не в курсе, **AssetStudio** – лучшая программа для изучения юнити ассетов. В ней есть фильтры, область предпросмотра, и возможность экспорта.

Теперь вы можете извлекать файлы через **AssetStudio**, а паковать с помощью **Patcher**.

Предварительная настройка:

1. Скачать [AssetStudioMod v0.18.0](#).
2. Скачать [патч совместимости](#) и поместить в папку программы, с заменой оригинального файла.
3. Открыть **AssetStudioModGUI**.
4. Перейти в настройки (**Options** → **Export Options**).
5. В качестве формата имён файлов выбрать **PatcherFormat**.
Нажать **OK**.



Инструкция импорта:

Всё делаете как обычно, только добавляете опцию `--ignore_name`. Дело в том, что патчер проводит валидацию импортируемых файлов по разным данным, включая имя ассета. С помощью данной опции вы отключаете дополнительную валидацию, которая, в случае с **AssetStudio** может провалиться (т.к. разная обработка имен).

Также в указанной игровой папке не должно быть архивов из разных игр, это может привести к некорректному импорту.

Пример команды:

```
Patcher pack "PatchFolder" --ignore_name
```

Резервная ссылка:

[Готовая AssetStudioModGUI с патчем совместимости](#)

Дисклеймер:

Возможно, таким способом вы не сможете импортировать все ассеты, но базовые, типа картинки, текста, и тому подобные – да. Что касается дампов, похоже, только для типа `MonoBehaviour` они извлекаются корректно в **AssetStudio**.

Шифрованные бандлы (UnityCN)

В китайской версии Unity есть встроенная функция для шифрования бандлов и ассетбандлов.

Признаки шифрованных бандлов:

- Не грузятся в программах моддинга, таких как **UABEA**, **AssetStudio**...
- При открытии файла в hex-редакторе вы видите хаотичные данные и отсутствие стандартной шапки, начинающейся с *UnityFS*.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Текст декодирован
00000000	26	1D	1A	07	0A	35	20	73	73	73	73	7B	46	5D	0B	5D	5 ssss{F].]
00000010	0B	73	41	43	41	42	5D	40	5D	41	42	15	42	73	73	73	.sACAB]@]AB.Bsss
00000020	73	73	7A	F2	3B	39	73	73	26	60	73	73	A1	1F	73	73	sszt;9ss&`ssŸ.ss
00000030	71	30	73	73	73	73	73	73	73	73	73	73	73	73	73	73	q0ssssssssssssssss
00000040	6E	73	72	73	33	67	12	73	71	66	73	31	5C	AA	73	70	nsrs3g.sqfsl\Esp
00000050	79	73	57	7C	C7	79	73	57	62	6D	79	73	56	7A	0C	79	ysW SysWbmysVz.y
00000060	73	67	B0	79	73	57	75	CF	79	73	57	7A	BD	79	73	57	sg°ysWuPlyswZSysW
00000070	74	EF	79	73	57	7F	90	79	73	57	7C	C4	79	73	57	69	tnysW.b.ŷysW DysWi
00000080	2D	79	73	56	6B	2B	79	73	67	43	79	73	57	7E	6E	79	-ysVk+ysqCysW~ny
00000090	73	57	7C	92	79	73	57	7F	07	79	73	57	6A	5C	79	73	sW 'ysW..ysWj\ys
000000A0	57	5F	0B	79	73	57	4B	23	79	73	57	57	DC	79	73	57	W_.ysWK#ysWWbysW
000000B0	51	FC	79	73	57	68	E3	79	73	57	57	C6	79	73	57	52	QbysWhrysWWWysWR
000000C0	40	79	73	56	6C	FD	79	73	67	C1	79	73	66	68	C7	73	@ysVlaysgBysfhSs
000000D0	57	6E	CC	67	73	57	6D	F6	79	73	57	53	BF	79	73	57	WnMgsWmmyWSiysW
000000E0	5C	21	79	73	57	45	B8	79	73	57	46	77	79	73	57	4A	\!ysWE&ysWFwysWJ
000000F0	FB	79	73	57	44	28	79	73	57	59	C0	79	73	57	57	5A	mysWD(ysWYAysWWZ
00000100	79	73	57	55	53	79	73	57	5B	6B	79	73	66	42	6D	73	ysWUSysW[kysfBms
00000110	57	45	C8	67	73	57	45	83	79	73	57	5D	B6	79	73	57	WEIGsWEfysW ŷysW
00000120	5F	CE	79	73	57	5A	24	79	73	66	59	39	72	57	54	FF	OysWZ\$ysfY9rWTa

Пример зашифрованного бандла

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Текст декодирован
00000000	55	6E	69	74	79	46	53	00	00	00	00	06	35	2E	78	2E	UnityFS.....5.x.
00000010	78	00	32	30	31	38	2E	34	2E	31	33	66	31	00	00	00	x.2018.4.13fl...
00000020	00	00	05	F3	87	51	00	00	00	DB	00	00	00	DB	00	00	...y+Q...H...H..
00000030	00	40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.@.....
00000040	00	00	00	00	00	01	05	F3	86	44	05	F3	86	44	00	40	y+D.y+D.@
00000050	00	00	00	03	00	00	00	00	00	00	00	00	00	00	00	00	
00000060	02	09	81	9C	00	00	00	04	43	41	42	2D	61	32	62	35	..Гъ....CAB-a2b5
00000070	38	64	65	30	38	37	64	61	33	62	61	66	62	36	37	65	8de087da3bafb67e
00000080	62	35	31	62	30	35	37	66	61	39	65	65	00	00	00	00	b51b057fa9ee....
00000090	00	02	09	81	9C	00	00	00	00	03	71	38	A8	00	00	00	...Гъ.....q8E...
000000A0	00	43	41	42	2D	61	32	62	35	38	64	65	30	38	37	64	.CAB-a2b58de087d
000000B0	61	33	62	61	66	62	36	37	65	62	35	31	62	30	35	37	a3bafb67eb51b057
000000C0	66	61	39	65	65	2E	72	65	73	53	00	00	00	00	00	05	fa9ee.resS.....
000000D0	7A	BA	44	00	00	00	00	00	78	CC	00	00	00	00	00	43	zeD.....xM.....C
000000E0	41	42	2D	61	32	62	35	38	64	65	30	38	37	64	61	33	AB-a2b58de087da3
000000F0	62	61	66	62	36	37	65	62	35	31	62	30	35	37	66	61	bafb67eb51b057fa
00000100	39	65	65	2E	72	65	73	6F	75	72	63	65	00	00	04	29	9ee.resource..)
00000110	AD	02	09	81	9C	00	00	00	11	00	04	29	D0	00	00	00	...Гъ.....)P...
00000120	00	32	30	31	38	2E	34	2E	31	33	66	31	00	13	00	00	.2018.4.13fl....
00000130	00	01	49	00	00	00	73	00	00	00	00	FF	FF	F4	6E	8E	..I....s.....яфнГ
00000140	30	EC	C2	93	49	3F	18	AB	27	13	42	10	EE	23	00	00	0мВ"1?.«'.В.о#..
00000150	00	E7	00	00	00	05	00	00	00	15	01	00	80	37	00	00	.з.....Б7..
00000160	80	FF	FF	FF	FF	00	00	00	00	00	80	00	00	01	00	01	Бяяяя.....Ъ.....
00000170	00	48	03	00	80	AB	01	00	80	FF	FF	FF	FF	01	00	00	.Н..Ъ«.Бяяяя...
00000180	00	01	80	08	00	01	00	02	01	31	00	00	80	31	00	00	..Ъ.....1..Б1..
00000190	80	FF	FF	FF	FF	02	00	00	00	01	40	08	00	01	00	03	Бяяяя.....@.....

Пример нормального бандла

Если у вас бандл, как на первом скриншоте, стандартные средства моддинга тут уже не работают. Вам нужен ключ расшифровки и специальные программы.

Помощь в извлечении ключей/поиск уже извлеченных ключей:
<https://discord.gg/C6txv7M> – дискорд сервер **UnityPy**, раздел DataMining

Говорят, что ключ иногда можно найти даже в файлах игры или в коде. Если никак не получится, то попробовать путём перебора паролей (брутфорс) – нужные функции есть в **UnityPy**, либо погуглить ключ (их иногда публикуют для других моддеров). Также есть отдельная версия **AssetStudioGUI** специально для таких игр (**CNStudio**, или же более новая её версия – **Studio** от Razmoth). В остальном я пока больше не могу сказать.

<https://github.com/AXiX-official/UnityCN-Helper>
<https://github.com/RazTools/Studio>

Наглядный туториал по использованию **RazStudio**:
[AssetStudioHelp.md · GitHub](#)

Nintendo Switch игры – распаковка и патчинг

Вся информация в [отдельном документе](#).

Font: замена и исправление размера шрифта

После экспорта шрифтов, в папке **Font** вы увидите кое-где текстуры с буквами, кое-где векторные шрифты в формате ttf/otf. Это потому, что **Font** ассеты бывают двух типов:

- Растровые подобно **SDF** отображают текст с помощью атласа и разметки, где указаны координаты и размеры.
- Векторные берут символы из встроенного ttf шрифта, другими словами отображаются динамически. Однако разработчик может использовать этот подход, только если сам создал ttf или имеет лицензию на него. В противном случае ему приходится создавать растровый шрифт либо отключить встраивание шрифта, если предполагается, что он уже установлен у пользователя.

Способ замены №1. UnityPatcher:

Для замены просто пакуем ttf/otf, таким образом растровые шрифты будут переделаны патчером в векторные. Просмотрите папку Font, где находятся извлеченные файлы. Если там есть векторный шрифт – берем его имя, иначе – имя дампа, без расширения **.dump.json** (SomeFont [source] #11.dump.json → SomeFont [source] #11). Затем найдите в интернете новые ttf файлы, поддерживающие кириллицу, и присвойте им эти имена соответственно (SomeFont [source] #11.ttf). Запакуйте.

Как это работает:

Разметка с координатами удаляется, вставляется векторный шрифт в **m_FontData**, и устанавливается значение -2 в поле **m_ConvertCase** (обозначение динамического шрифта или вроде того).

Способ замены №2. UABEA:

Открыть юнити файл в программе. Найти интересующий Font объект. Нажать кнопку Plugins для экспорта или импорта ttf/otf.

Способ замены №3. Через движок (хотя я не вижу в этом смысла):

 [unity modding] How to modify ttf file without unityex ultimate

- В Unity проект закинуть векторный файл, в строке **Character** выбрать **Dynamic** – для динамического отображения, или любой другой тип – для создания растрового шрифта.
- Собрать проект, вытащить оттуда **Font**:

Patcher unpack -t Font

Переименовать файлы под оригиналы. Адаптировать дамп, перенеся из оригинала значения `m_DefaultMaterial` и `m_Texture`.

- Запаковать в файлы игры.

Но это ещё не всё. Когда вы зайдете в игру, могут возникнуть проблемы из-за неправильных размеров новых шрифтов. Через дамп, похоже, исправить не удастся, только путем скейлинга символов в векторном редакторе.

Исправление через **FontCreator**:

- Откройте **Transform Wizard**
- **Glyphs** → **All**
- **Outlines** → **Scale** (кликнуть дважды, чтобы добавилось в окошко **Script**)
- Внизу отрегулировать скейл, указав одинаковый процент по горизонтали и вертикали. Например, уменьшение в два раза – 50%.
- Нажать **OK**. Экспортировать шрифт.

Исправление через **FontStudio5** (спасибо DragonZH за инструкцию):

- **Tools** → **Action Set** → **Contour** → **Scale**

DLL файлы: редактирование и перевод (dnSpy)

Здесь я расскажу вкратце как вручную переводить библиотеки через **dnSpy**, а также как вносить фиксы. Относится только к играм, где есть папка **Managed** с библиотеками (но не та, которая получена программой **IL2CppDumper!**).

Ссылки на утилиту:

- [dnSpy](#) – оригинальная версия, заброшенная
- [dnSpyEx](#) – форк **dnSpy**, активно разрабатывается

Примечание: также исходный код можно изучить при помощи **ILSpy**, но он не позволяет его редактировать.

Внимание: после обновлений игры вам придется вручную переносить свои правки в новую библиотеку. Поэтому прежде всего подумайте, нужно ли вам это. Возможно вы захотите внести исправления с помощью **BepInEx** (или другого загрузчика модов), что будет более гибким и продвинутым вариантом, но требует навыков программирования. Либо использовать готовые плагины для перехвата и подмены строк, которые заносятся в объекты **TextMeshPro**:

[XUnity.AutoTranslator – автоперевод юнити игр](#)

[TMPro_Translator – перевод строк TMPro](#)

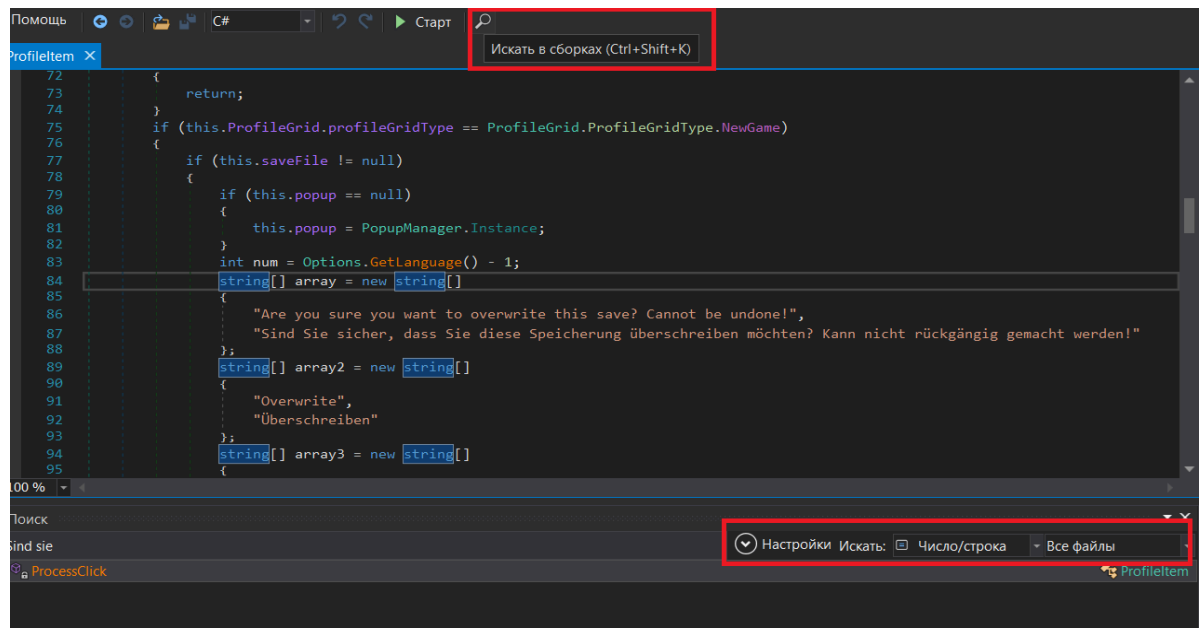
Однако если в игре много захордкоженного текста, вам придется делать собственный плагин для их подмены.

Также учтите, что плагины нельзя будет внедрить в консольные и мобильные версии игр, если вам потребуется портировать свой перевод.

Пошаговое руководство:

- Открыть библиотеку в **dnSpy**.
- В **обозревателе сборок** щелкнуть на библиотеку.
- Нажать на кнопку лупы, указать параметры поиска:
Число/строка, Выбранные файлы (или **Все файлы**).

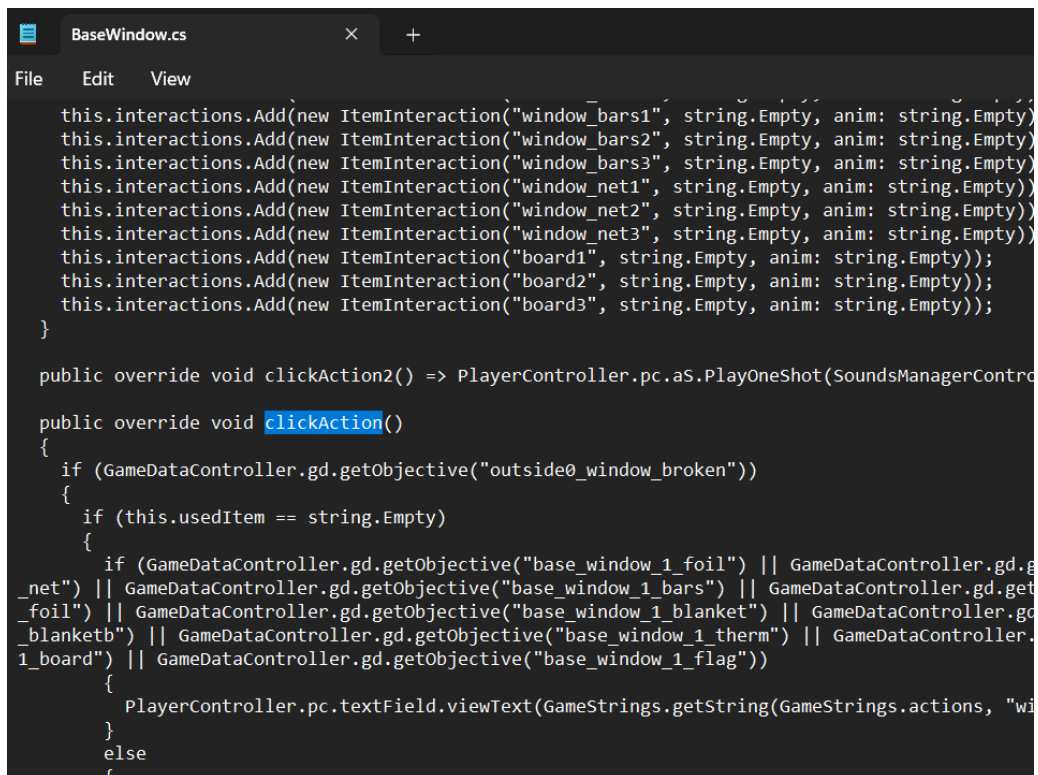
- Вписать часть строки в поисковое поле. Затем в выдаче если что-то найдётся, вы сможете перейти к этому участку кода.



- Воспользуйтесь сочетанием **Ctrl+F** для быстрого перехода к строке. ПКМ по строке → **Изменить IL инструкции**. Переведите текст, нажмите **OK**.
- После всех правок надо сохранить изменения: **Файл** → **Сохранить модуль**.

Ещё один способ перевода:

- Зачастую весь игровой код хранится в файле **Assembly-CSharp.dll**, и нужный текст, скорее всего, будет там. Чтобы убедиться, попробуйте сначала найти текст в папке **Managed** с помощью **TotalCommander**, в случае обнаружения он вам выдаст библиотеку. Для поиска конкретного класса и метода декомпилируйте эту библиотеку с помощью **JetBrains dotPeek**, откройте папку с декомпилированным кодом в удобном редакторе и воспользуйтесь текстовым поиском по всем файлам (пример редакторов: Notepad++, SublimeEditor, Atom editor, Visual Studio Code). В результатах будет файл с расширением **.cs**. Это декомпилированный класс, имя которого вам нужно запомнить.



```
BaseWindow.cs
File Edit View

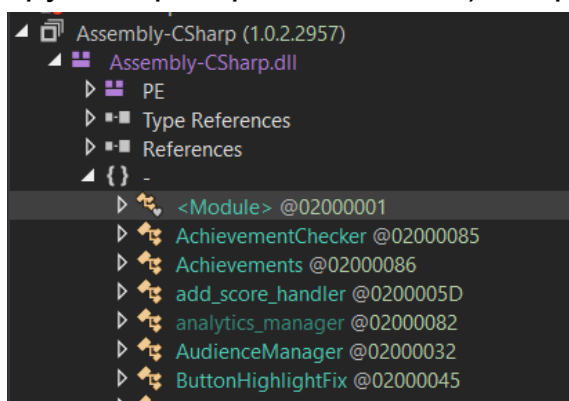
this.interactions.Add(new ItemInteraction("window_bars1", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("window_bars2", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("window_bars3", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("window_net1", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("window_net2", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("window_net3", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("board1", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("board2", string.Empty, anim: string.Empty));
this.interactions.Add(new ItemInteraction("board3", string.Empty, anim: string.Empty));
}

public override void clickAction2() => PlayerController.pc.aS.PlayOneShot(SoundsManagerContro

public override void clickAction()
{
    if (GameDataController.gd.GetObjective("outside0_window_broken"))
    {
        if (this.usedItem == string.Empty)
        {
            if (GameDataController.gd.GetObjective("base_window_1_foil") || GameDataController.gd.g
_net") || GameDataController.gd.GetObjective("base_window_1_bars") || GameDataController.gd.get
_foil") || GameDataController.gd.GetObjective("base_window_1_blanket") || GameDataController.gd
_blanketb") || GameDataController.gd.GetObjective("base_window_1_therm") || GameDataController.
1_board") || GameDataController.gd.GetObjective("base_window_1_flag"))
            {
                PlayerController.pc.textField.viewText(GameStrings.getString(GameStrings.actions, "wi
            }
        }
    }
    else
    {

```

- Теперь, когда вы знаете точное расположение текста в библиотеке, откройте её через **dnSpy**. Раскройте ветки как показано на скриншоте ниже (хотя класс может быть и в другом пространстве имён), откройте нужный класс.



- Воспользуйтесь сочетанием **Ctrl+F** для перехода к строке. ПКМ по строке → **Изменить IL инструкции**. Переведите текст, нажмите **ОК**.
- После всех правок надо сохранить изменения: **Файл** → **Сохранить модуль**.

Как отредактировать участок кода:

- Щёлкните ПКМ по коду → **Изменить метод**.

- Перед вами откроется окно редактирования декомпилированного IL кода в C#.
- Но не торопитесь, не тратьте своё время впустую! Само собой обратная рекомпиляция такого кода в IL может провалиться, поэтому всякий раз перед редактированием нового участка кода нажимаете **Компилировать** (не внося никаких исправлений). Если всё прошло без ошибок – вам повезло, вы можете спокойно вносить исправления на языке C#, но иначе придётся повозиться, например, с IL кодом. **BepInEx** и другие похожие инструменты вам также могут помочь с моддингом кода.
- Но ошибки могут появиться и в тех случаях, когда программа не может найти зависимые библиотеки. Надо в **Обозревателе сборок** убрать все сборки и добавить вручную те, что были выделены в коде красным цветом в *using* строках. Если библиотека лежит отдельно от остальных файлов, то более надежным будет перекинуть её в папку со всеми зависимостями, чтобы **dnSpy** не подтягивала автоматически левые библиотеки из системного диска (в этом случае у вас будет ошибка вида “добавлено несколько библиотек с одинаковыми сигнатурами, удалите ненужные”, а удалить их невозможно, потому что после удаления они сами добавляются в **Обозреватель сборки**).

Дампинг MonoBehaviour

Для операций, связанных с **MonoBehaviour**, обязательно наличие в папке игры файлов **globalgamemangers** и папки **Managed**.

Отсутствие **Managed** означает, что вам не повезло – разработчик решил усложнить доступ к исходному коду. Однако, это можно обойти при помощи утилиты [IL2CppDumper](#), если в папке игры лежат файлы **global-metadata.dat** и **GameAssembly.dll**:

- Скачайте **IL2CppDumper**, запустите исполняемый файл.

- Появится окно с запросом файла. Сперва выберите `GameAssembly.dll`, а при втором запросе – `global-metadata.dat`.
- Когда вы увидите в консоли сообщение про успех, рядом с утилитой должна появиться папка `DummyDll`, переименуйте её в `Managed` и переместите в игровую папку Data. Теперь проблем быть не должно.

Что делать если нет указанных файлов? Возможно понадобятся другие утилиты или даже проведение реверс-инжинеринга, но это не так просто.

Также [global-metadata.dat может быть обфусцирован](#).

P.S. Через **Il2CppDumper** можно получить *фиктивные* библиотеки. Они нужны лишь для чтения `MonoBehaviour` с целью изучения/экспорта/редактирования. Для моддинга кода это не подойдёт. В il2Cpp играх используют инжекторы с целью внедрения нового кода в работающий процесс игры, что несколько сложнее, чем редактирование готовых библиотек, но зато на эту тему есть множество обучающих видео.

Известные мне **TypeTree** генераторы, которые вы можете использовать при написании своих программ на основе **UnityPy** и **AssetsTools.NET**:

- [Unity Type Tree Generator](#) от AhmedAhmedEG
- [typetree_unity](#) от jrobinson3k1
- Также начиная с **UnityPy 2** появился встроенный API для генерации дерева типов ([пример использования](#))

Пример пайтон кода на основе UnityPy 1.10, демонстрирующий использование TypeTree файла:

```
@staticmethod
def get_typetree(obj, script, game_folder: str, get_nodes=False):
    assembly_name, namespace, class_name = script.m_AssemblyName,
    script.m_Namespace, script.m_ClassName
    class_path = f"{namespace}.{class_name}" if namespace else
    class_name
```



```

nodes = TypeTreeManager.typetree_cache.get(assembly_name,
{}).get(class_path)

if not nodes:
    assembly_folder = TypeTreeManager.assembly_folder or
find_managed_folder(game_folder)
    if not assembly_folder:
        logging.warning("[WARN] Typetree was not generated: Managed
folder not found in game directory.")
        return None

# Using the TypeTree Generator
data:json = generate_typetree(
    assembly_folder=assembly_folder,
    unity_version=".".join(map(str, obj.version)),
    libraries=assembly_name.replace(".dll", ""),
    class_names=class_path,
    disable_output=True,
)

if data and class_path in data:
    TypeTreeManager.typetree_cache.setdefault(assembly_name,
{}).update(data)
    nodes = data[class_path]

if nodes:
    return nodes if get_nodes else obj.read_typetree(nodes)

logging.error("Nodes not found: <%s>", class_path)
return None

```

Объяснение кода:

Этот метод пытается получить дерево типов (TypeTree) для указанного MonoBehaviour объекта (`obj`), используя информацию из прикрепленного к нему скрипта (`script`). Сначала он извлекает название сборки (`assembly_name`), пространство имен (`namespace`) и имя класса (`class_name`), затем формирует строку `class_path`, которая служит ключом для поиска данных в кэше `TypeTreeManager`. Если нужное дерево типов уже есть в кэше, оно сразу используется. Если данных нет, метод пытается найти папку `Managed` в `game_folder`, так как в ней хранятся библиотеки игры.

Если папка не найдена, выводится предупреждение, и метод завершает работу. Если папка найдена, вызывается `generate_typedtree`, которая пытается создать дерево типов, используя переданные параметры: путь к `Managed`, версию Unity, имя библиотеки (без расширения `.dll`) и имя класса. Если генерация успешна, полученные данные добавляются в кэш `TypeTreeManager`, после чего метод снова пытается извлечь нужные узлы. В итоге, если дерево типов найдено, метод возвращает либо сами узлы (если `get_nodes=True`), либо обработанные данные через `obj.read_typedtree(nodes)`. Если же данные так и не были найдены, выводится сообщение об ошибке, и метод возвращает `None`.

Теоретическая часть

`MonoBehaviour` — это основной класс для создания скриптов в Unity, но его бинарное представление в `.assets` и `.bundle` файлах довольно специфичное. Дампинг `MonoBehaviour` означает извлечение этих данных в читаемый формат, например JSON или TXT.

Unity использует **TypeTree** (дерево типов) для сериализации ассетов. Другими словами, это описание структуры объекта. Иногда она сразу встраивается в объекты, и тогда их можно прочитать моддинговыми утилитами без каких-либо проблем, иначе для чтения понадобится файл дерева типов, сгенерированный из соответствующей библиотеки папки `Managed`. Например **AssetStudio** делает это автоматически при помощи собственного **TypeTree** генератора после запроса папки **Managed**.

[Вот тут](#) немного подробнее написано про **TypeTree** и его структуру.

Дампинг объектов другого типа (не MonoBehaviour)

Эта информация больше для разработчиков. Обычно `typedtree nodes` вложены в такие объекты, поэтому лишних действий не

потребуется. В противном случае ветки можно извлечь [отсюда](#), данный репозиторий содержит все структуры стандартных типов юнити объектов, начиная с самых ранних версий движка. Его автор даже разработал формат [Trk](#) – это сжатый файл со всеми структурами, правда инструкции по использованию я не нашел. В UABEA автор разработал [свой формат Trk](#).

Титры

Дата создания документа: 16 октября 2024

Автор: Ambigram (@ambigram)

Обратная связь: [Discord сервер TDoT](#)

Залетайте к нам на форум переводчиков!

Если это руководство оказалось для вас полезным, вы можете поддержать мой труд небольшим донатом. Ваша поддержка вдохновляет и помогает создавать ещё больше полезных материалов. Спасибо!

Украина (ПУМБ): <https://mobile-app.pumb.ua/Xk15NyXS2HkqNSTP7>

P.S. Ещё больше инструкций по моддингу доступно [в теме UnityEX](#) на форуме ZOG.